République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Direction Générale des Etudes Technologiques
ISET de Rades
Département Technologies de l'Informatique

الجمهورية التونسية
وزارة التعليم العالي والبحث العلمي
الإدارة العامة للدراسات التكنولوجية
المعهد العالي للدراسات التكنولوجية برادس
قسـم تكنولوجيات الإعلامية

Final Year Project Report

Presented in partial fulfillment of the requirements for the degree of

NATIONAL BACHELOR'S DEGREE IN INFORMATION TECHNOLOGIES

Specialization: *Information Systems Development* ***(DSI)***

Development of Manufacturing Execution System (MES) application integrated with Dynamics 365 Business Central.

Company: MAVISION

Developed By:
SLEIMI Mohamed Dhia and BOUZIRI Mustapha

Supervised By:

Ms. CHAABANI Marwa : ISET Supervisor

Mr. YOUNSI Mourad : Company Supervisor

Code PFE *: DSI-26-R628* ***Academic Year*** *: 2025/2026*

Adresse : Rue El Quods, BP 172 - 2098 - Radès
Téléphone : 71 460 100 Fax : 71 442 322
Site Web : www.isetr.rnu.tn

العنوان: نهج القدس، ص.ب-172-2098- رادس
الهاتف: 71 460 100 الفاكس: 71 442 322
موقع الواب: www.isetr.rnu.tn

Abstract

Français:

Un Système d'Exécution de la Fabrication intégré à Microsoft Dynamics 365 Business Central, développé lors d'un stage de quatre mois chez MAVISION. Le système fait le lien entre les ateliers fonctionnant sur papier et une visibilité en temps réel grâce à quatre couches : une extension AL avec des endpoints OData V4, un proxy d'authentification Node.js, une application Flutter multiplateforme, et un agent IA Python FastAPI pour les requêtes en langage naturel. Les opérateurs et superviseurs gèrent la surveillance des machines, le suivi des nomenclatures, la lecture de codes-barres, le signalement des rebuts et l'impression d'étiquettes ; les administrateurs gèrent les utilisateurs, les codes-barres et un journal d'audit. Support trilingue (anglais, français, arabe) avec mise en page RTL automatique, livré en cinq sprints Scrum.

Mots-clés: MES, Système d'Exécution de la Fabrication, Business Central, Flutter, OData V4, Intégration ERP, Industrie 4.0, Suivi de Production, Scrum.

English:

A Manufacturing Execution System integrated with Microsoft Dynamics 365 Business Central, built during a four-month internship at MAVISION. The system bridges paper-based shop floors with real-time visibility through four layers: an AL extension with OData V4 endpoints, a Node.js authentication proxy, a cross-platform Flutter app, and a Python FastAPI AI agent for natural-language queries. Operators and supervisors handle machine monitoring, BOM tracking, barcode scanning, scrap reporting, and label printing; admins manage users, barcodes and an audit log. Trilingual support (English, French, Arabic) with automatic RTL layout, delivered across five Scrum sprints.

Keywords: MES, Manufacturing Execution System, Business Central, Flutter, OData V4, ERP Integration, Industry 4.0, Production Tracking, Scrum.

العربية:

نظام تنفيذ التصنيع مدمج مع Microsoft Dynamics 365 Business Central، تم تطويره خلال تدريب مدته أربعة أشهر في شركة MAVISION. يربط النظام بين بيئات العمل الورقية في المصانع والرؤية الفورية في الوقت الحقيقي من خلال أربع طبقات: امتداد AL مع نقاط نهاية OData V4، وبروكسي مصادقة Node.js، وتطبيق Flutter متعدد المنصات، ووكيل ذكاء اصطناعي بـ FastAPI Python للاستعلامات باللغة الطبيعية. يتولى المشغلون والمشرفون مراقبة الآلات وتتبع قوائم المواد ومسح الباركود والإبلاغ عن الهدر وطباعة الملصقات؛ فيما يدير المسؤولون المستخدمين والباركود وسجل المراجعة. يدعم النظام ثلاث لغات (الإنجليزية، الفرنسية، العربية) مع تخطيط تلقائي من اليمين إلى اليسار، وقد تم تسليمه عبر خمسة سبرينتات Scrum.

الكلمات المفتاحية: نظام تنفيذ التصنيع، Business Central، OData Flutter، V4، تكامل ERP، الصناعة 0.4، تتبع الإنتاج، Scrum.

Dedication

To my parents

Who have always encouraged me in my studies. Their unconditional love and their sacrifices have been a constant source of inspiration for me. I am grateful to have had their support throughout this journey.

To my family

For the efforts you have made to support me throughout my studies.

To my project partner

Your collaboration and invaluable support have been essential to our shared success.

To all my friends

Their sincere friendship, presence and support have made this experience even more enjoyable and meaningful.

SLEIMI Mohamed Dhia

To my family

For your love, patience, and constant support, which have been my greatest source of strength throughout this journey.

To my friends

For your encouragement, understanding, and motivation during this experience.

To my teammate

For your collaboration, effort, and commitment, which were essential to the success of this project.

BOUZIRI Mustapha

Acknowledgements

First and foremost, We would like to thank our academic supervisor, ***Ms.*** `CHAABANI Marwa`*, for their expert guidance and unwavering commitment. Their in-depth knowledge, availability and unconditional support have been of paramount importance in the completion of this project.*

We would also like to express our gratitude to our professional supervisor at ***MAVISION****,* ***Mr.*** `YOUNSI Mourad`*. Their contribution to this project, both on a technical and organisational level, has been invaluable. Their strategic vision, domain expertise and availability greatly facilitated our progress and allowed us to gain an enriching professional experience.*

Contents

List of Tables

List of Figures

List of Abbreviations

Abbreviation	*Definition*
AL	*Application Language (Microsoft Dynamics 365 Business Central)*
API	*Application Programming Interface*
AI	*Artificial Intelligence*
BOM	*Bill of Materials*
ERP	*Enterprise Resource Planning*
BI	*Business Intelligence*
CORS	*Cross-Origin Resource Sharing*
GUID	*Globally Unique Identifier*
HTTP	*Hypertext Transfer Protocol*
JSON	*JavaScript Object Notation*
KPI	*Key Performance Indicator*
LLM	*Large Language Model*
MES	*Manufacturing Execution System*
OData	*Open Data Protocol*
PDF	*Portable Document Format*
QR	*Quick Response (code)*
REST	*Representational State Transfer*
RTL	*Right-to-Left*
SaaS	*Software as a Service*
SCRUM	*(no acronym — Scrum is a proper noun)*
SHA-256	*Secure Hash Algorithm 256-bit*
SSPI	*Security Support Provider Interface*
TOTP	*Time-based One-Time Password*
TTL	*Time To Live*
URL	*Uniform Resource Locator*
UML	*Unified Modelling Language*
UTC	*Coordinated Universal Time*
XML	*Extensible Markup Language*
2FA	*Two-Factor Authentication*

General Introduction

In an industrial context undergoing rapid digital transformation, manufacturing companies face the necessity of optimising their production processes, reducing downtime and ensuring complete traceability of workshop operations. Manufacturing Execution System (MES), represent a direct response to these challenges by bridging the gap between decision-making layers Enterprise Resource Planning (ERP) and the production floor.

*It is within this context that the present end-of-studies project was carried out at **MAVISION**. The project consists of the design and development of an integrated MES system, structured around two main components: a business back-end developed in AL language on the Microsoft Dynamics 365 Business Central platform, and a cross-platform mobile/web application developed with the Flutter framework. The system enables machine management, production order tracking, real-time production reporting, as well as user and role-based access management.*

This report is structured into several chapters. The first chapter introduces the host organisation and presents the general framework of the project, highlighting the limitations of existing solutions and the relevance of the proposed solution. The second chapter focuses on requirements analysis and specification, as well as the technology choices made. The subsequent chapters trace the various development phases, exploring the conceptual aspects and illustrating the results obtained at the end of each sprint.

Through this approach, we aim to demonstrate how a well-designed digital solution can transform workshop production management by providing visibility, responsiveness and performance, serving both operators and management alike.

Chapter 1

Global Project Context

Contents

Introduction

This first chapter serves to contextualise our project. It begins with a brief introduction to the host organisation, then describes the objectives to be achieved, provides an in-depth analysis of the existing situation, and finally describes the methodology used to implement this work.

1.1 Presentation of the Host Company

To properly organise this section, we begin with some general details about MAVISION before addressing the activities directly relevant to us.

1.1.1 Overview of MAVISION

Founded in 2014 and headquartered in Manar II, Tunis, MAVISION [1] is a software development company specialising in consulting, auditing and implementing ERP information systems. MAVISION supports companies in their IT development projects and helps its clients better organise themselves around their information systems. In addition to its ERP offerings, MAVISION develops and integrates reporting solutions to help its clients manage their business activities more effectively. In order to build lasting trust, MAVISION places emphasis on its professionalism, commitment and the technical and functional expertise of its team.

Figure 1.1. MAVISION Logo [Source: MAVISION, https://www.mavision.tn, visited April 2025]

1.1.2 Areas of Expertise

MAVISION's activities are structured around three main areas:

ERP Integration: *Consulting, implementation and integration of ERP solutions, with a strong focus on Microsoft Dynamics 365 Business Central, as well as project and platform management, architecture and custom business software design.*

Business Intelligence: *Design and deployment of BI solutions enabling clients to gain strategic visibility over their operations through dashboards, reporting tools and data analysis.*

Web & Mobile Technology: *Development and integration of mobile phone applications and web platforms that complement ERP systems, enabling clients to manage their business activities with greater agility and efficiency.*

1.2 Project Context

This project forms part of our end-of-studies internship, an essential step for obtaining the `bachelor's` *degree at the* `Higher Institute of Technological Studies`*. This intern-*

ship, lasting four months, took place from February 2nd to May 30th at MAVISION. During this period, we were assigned to the development team.

This team is responsible for designing and maintaining software solutions that meet the company's business needs, leveraging modern technologies such as Microsoft Dynamics 365 Business Central and Flutter, and agile practices. Within this context, we worked on the development of the MES (Manufacturing Execution System) project, a mobile and web solution enabling intelligent management of machines and production operations on the workshop floor.

1.3 State of the Art

1.3.1 Subject Overview

With the rise of Industry 4.0 and the widespread adoption of ERP systems in manufacturing companies, the need to connect decision-making levels with floor operations has become critical. A MES (Manufacturing Execution System) is an information system that provides this link in real time: it supervises the execution of production orders, monitors machine status, records production declarations and provides operators and supervisors with a consolidated view of the workshop.

In this context, our project reflects a desire to provide MAVISION and its industrial clients with a MES solution natively integrated with Microsoft Dynamics 365 Business Central. The primary objective is to allow operators to start and monitor their production operations from an intuitive mobile/web application, while ensuring data consistency with the ERP.

1.3.2 Problem Statement

The problem of optimising production floor operations arises with particular force in manufacturing companies that rely on an ERP system such as Microsoft Dynamics 365 Business Central. Indeed, while Business Central effectively handles production planning and order management at the decision-making level, no unified, intelligent and dynamic solution exists to bridge the gap between the ERP and the shop floor in real time.

This gap leads to several consequences: inability to track the progress of production orders as they are being executed, manual entry of production declarations through paper routing sheets or spreadsheet files, lack of visibility for supervisors and managers on machine states and operation statuses, and insufficient traceability of actual shop-floor activity. Thus, a central question arises: how can MAVISION provide its industrial clients with a solution that optimises the management of production operations, ensuring effective, accurate and real-time synchronisation between the shop floor and the ERP system?

1.3.3 Study of the Existing Situation

Following an in-depth analysis, it is clear that there is a genuine need within manufacturing companies for a centralised solution to efficiently manage production operations directly from

the workshop floor.

Currently, production tracking is often carried out manually, via paper route sheets or spreadsheet files, leading to data entry errors, delays in information reporting and the absence of reliable traceability.

Supervisors and production managers have no real-time visibility into the progress of production orders, nor the current status of machines (running, paused). There is often no tool to automatically correlate production declarations with orders planned in the ERP.

1.3.4 Critical Analysis of the Existing Situation

In the context of this study, we briefly analysed the MES solution proposed by Delmia Apriso, one of the leading enterprise-grade Manufacturing Execution Systems on the market, whose interface is illustrated in the figure below.

Delmia Apriso is a comprehensive cloud-based MES platform developed by Dassault Systemes [2], designed for large-scale industrial enterprises. It covers the full spectrum of manufacturing operations including production execution, quality management, labour tracking and supply chain integration. While technically mature and feature-rich, it targets large multinational manufacturers with complex and highly customised deployments, resulting in significant licensing, implementation and maintenance costs that place it out of reach for small and medium-sized manufacturers.

Figure 1.2. Logo of the Delmia Apriso solution [Source: Dassault Systèmes, https://www.3ds.com/products/delmia/apriso, visited April 2025]

From MAVISION's perspective as a software provider, the goal is not to use an MES solution internally, but to deliver one to its industrial clients. The solution must therefore be cost-effective to sell, straightforward to deploy and maintain at client sites, and natively integrated into Microsoft Dynamics 365 Business Central, which is already MAVISION's core ERP offering. A heavyweight platform such as Delmia Apriso, which requires a separate infrastructure and lengthy implementation cycles, is incompatible with this commercial model. Table 1.1 summarises the key attributes of Delmia Apriso against the specific needs of the project.

Table 1.1. Comparison table with Delmia Apriso

Features	***Project needs***	***Delmia Apriso***
Real-time production order tracking	*Yes*	*Yes*
Machine status management	*Yes*	*Yes*
Production declaration from mobile	*Yes*	*Partial*
Native Business Central integration	*Yes*	*No*
Cross-platform mobile interface	*Yes*	*Partial*
Role management (Admin / Supervisor / Operator)	*Yes*	*Yes*
Secure token-based authentication	*Yes*	*Yes*
Adaptability to internal needs	*High (custom solution)*	*Low (proprietary platform)*
Ease of deployment at client sites	*High*	*Low*
Compatible with SME commercial model	*Yes*	*No*

1.3.5 Proposed Solution

The diagnosis of the current situation has highlighted several shortcomings and limitations, as detailed in the previous section. To address these, we propose to design and deploy an innovative MES solution natively integrated with Microsoft Dynamics 365 Business Central.

This solution consists of four complementary parts:

- ***An AL back-end (Business Central)**: developed in AL language, it exposes OData V4 endpoints enabling user management, token-based authentication, machine and operation tracking, as well as production declaration.*
- ***A Flutter application (front-end)**: a cross-platform mobile and web application allowing operators and supervisors to view production orders assigned to their machine, start, pause or complete an operation, declare produced quantities, and visualise progress in real time.*
- ***A Node.js middleware layer**: a reverse-proxy Express 5.x server handling SSPI/Windows authentication toward Business Central and exposing a unified REST API to both the Flutter client and the Python AI agent.*

- ***A Python AI agent (FastAPI)****: provides the conversational intelligence layer, intent classification, multi-provider LLM synthesis, fuzzy machine resolution, and contextual redirect actions.*

The objective is to optimise every phase of production management by establishing a reliable, traceable and ergonomic digital framework, serving both operators and management.

1.3.6 Objectives

Our MES solution aims to:

- *Provide real-time tracking of machine status and production operations directly from the Business Central ERP.*
- *Allow operators to start, pause and complete operations from an intuitive reactive interfaces.*
- *Record production declarations (produced quantities, scrap, scan) and synchronise them with production orders in Business Central.*
- *Manage users by roles (Administrator, Supervisor, Operator) with secure token-based authentication.*
- *Offer supervisors a consolidated view of production progress across their work centres.*
- *Reduce manual data entry and the risk of errors associated with paper-based or non-integrated processes.*
- *Provide an extensible architecture, ready to accommodate new features (alerts, direct machine connection, etc.).*

1.3.7 Adopted Methodology

The development method corresponds to an ordered and structured sequence of project phases, for efficient and optimal management.

Selection of the Agile Methodology

Agile is an iterative and collaborative working method [3] that takes into account the initial needs of the client and any changes that may arise. Its central ingredient is customer-centred iterative development, which allows a team to gather client feedback on a regular basis and make the necessary adjustments at the right time.

Thus, the client has a better overview of work progress, leading to regular sessions that guarantee the delivery of a functional application at all times throughout the improvement cycles. Consequently, the underlying key idea is to work faster in order to satisfy a client.

Adoption of the Scrum Framework

We adopted the Scrum framework [4], an agile approach based on short sprints and regular ceremonies. This choice allowed us to deliver functional increments at the end of each sprint, while maintaining smooth communication with the professional and our academic supervisors.

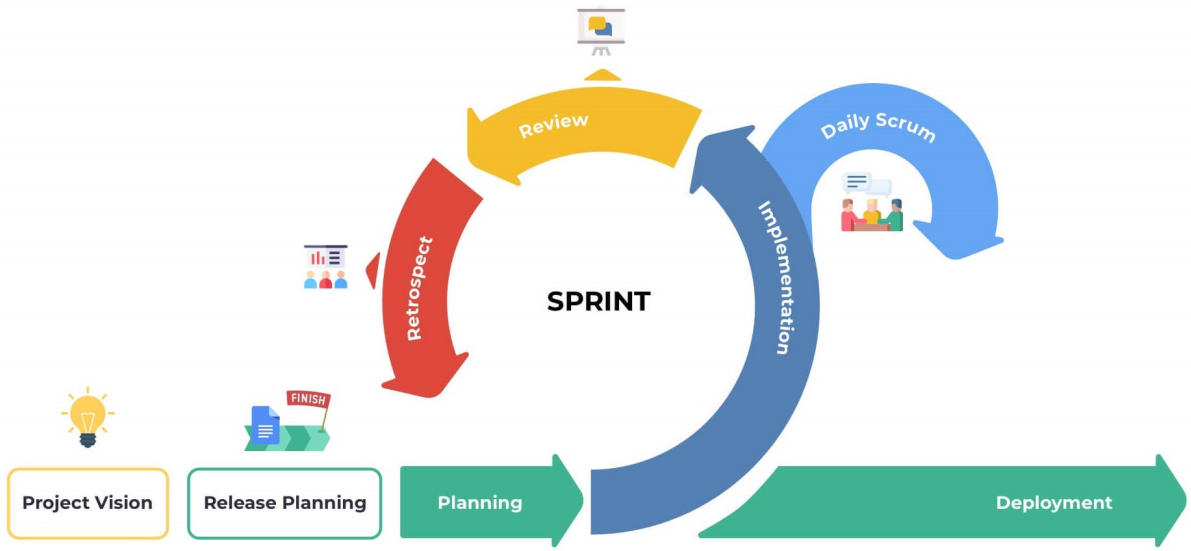

Figure 1.3. Adopted Scrum development cycle

Conclusion

In this chapter, we presented MAVISION, the company where the project is being carried out. We then defined the subject context, as well as the objectives of our work and the chosen methodology. In order to ensure the report progresses logically, we will examine the existing solutions used in the company. We will then explain the functional and non-functional requirements of our solution. This will allow us to evaluate existing solutions and identify the weaknesses and strengths for which we should make improvements to our solution. Finally, we will formulate the specific requirements for our solution.

Chapter 2

Requirements Analysis and Specification

Contents

Introduction

Requirements analysis and specification is a crucial phase in the information system development process. It consists, first and foremost, of clarifying the functional and non-functional requirements of the system. This step then allows a complete understanding of the subject in order to position oneself clearly, precisely and without ambiguity.

2.1 Requirements Analysis

This step allows us to understand the overall context of our project, that is to say to identify the main features and determine the actors who will interact with the system.

2.1.1 Identification of Actors

Actors are entities external to the system (people, hardware, systems) that are likely to exchange information with it. The following is a list of the actors who will interact with our system:

Table 2.1. List of MES application users

Actor	***Description***
Administrator	*This is the person responsible for the MES platform. They manage all users, passwords and account statuses, etc. They have access to all administration application dashboards.*
Supervisor	*An intermediate user who is a superset of the Operator role. In addition to their own access, they inherit all Operator permissions. They can monitor the progress of production operations on their work centre, view machine status, and track the progress of production orders. They can mark operations as finished, cancel, or interrupt unneeded operations, view machine dashboards, and declare on behalf of an Operator.*
Operator	*This is the end user of the MES application. Assigned to a work centre (Work Center), they can view the production orders assigned to their machine, start, pause or resume an operation, declare produced quantities, declare defective material, and scan components for consumption.*

2.1.2 Identification of Requirements

At this stage, we will define the specific requirements of our application, clearly distinguishing functional requirements from non-functional requirements.

Functional Requirements

Each actor of the platform has specific expectations referred to as functional requirements. To define the functional scope, it is necessary that the conceptual solution meets these functional

requirements.

- ***Authentication**: This feature allows any user to log in to the platform using their Auth ID and password. Upon first login, the user is prompted to change their temporary password. A secure session token (GUID, 12 h validity) is issued upon each successful login.*
- ***Application Setup**: On first launch, the user must configure the middleware host URL (entered manually or obtained by scanning a QR code) so that the application connects to the correct server instance. The host is persisted across sessions and can be changed from the login page.*
- ***User Management**: This feature allows the administrator to:*
  - *View the list of MES users.*
  - *Export the list of users.*
  - *Create a new user account by linking it to a Business Central employee and assigning them a role and work centre(s).*
  - *Generate and assign a temporary password.*
  - *Activate or deactivate an account (automatic revocation of active tokens).*
  - *Change a user's role (Operator, Supervisor, Administrator) and work centre assignment.*
  - *View, print, and regenerate a user's badge QR code for 2FA purposes.*
- ***Machine Management**: This feature allows users to:*
  - *View the list of machines in a work centre with their current status (Idle, Working).*
  - *View the production order currently running on each machine.*
- ***Production Order Management**: This feature allows the Operator and the supervisor to:*
  - *View production orders (Planned, Firm, Released) assigned to their machine.*
  - *Filter and sort orders by status or date.*
  - *View the details of an order (item, quantity, planned dates, operation description).*
- ***Production Operation Management**: This feature is the core of the system. It allows the operator or Supervisor to:*
  - *Start an operation (validation of prerequisites: released order, free machine, previous operation completed if applicable).*

  - *Pause or resume an ongoing operation.*
  - *Declare produced quantities (validation: quantity > 0, enough scanned component, no excess of the ordered quantity, etc).*
  - *View progress in real time (produced quantity, completion percentage, operation status, scrapped units).*
  - *View the Bill of Materials for the current operation with component consumption*
  - *Report scrapped units with a reason code.*
  - *scan components into the operation to record consumption against the BOM.*

- ***Progress Dashboard**: Allows the supervisor to view an overview of all machines information (uptime, operation completed, produced units, etc)*
- ***Password Change**: Allows any logged-in user to change their password by entering their old password and the new one.*
- ***AI Chat Assistant**: An embedded natural-language assistant allows operators and supervisors to query machine status, production progress, scrap data, BOM, and history without navigating the full interface. The assistant resolves ambiguous machine references, answers fleet-wide questions, surfaces operational anomalies, and provides contextual navigation shortcuts.*

Non-Functional Requirements

To define the non-functional scope, it is necessary that the conceptual solution meets these non-functional requirements:

- ***Performance**: The platform performs efficiently in terms of handling routine operations (login, order consultation, production declaration).*
- ***Security**: Implementation of a token-based authentication system (signed GUID, 12 h expiry) and role management (Admin, Supervisor, Operator) to control access to features. Passwords are stored as SHA-256 hashes with a salt.*
- ***Security Policy**: The administrator can configure a password rotation period (in days). A scheduled background job runs every 24 hours and flags users whose password age exceeds the configured period, forcing them to change their password on next login.*
- ***Two-Factor Authentication (2FA)**: When enabled globally by the administrator, users must complete a second authentication step by scanning their personal badge QR code after entering their credentials. Administrators can view, print, and regenerate badge secrets for any user.*

- ***Maintainability***: *The AL back-end code is organised in layers (Tables, Enums, Codeunits, API Pages) and the Flutter code follows a layered architecture (Data, Domain, Presentation) with the Provider pattern, facilitating updates and the addition of new features.*
- ***Ergonomics***: *The user interface is adaptive (mobile, tablet, PC) thanks to responsive layouts dedicated to each form factor, minimising the learning curve for workshop operators.*
- ***Portability***: *The Flutter application is compiled for Android, Web, Linux, macOS and Windows from a single codebase. The Business Central back-end is deployable both on-premises and in SaaS (cloud) mode.*

2.1.3 Global Use case diagram

the following Figure 2.1 dipects the global use case diagram of the MES system, illustrating the interactions between the various user roles and the system's functionalities.

Figure 2.1. MES System Global Use Case Diagram

2.2 Backlog Features

Domain 1: Authentication & User Management

ID	*User Story*	*Acceptance Criteria*	*Priority*
US-1.1	*As an Administrator, I need to assign a user account for each employee so that users only access features appropriate to their role.*	• *Ability to assign a user as Operator or Supervisor or Administrator.* • *Unauthorized actions are blocked with clear error messages.*	*High*
US-1.2	*As a User (Operator / Supervisor / Administrator), I need to log into and out of the system with my credentials so that I can access features appropriate to my role.*	• *User can enter Auth-ID and password.* • *System validates credentials against the MES.* • *Session token is created upon successful authentication.* • *User is redirected to the appropriate dashboard based on role.*	*High*
US-1.3	*As an Administrator, I need to search users.*	• *Ability to filter users based on their role.* • *Ability to search for a specific user.*	*High*
US-1.4	*As an Administrator, I need to see a list of all users with their details so that I can manage their accounts.*	• *The user list displays each account's information.* • *The list refreshes automatically after a new user is created.* • *Clicking on the action button shows a dropdown menu with available actions(change role, generate password, view badge, and toggle activation).*	*High*

Continued on next page

ID	*User Story*	*Acceptance Criteria*	*Priority*
US-1.5	*As an Administrator, I need to generate and assign temporary passwords to users so that they can log in for the first time.*	• *Admin can generate a random secure password for any user.* • *The generated password is sent to the MES via* `AdminSetPassword`. • *The target account is flagged* Need To Change Password *after the password is set.* • *Success or failure is reported to the admin with a clear message.*	*High*
US-1.6	*As an Administrator, I need to change the role and work-centre assignment of an existing user so that access rights stay current.*	• *Admin can select a new role for any other user.* • *Work-centre list updates based on the selected role.* • *All active tokens for the target user are revoked on save.* • *The user list refreshes after a successful change.*	*High*
US-1.7	*As an Administrator, I need to activate or deactivate a user account so that access can be revoked without deleting the account.*	• *Admin can toggle the active status of any other user.* • *Deactivation immediately revokes all active tokens.* • *The user row updates its visual status indicator immediately.*	*High*
US-1.8	*As a User, I need to complete a badge QR code scan after entering my credentials so that my identity is verified.*	• 2FA *prompt appears only when* TwoFA Enabled. • *Camera opens and scans a QR badge code.* • *Manual entry field available as a fallback.* • *Session token is persisted only after successful badge verification.* • *Failed scan shows an error; scanner reactivates.*	*High*

Continued on next page

ID	***User Story***	***Acceptance Criteria***	***Priority***
		• *User can cancel and return to the login page.*	
US-1.9	*As an Administrator, I need to view, print, and regenerate a user's badge QR code so that I can distribute it and invalidate a lost or compromised badge.*	• *Badge QR rendered from the user's* Badge Secret *via a barcode widget.* • *Print button generates a PDF.* • *Regenerate action shows a confirmation dialog before replacing the secret.* • *New QR is displayed immediately after regeneration.*	*High*
US-1.10	*As an Administrator, I need to globally enable or disable* 2FA *and configure a password rotation period so that I can control the organisation's authentication security level.*	• 2FA *toggle and password rotation period field available on a Settings page.* • *Changes take effect on the next login attempt.* • *Settings are persisted.* • *A scheduled job (every 24 h) flags users whose password age exceeds the configured period.* • *Unsaved changes display a persistent banner with Discard and Save actions.*	*High*
US-1.11	*As a user, I need to configure a host URL so that the application connects to my company's server.*	• *Paring page shown on first launch when no host is saved.* • *URL can be entered manually or obtained by scanning a QR code.* • *URL is validated (no spaces, no illegal characters).* • *Saved host persists across app restarts.* • *Change Host link on the login page allows re-entering the pairing screen.*	*High*

Continued on next page

ID	*User Story*	*Acceptance Criteria*	*Priority*
US-1.12	*As an Administrator, I need to export the list of users so that I can use the information in other applications or processes.*	• *The ability to export users.* • *The information is structured in the form of an excel file.*	*Medium*

Domain 2: Machine & Production Management

ID	*User Story*	*Acceptance Criteria*	*Priority*
US-2.1	*As an Operator or Supervisor, I need to see the list of machines in my work center with their current status so that I can see which machines are currently active.*	• *Machine list is scoped to the authenticated user's work centre.* • *Each card shows machine name, current status (*Idle*,* Working*), active production order number, operation No, item No, and the item name.* • *The list refreshes automatically every 20 seconds.*	
US-2.2	*As an Operator or Supervisor, I need to view the operations of the production orders assigned to a machine so that I know what work needs to be done.*	• *Only* Planned*,* Firm Planned*, or* Released *orders are shown.* • *Each card displays order number, product description, planned quantity, planned start and end dates, and operation description.*	*High*
US-2.3	*As an Operator or Supervisor, I need to start a production operation so that the system reflects the change.*	• *Only* operation *from* Released *orders may be started.* • *A machine cannot run two operations simultaneously.* • *The previous operation in the same production order (if any) must be* Finished*,* Cancelled*,* Interrupted*, or currently* Running *with enough send-ahead quantity.*	*High*

Continued on next page

ID	*User Story*	*Acceptance Criteria*	*Priority*
		• *A new MES Operation State record is created with status change so it can be tracked.*	
US-2.4	*As a Supervisor or Operator, I need to monitor the current status of all active operations on a machine so that I can follow the progress of each order.*	• *The Operations Progress tab shows all operations whose latest status record is* Running *or* Paused*.* • *Each card shows order number, operation number, current status, operation description, last updated timestamp, and progress.* • *Data refreshes automatically every 5 seconds.*	*High*
US-2.5	*As an Operator or Supervisor, I need a tabbed view for a machine so that I can navigate between pending orders, active operation progress and the operation history of the machine without leaving the machine context.*	• *Selecting a machine from the list opens a dedicated machine page.* • *The page provides three tabs:* Orders*,* Progress*, and* History*.* • *The active tab is visually highlighted in the navigation bar.*	*High*
US-2.6	*As an Operator or Supervisor, I need to filter and sort production orders so that I can quickly find the relevant work.*	• *Orders can be searched by order number, item description, or date.* • *Orders can be filtered by status (*All*,* Planned*,* Firm Planned*,* Released*).* • *Orders can be sorted by planned start date ascending or descending.* • *On mobile, the status filter is accessible via a compact popup menu.*	*High*

Continued on next page

ID	*User Story*	*Acceptance Criteria*	*Priority*
US-2.7	*As a Supervisor or Operator, I need to view the history of completed operations for a machine so that I can audit past production.*	• *The History tab shows all operations with status* Finished,Interrupted*, or* Cancelled*.* • *Each card shows order number, status, product name, produced quantity, start time, and end time.* • *The list supports free-text search and sort by start date.* • *Tapping a card navigates to the full operation detail page.*	*High*

Domain 3: Production Execution & Traceability

ID	*User Story*	*Acceptance Criteria*	*Priority*
US-3.1	*As an Operator or Supervisor, I need to declare the produced quantity in a cycle so that the system records my progress.*	• *Declared quantity must be positive and not exceed remaining quantity that need to be produced.* • *Declared quantity must be less than the amount that can be produced with the available components scanned remaining.* • *Each declaration creates a new progression cycle record.* • *Produced quantity and progress percentage update immediately.*	*High*
US-3.2	*As an Operator or Supervisor, I need to pause and resume an active operation so that interruptions are tracked accurately.*	• *A Running operation can be paused; a Paused one can be resumed.* • *A machine cannot resume if another operation is already running.*	*High*

Continued on next page

ID	***User Story***	***Acceptance Criteria***	***Priority***
US-3.3	*As a Supervisor, I need to finish, interrupt or cancel an operation so that the machine is released and the order is closed in the system.*	• *Finishing a complete order (100%) shows a confirmation dialog.* • *Interrupting or Cancelling an operation shows a warning dialog.* • *If the operation is started then it is marked as interrupted otherwise as cancelled* • *Closing sets the execution end time and inserts an Idle machine status.*	*High*
US-3.4	*As an Operator or Supervisor, I need a dedicated operation detail page with up-to-date data in one place.*	• *Shows status, order number, product, required and produced quantities.* • *The information updates automatically without a manual refresh.*	*High*
US-3.5	*As an Operator or Supervisor, I need to see all production cycles declared for an operation so that I can review each declaration's information.*	• *Cycle table lists operator, quantity, cumulative total, and timestamp.* • *Records ordered newest-first with pagination.* • *Updates after each new declaration.*	*High*
US-3.6	*As an Operator or Supervisor, I need a visual chart of declarations over time to identify performance trends.*	• *Line chart plots cycle quantity vs. declaration time.* • *Touch tooltip shows operator, quantity, total production, timestamp.* • *Auto-scrolls to most recent data point on load and refresh.*	*Medium*

Continued on next page

ID	***User Story***	***Acceptance Criteria***	***Priority***
US-3.7	*As an Operator or Supervisor, I need to see the Bill of Materials for a production operation so that I know which components are required, how much has already been consumed, and the amount available in the inventory.*	• *Components are colour-coded by remaining amount (the needed amount (green) / less then necessary to make a single item (red) otherwise (yellow)).* • *BOM data updates after every consumption event.* • *can be expanded to a dialog for a better view on larger screens.* • *search by name and filter by state*	*High*
US-3.8	*As an Operator or Supervisor, I need to report scrapped units with a reason code so that waste is accurately tracked.*	• *A reason code must be selected before submission.* • *Need to select either input material or output scrap.* • *the quantity of scrapped units cannot exceed the quantity that can be produced with the available components scanned remaining.* • *Each declaration's quantity, code, and operator is stored for tracibility.*	*High*
US-3.9	*As a Supervisor, I need to declare production or scrap on behalf of an operator so that output is attributed to the correct person.*	• *Supervisor role sees an operator selector in the declare dialog.* • *Selected operator's ID is stored with the declaration record for tracibility.* • *If no operator is selected, it is declared as if the supervisor made the declaration.*	*Medium*
US-3.10	*As an Operator or Supervisor, I need to scan components into an operation so that consumption is recorded.*	• *any camera connected to the device reads barcodes.* • *Duplicate scan increments quantity on the existing row.* • *Confirming scans inserts consumption records and refreshes the BOM.*	*High*

Continued on next page

ID	*User Story*	*Acceptance Criteria*	*Priority*
		• *After confirmation, the inventory in the ERP is updated to reflect the consumed quantity.*	
US-3.11	*As an Operator or Supervisor, I need to print a label for the finished production operations (100% progress) so that produced parts are correctly identified.*	• *Label includes DataMatrix barcode encoding order, machine, item data.* • *PDF generation supports landscape/portrait toggle.* • *Print button available only when operation is Finished or progress* $\geq$ *100%.*	*Medium*
US-3.12	*As an Operator or Supervisor, I need a label printed for each declared unit so that individual pieces carry traceable information.*	• *One PDF page per declared unit with a label counter.* • *Same DataMatrix encoding as the production label.* • *Launched automatically after a successful production declaration.*	*Medium*

Domain 4: Administration & Monitoring

ID	*User Story*	*Acceptance Criteria*	*Priority*
US-4.1	*As an Supervisor, I need a performance dashboard for all machines so that I can monitor production health across the facility.*	• *Dashboard shows uptime, operation cancelled or finished count, total produced, and scrap per machine.* • *Configurable time-range (hours back) filter.*	*Medium*
US-4.2	*As an Administrator, I need a filterable activity log of all system actions so that I can audit operator and machine activity.*	• *Log entries includes all the relevant fields with timestamps.* • *Filterable by action type and time range.* • *Searchable by user name.*	*High*

Continued on next page

ID	User Story	Acceptance Criteria	Priority
US-4.3	*As any user, I need the application available in English, French, and Arabic so that I can use it in my preferred language.*	• *Language selector on login screen and in profile page.* • *All UI strings externalised; no hard-coded text.* • *Arabic locale activates RTL layout automatically.* • *Language preference persisted across sessions in the same device.*	*Medium*
US-4.4	*As a first-time user, I need guided coach-mark tutorials so that I can learn the interface without external documentation.*	• *Tutorials shown exactly once per install per screen.* • *Coach marks cover: machine list, machine detail tabs, operation detail, and admin dashboard.* • *Localised skip button text.*	*Medium*
US-4.5	*As an Administrator, I need a searchable barcode page so that I have a source of the mes barcodes labels.*	• *the barcode page is searchable.* • *Ability to print out the barcode labels.* • *the ability to reload the barcode list.*	*Medium*

Domain 5: AI Chat Assistant & Analytical Intelligence

ID	User Story	Acceptance Criteria	Priority
US-5.1	*As an Operator or Supervisor, I need a natural-language chat assistant so that I can query shop floor status without navigating the full UI.*	• *Chat panel opens as a side overlay on wide screens and as a dialog on mobile.* • *Assistant answers questions about machines, orders, live operations, BOM, history, and scrap.* • *Responses are scoped to the user's own work centres (access control enforced).* • *Multi-turn conversation history is maintained within the session.*	

Continued on next page

ID	*User Story*	*Acceptance Criteria*	*Priority*
		• *Conversation can be cleared by the user.*	
US-5.2	*As an Operator or Supervisor, I need the assistant to answer fleet-wide questions so that I get a consolidated view without visiting each machine individually.*	• *Queries about all machines trigger a parallel fan-out across work centres.* • *Results are filtered by running / idle / overdue / scrap status as requested.* • *Fan-out is capped to control response time.* • *Fleet summary includes machine count, utilisation percentage, and active orders.*	*Medium*
US-5.3	*As an Operator or Supervisor, I need the assistant to provide contextual navigation shortcuts so that I can open a machine, operation, or dashboard directly from the chat response.*	• *Chat responses include up to 4 action buttons when a redirect is relevant.* • *Tapping an action navigates to the corresponding screen.* • *Action buttons appear only on the last assistant message.*	*Low*
US-5.4	*As a Supervisor, I need the assistant to proactively highlight anomalies so that I can act before problems escalate.*	• *Scrap rate above 5% is flagged as a warning; above 15% as critical.* • *Scrap spikes (recent rate $>$ 2.5$\times$ baseline) are explicitly mentioned.* • *Overdue orders (past planned end) and at-risk orders ($<$ 4 h remaining) are highlighted.* • *Paused operations exceeding the configured threshold are surfaced.*	*Medium*

2.3 Sprint Plan

The project was organised into five two-week sprints, each delivering a functional increment verified by a sprint review with the professional supervisor.

Table 2.7. MES project sprint plan

Sprint	***Domain***	***Scope***
Sprint 1	*Authentication & User Management*	*Secure login,2 factor authentification(2FA), change password, profile page and session token management, role-based access control, administrator user creation, password generation and reset, role and work-centre change, account activation/deactivation, badge management (view, print, regenerate), password expiry policy configuration, export users and first-launch host pairing.*
Sprint 2	*Machine Monitoring & Production Order Management*	*Real-time machine status tracking, production order list per machine, operation start, tabbed machine view with Orders / Progress / History tabs, order filtering and sorting.*
Sprint 3	*Production Execution & Traceability*	*Production cycle declarations, pause/resume/finish/cancel/interupt operations, operation detail page with up-to-date data, production chart, Bill of Materials visibility, component barcode scanning, scrap declaration, supervisor proxy declaration, and label printing.*
Sprint 4	*Administration and System Monitoring & Configuration*	*Monitoring dashboard and Administrator activity log, barcode view and printing, multi-language support (EN/FR/AR) and RTL layout, in-app onboarding tutorials.*
Sprint 5	*AI Assistant & Analytical Intelligence*	*Natural-language chat assistant,* `Node.js` *reverse proxy middleware,* `Python` *AI agent with intent classification, multi-provider LLM support, fuzzy machine resolution, composite fan-out queries, data enrichment, anomaly detection (scrap spikes, overdue orders), and contextual redirect actions.*

2.4 Global Application Architecture

2.4.1 Physical Architecture

The physical architecture of the MES system rests on three main nodes:

- ***Business Central Server (on-premises / SaaS)****: hosts the AL extension that exposes*

OData V4 endpoints and the business REST APIs. In on-premises mode, the server runs on a BC210 instance accessible on port 7048. In SaaS mode, the base URLs follow the schema `api.businesscentral.dynamics.com/v2.0/<tenantId>/<environment>/ODataV4/`.

- ***Node.js Middleware*** *[5, 6]: a reverse-proxy layer running between the Flutter client and Business Central. It handles SSPI/Windows authentication toward BC, exposes a unified REST API consumed by Flutter, and routes AI chat requests to the Python AI agent.*
- ***Flutter Client (mobile / web / desktop)*** *[7]: application compiled for Android, Web or desktop, communicating with the Node.js middleware via HTTP requests. The middleware proxies these to the BC OData V4 endpoints.*
- ***A Python AI agent (Server)*** *[8, 9]: a module that acts as an interface between the application and the llm model chosen,includes intent classification tool calls.*

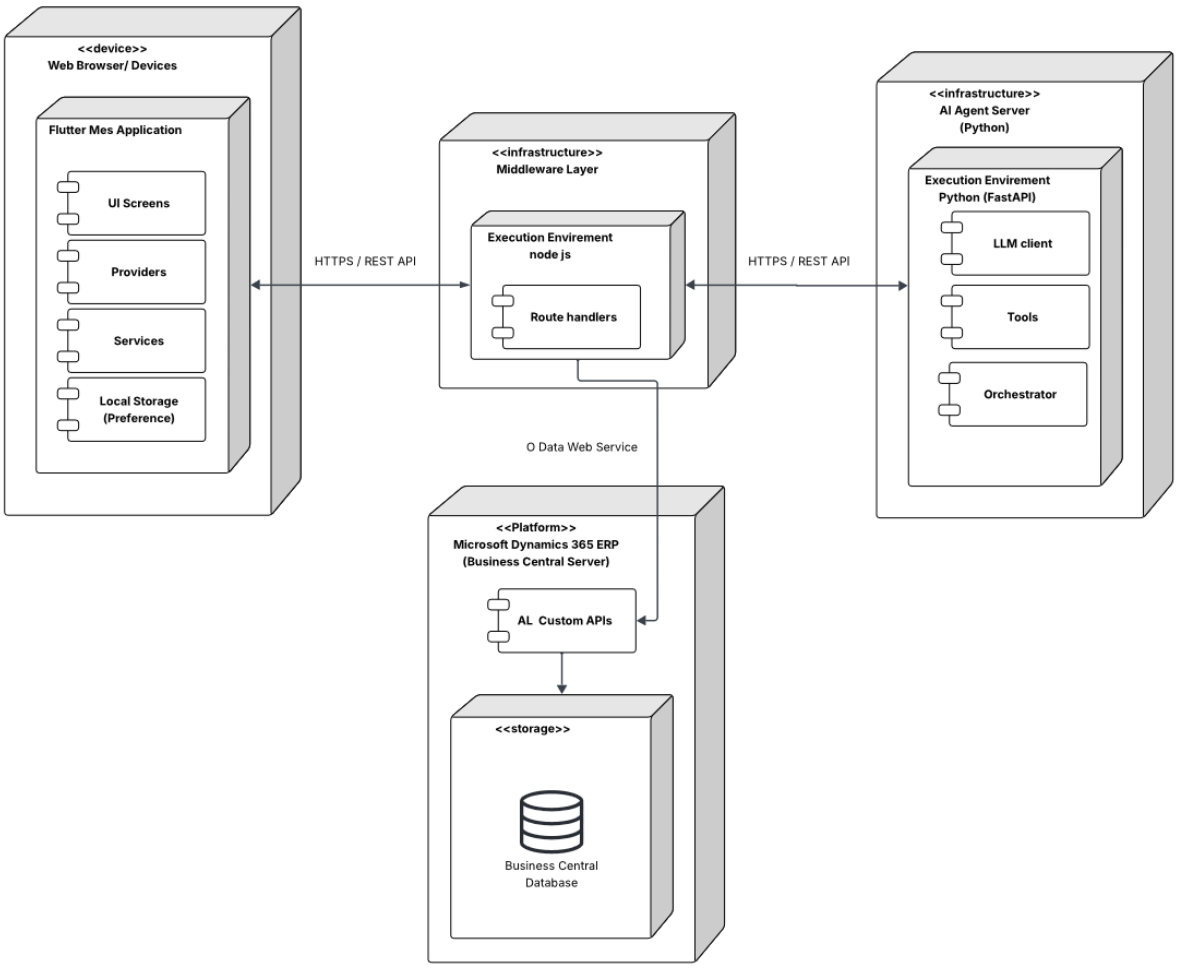

Figure 2.2. MES System Physical Architecture

2.4.2 Software Architecture

The software architecture of the system is organised around four distinct layers.

AL Back-end (Business Central)

The back-end is structured in five layers:

- ***Tables (1-Tables)**: define the persistent data model; MES User, MES Auth Token, MES Machine Status, MES Operation State, MES Operation Scrap, MES Operation Execution, MES Operation Progression, MES Settings, MES Operation Scan, MES User Work center, and MES User Execution Interaction.*
- ***Enums (2-Enum)**: define the enumerated types; MES User Role (Operator, Supervisor, Admin), MES Machine Status (Idle, Working), MES Operation State (Running, Paused, Finished, Cancelled, Interrupted), MES Token State (Revoked, Pending, Active).*
- ***Codeunits (3-CodeUnit)**: contain the business logic; MES Auth Mgt (authentication, tokens), MES Password Mgt (SHA-256 hashing), MES Machine Fetch (read operations), MES Machine Write (state orchestration), MES Machine Insert (table writes), MES Machine Validation (business rule guards), MES Authentification (user authentication), and the* Mes Web Service *(wrappers exposed as web services).*
- ***API Pages (4-API)**: expose data for reading/writing via the BC REST API; scrap codes, work centres.*
- ***Pages (5-Pages)**: administration and debugging interfaces in the BC client (lists, cards, API debug page, etc...).*

Node.js Middleware

The Node.js middleware is the network bridge between the Flutter client and Business Central. It handles SSPI/Windows authentication for BC OData V4 requests, exposes a unified REST API to the Flutter application, and routes AI chat requests (`POST/api/ai/chat`) to the Python AI agent.

Python AI Agent

The Python AI agent (FastAPI) provides the conversational intelligence layer. It classifies user intent, executes tool chains against Business Central via the Node.js middleware, enriches results with data-analysis helpers, and synthesises natural-language replies using a configurable multi-provider LLM client.

Flutter Front-end

The Flutter front-end follows a three-layer architecture:

- ***Data**: contains data models (models) and HTTP services (services) that communicate with BC endpoints via the `http` library.*
- ***Domain**: contains the providers (Provider pattern) that expose the application state*

to widgets — AuthProvider, MesUserProvider, MesMachinesProvider, MachineordersProvider, etc.

- ***Presentation**: contains the pages and widgets of the user interface, organised by functional domain (auth, admin, machine, etc) with dedicated responsive layouts.*

2.5 Work Environment

2.5.1 Software Environment

Table 2.8 summarises the main tools and technologies used in this project.

Table 2.8. MES project software environment

Logo	***Tool / Technology***	***Role in the project***
	*Microsoft Dynamics 365 Business Central [10]*	*ERP platform hosting the AL back-end and business data*
	*AL Language (Application Language)*	*Development of the BC extension (tables, codeunits, API pages)*
	*Flutter (Dart) [7, 11]*	*Development of the cross-platform mobile/web application*
	*Visual Studio Code*	*Primary development environment (AL)*
	*Git / GitHub*	*Version control and collaboration*
	*Postman [12]*	*Testing and validation of OData V4 endpoints*
	*Python (FastAPI)*	*AI agent layer: intent classification, multi-provider LLM synthesis, fuzzy machine resolution, and analytics enrichment.*
	*Node.js (Express 5.x)*	*Reverse-proxy middleware: Windows SSPI authentication toward Business Central, unified REST API for Flutter and Python clients.*

Continued on next page

Logo	*Tool / Technology*	*Role in the project*
	*lucid chart*	*UML and system architecture diagramming*
	*Android Studio*	*Primary development environment (Flutter)*

2.5.2 Development Environment

The project was developed on two workstations: the first machine equipped with an 11th Gen Intel i5 processor, and the second machine with a 13th Gen Intel i7 processor. Both had 16 GB RAM and ran a 64-bit `Windows` *operating system.*

The software toolchain consisted of `Flutter3.41.0` *and* `Dart3.11.0` *for the mobile frontend,* `Node.jsv24` *for the middleware layer,* `Python3.14` *with* `FastAPI` *for the AI agent, and* `MicrosoftDynamics365BusinessCentralNAV22` *as the ERP backend. The extensions* `AL-Language`, `Flutter`, *and* `Dart` *were installed in* `VisualStudioCode`.

The on-premises `BusinessCentral` *instance was configured to expose both the* OData V4 *and* API Services *endpoints, which are consumed by the middleware and the mobile frontend.*

Conclusion

In this chapter, we analysed and specified the functional and non-functional requirements of the MES system, identified the actors and their interactions, and presented the overall architecture of the solution. We also described the work environment and the adopted Scrum methodology. The following chapters detail the work carried out sprint by sprint, presenting the conceptual aspects and the results obtained at the end of each iteration.

Chapter 3

Sprint 1 Authentication & User Management

Contents

Introduction

This sprint focuses on delivering a complete authentication and user management module for the MES *application. The primary goal is to develop a secure authentication system covering login, logout, and password management, alongside role-based page access control to ensure each user can only reach features appropriate to their role.*

In addition, the Administrator is given the ability to add new MES users, generate temporary secure passwords on their behalf for the first login, change a user's role and work-centre assignment, export users into a file, and activate or deactivate accounts. The sprint also delivers two-factor authentication via badge QR code scanning, administrator badge management (view, print, regenerate), a configurable password expiry policy, and the first-launch host pairing screen that connects the Flutter application to the correct `Node.js` *middleware instance.*

3.1 Sprint Backlog

This sprint covers ***Domain 1:*** **Authentication & User Management**.

Table 3.1. Story Backlog

ID	*User Story*	*Tasks*
US-1.1	*As an Administrator, I need to assign a user account for each employee so that users only access features appropriate to their role.*	*Business Central (AL)* • *Create the tables* `MESUser`, `MESUserWorkCenter` *and* `MESAuthToken`. • *Create enum* `MESUserRole` *and* `MESTokenstate`. • *Implement the functions* `AdminCreateUser()` *and* `AdminSetActive()` *in codeunit* `MESAuthenticationActions`. • *Expose* `AdminCreateUser()` *and* `AdminSetActive()` *through codeunit* `MESWebService`. *Flutter* • *Implement* `ApiService` *to consume the new endpoints.* • *Implement* `MesUserService` *to consume* `fetchMesUsers()` *and* `AdminCreateUser()` *endpoints.* • *Create provider* `MesUserProvider` *to manage its state.* • *Create widget* `AddUserDialog`.

Continued on next page

ID	User Story	Tasks
		• *Implement role-based navigation in* `_AuthGate`.
US-1.2	*As a User (Operator / Supervisor / Administrator), I need to log into and out of the system with my credentials so that I can access features appropriate to my role.*	*Business Central (AL)* • *Implement functions* `MakeSalt()`, `HashPassword()`, *and* `VerifyPassword()` *in codeunit* `MESPasswordMgt`. • *Implement functions* `CreateUser()`, `SetPassword()`, `ValidateCredentials()`, `IssueNewToken()`, `ValidateToken()`, `TouchToken()`, `Logout()`, `ValidateChangePassword()`, `ValidateAdminToken()`, `SetActive()`, *and* `CleanupExpiredTokens()` *in codeunit* `MESAuthMgt`. • *Implement functions* `Login()`, `Logout()`, `Me()`, *and* `ChangePassword()` *in codeunit* `MESAuthenticationActions`. • *Expose* `Login()`, `Logout()`, `Me()`, *and* `ChangePassword()` *in codeunit* `MESWebService`. *Flutter* • *Implement* `ApiService` *to consume the new endpoints.* • *Create provider* `AuthProvider` *to manage its state.* • *Implement authentication routing in* `_AuthGate`. • *Create page* `LoginPage`. • *Create a* `logoutbutton`.
US-1.3	*As an Administrator, I need to search users.*	*Flutter* • *Implement user search in* `AddUserPage` *using either the name or the email of the user.* • *Implement role filtering in* `AddUserPage`.

Continued on next page

ID	User Story	Tasks
US-1.4	*As an Administrator, I need to see a list of all MES users with their details so that I can manage accounts.*	*Business Central (AL)* • *Create the function `fetchAllMESUsers` in codeunit `MESAuthenticationActions`.* • *Expose `fetchAllMESUsers` through codeunit `MESWebService`.* *Flutter* • *Create model `MesUser`.* • *Implement `MesUserService` to consume the new endpoint.* • *Implement `MesUserProvider` to manage its state and its streams.* • *Implement user list display in `AddUserPage`.*
US-1.5	*As an Administrator, I need to generate and assign a temporary password to a user so that they can log in for the first time.*	*Business Central (AL)* • *Implement function `AdminSetPassword()` in codeunit `MESAuthenticationActions`.* • *Expose `AdminSetPassword()` through codeunit `MESWebService`.* *Flutter* • *Create the widgets `GeneratePasswordDialog` and `EmployeeAvatar`.* • *Update `ApiService` to consume `adminSetPassword()`.* • *Update `AuthProvider` to account for the new functions in the service.*
US-1.6	*As an Administrator, I need to change the role and work-centre assignment of an existing user so that access rights stay current.*	*Business Central (AL)* • *Implement function `AdminChangeUserRole()` in codeunit `MESAuthenticationActions`.* • *Expose `AdminChangeUserRole()` through codeunit `MESWebService`.* *Flutter* • *Update `MesUserService` to consume `AdminChangeUserRole()`.* • *Update `MesUserProvider` to account for `AdminChangeUserRole()`.* • *Create widget `ChangeRoleDialog`.*

Continued on next page

ID	User Story	Tasks
US-1.7	*As an Administrator, I need to activate or deactivate a user account so that access can be revoked without deleting the account.*	*Flutter* • *Update* `ApiService` *to account for* `toggleUserActiveStatus()`. • *Update* `AuthProvider` *to account for* `toggleUserActiveStatus()`. • *Integrate activate and deactivate actions in* `AddUserPage`.
US-1.8	*As a User (Operator / Supervisor / Administrator), I need to complete a badge QR code scan after entering my credentials so that my identity is verified by a second factor.*	*Business Central (AL)* • *Add field* `BadgeSecret` *(Text[64]) to table* `MESUser`. • *Implement* `GenerateBadgeSecret()` *in* `MESUserOnInsert`. • *Implement* `VerifyBadge()`, `GetBadgeSecret()`, *and* `RegenerateBadgeSecret()` *in codeunit* `MESAuthMgt`. • *Expose the three procedures through codeunit* `MESWebService`. • *Add field* `TwoFAEnabled` *(Boolean) to table* `MESSettings`. • *Return* `twoFAEnabled` *flag in the* `Login()` *response.* *Flutter* • *Update* `AuthProvider.login()` *to detect* `twoFAEnabled` *flag and hold pending session state.* • *Create page/dialog* `BadgeScanDialog` *with live camera feed (*`mobile_scanner`*) and manual entry fallback.* • *Implement* `verifyBadge()`, `cancelBadgeScan()` *in* `AuthProvider`. • *Integrate badge verification step into* `_AuthGate` *routing logic.*

Continued on next page

ID	User Story	Tasks
US-1.9	*As an Administrator, I need to view, print, and regenerate a user's badge QR code so that I can distribute it and invalidate a compromised badge.*	*Flutter* • *Create dialog* `UserBadgeDialog` *rendering badge QR via* `barcode_widget` *package.* • *Implement* `getBadgeSecret()` *and* `regenerateBadge()` *in* `AuthProvider`*.* • *Implement PDF export in* `UserBadgeDialog._exportPdf()`*.* • *Add View Badge action to* `UserActionMenu` *(active users only).*
US-1.10	*As an Administrator, I need to globally enable or disable* 2FA *and configure a password rotation period so that I can control the organisation's authentication security level.*	*Business Central (AL)* • *Add field* `PWchangeperiod` *(Duration) to table* `MESSettings`*.* • *Implement* `GetMESSettings()` *and* `UpdateMESSettings()` *in codeunit* `MESSettingsFunctions`*.* • *Expose both procedures through codeunit* `MESWebService`*.* • *Implement codeunit* `MESPasswordExpiryWorker` *(Job Queue, runs every 24 h) that flags users whose password age exceeds the configured period.* • *Register the worker in* `MESSetup.RegisterPasswordExpiryWorker()`*.* *Flutter* • *Create model* `SettingModel` *with* `pwChangePeriod` *and* `twoFAEnabled` *fields.* • *Create* `SettingService` *consuming* `fetchSettings` *and* `updateSettings` *endpoints.* • *Create* `MesSettingsProvider` *managing fetch, dirty-state detection, and save.* • *Create page* `MesSettingsPage` *with toggle and numeric input; animated dirty banner with Discard / Save actions.* • *Add Settings section to* `AdminPage` *sidebar.*

Continued on next page

ID	*User Story*	*Tasks*
US-1.11	*As a first-time user, I need to configure the middleware host URL (or scan a QR code) so that the application connects to the correct server instance.*	*Flutter* • *Create page* `ParingPage` *shown on first launch when* `AppConstants.host` *is empty.* • *Implement URL validation (no spaces, no backslash, no illegal characters).* • *Implement QR scanning via* `QrScannerDialog` *to populate the host field.* • *Persist accepted host via* `AppConstants.changeHost()` *(SharedPreferences).* • *Add Change Host link on* `LoginPage` *for reconfiguration.* • *Add* `AppConstants.loadHost()` *call to the app initialisation sequence in* `main()`.
US-1.12	*As an Administrator, I need to be able to export the list of users so that I can use the info in other applications or processes.*	*Flutter* • *implemented export user service so when clicking the export button and excel file is created with all user info* • *Updated the* `AddUserPage` *to contain the button export users*

3.2 Design

3.2.1 Detailed Use Case Diagram

The UML Use Case Diagrams (Figure 3.1 to 3.3) shows the three primary actors (Operator, Supervisor, Administrator) and their interactions with the MES *authentication and user management system. In addition to the core login, logout, change password, and user creation flows, the diagram includes the Change Role and Work Centre, Activate / Deactivate Account, and export users use cases available exclusively to the Administrator, as well as the settings management Badge QR Scan two-factor verification step and the automatique password change.*

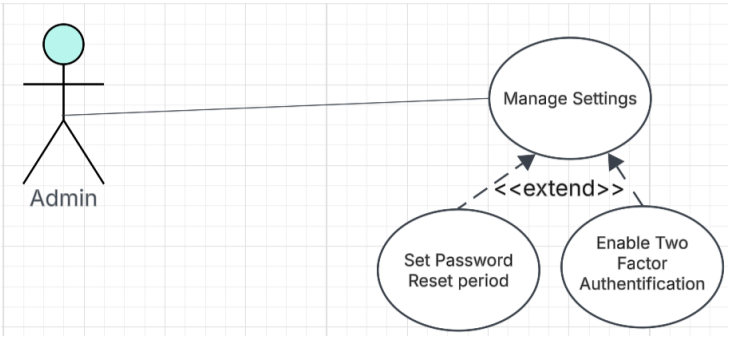

Figure 3.1. UML Use Case Diagram — Manage Settings.

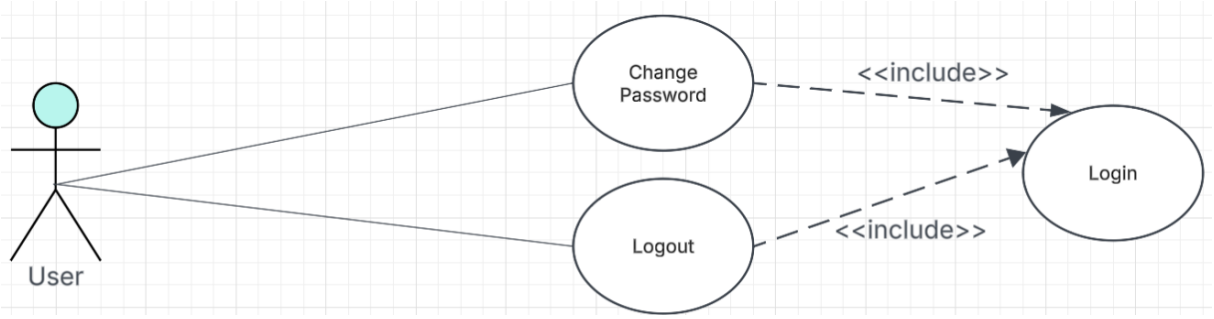

Figure 3.2. UML Use Case Diagram — Authentication.

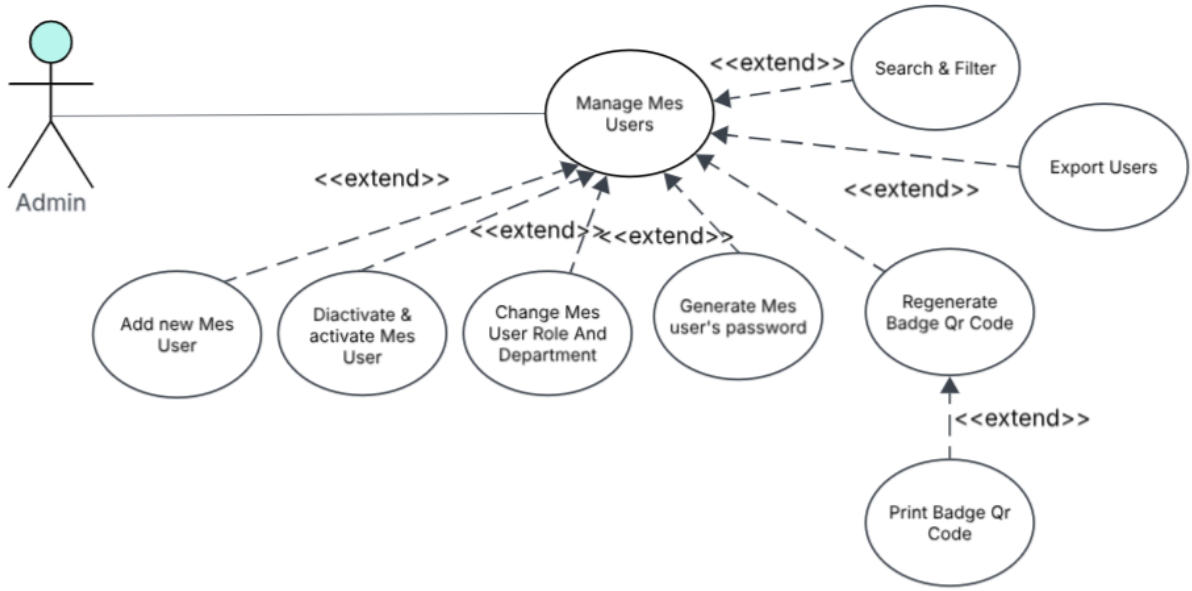

Figure 3.3. UML Use Case Diagram — Manage Users.

3.2.2 Textual Use Case Description

Use Case Name	*Login*
Primary Actors	*Operator, Supervisor, Administrator*
Preconditions	• *The employee account exists in the ERP system.* • *The employee has a user account in the MES that is active.* • *The user has an assigned role.* • *The account must have a password.*
Postconditions	• *If the credentials are valid, the user is successfully authenticated.* • *A session token is generated and stored securely.*

Continued on next page

	• *If* 2FA *is enabled, the badge QR scan step is completed before a session token is issued.* • *The user is redirected to the appropriate dashboard based on their role (Admin or MES interface).* • *If the* need change password *flag is set to true, they are redirected to the Change Password page.* • *If the credentials are invalid, access is denied and an error message is displayed.*
Main Scenario	1. *The user launches the application.* 2. *If no host URL is configured, the system displays the Paring page, the user enters or scans the middleware URL.* 3. *The system displays the login screen.* 4. *The user enters Auth ID and password.* 5. *The system automatically detects the device ID.* 6. *The user clicks the Login button.* 7. *The system validates the credentials against the* `MES` *database.* 8. *The system generates a secure session token, If* 2FA *is enabled it is set to state* Pending *otherwise it is set to state* Active. 9. *If* 2FA *is enabled, the system opens the Badge Scan dialog, the user scans their badge QR or enters the code manually.* 10. *The system verifies the badge secret and sets the token state to* Active. 11. *The system checks the user role.* 12. *The system redirects the user to the appropriate interface(admin* `dasboard`*, or machine list).*

3.2.3 Sequence Diagrams

The following sequence diagrams illustrate the communication flow between the `Flutter` *frontend, the* `BusinessCentral` *OData layer, and the* `AL` *business logic for each authentication flow.*

Figure 3.4 depicts the login flow, in which the user submits credentials, the `AL` *layer validates them against the* `MES`*, and the application either returns a structured error or issues a session token (or, when* 2FA *is enabled, holds a pending state until the badge scan is verified) and redirects the user to the appropriate page based on their role and password-change flag.*

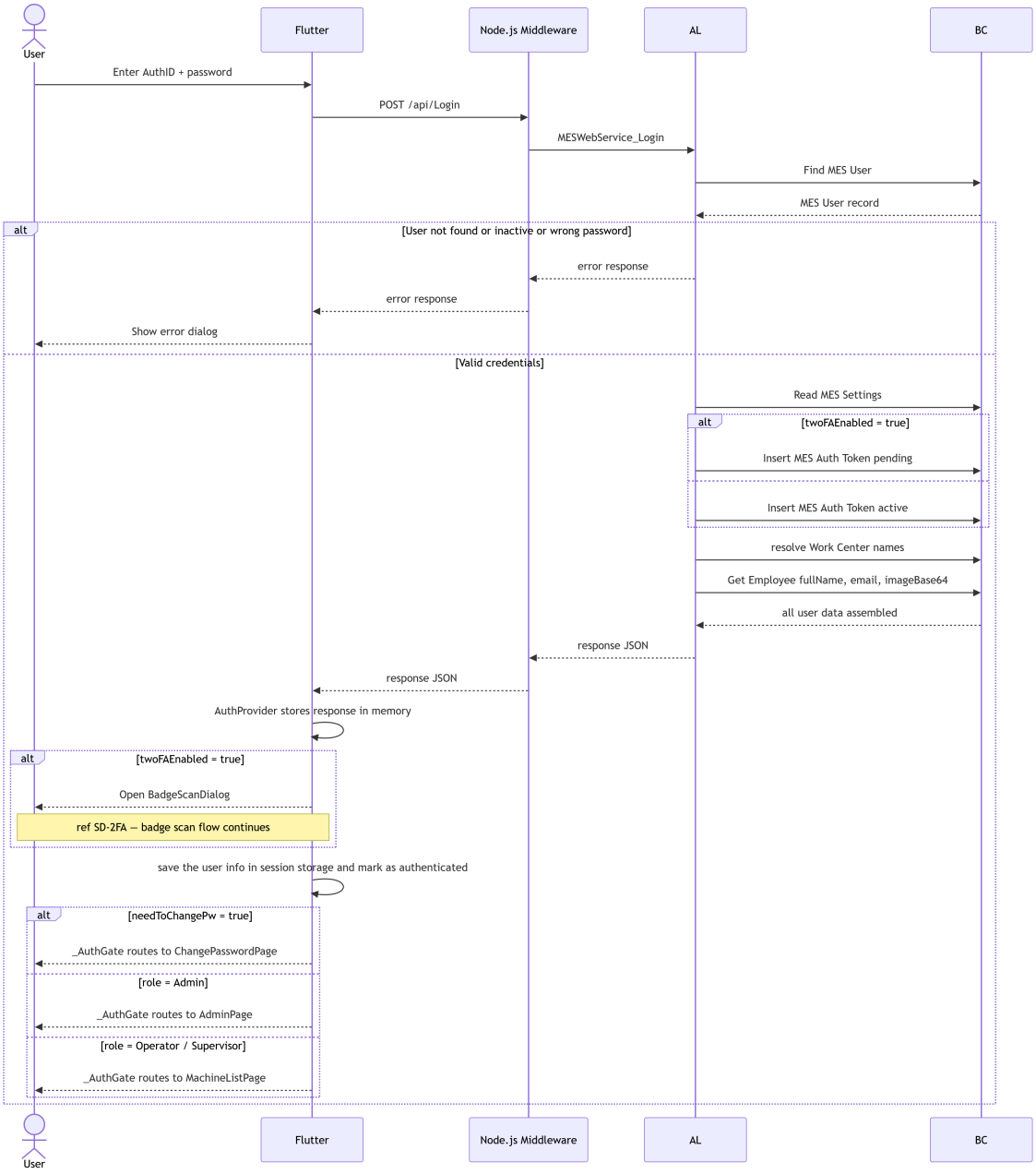

Figure 3.4. SD-01: Login.

Figure 3.5 depicts the badge scan flow, which is entered when 2FA *is enabled after credentials have been validated (see SD-01). The user presents a* QR *badge or enters the secret manually;* `Flutter` *forwards it to the* `AL` *layer, which verifies the secret against the pending token record and either activates the session or returns an error. Cancelling at any point clears the pending state and returns the user to the* `LoginPage`.

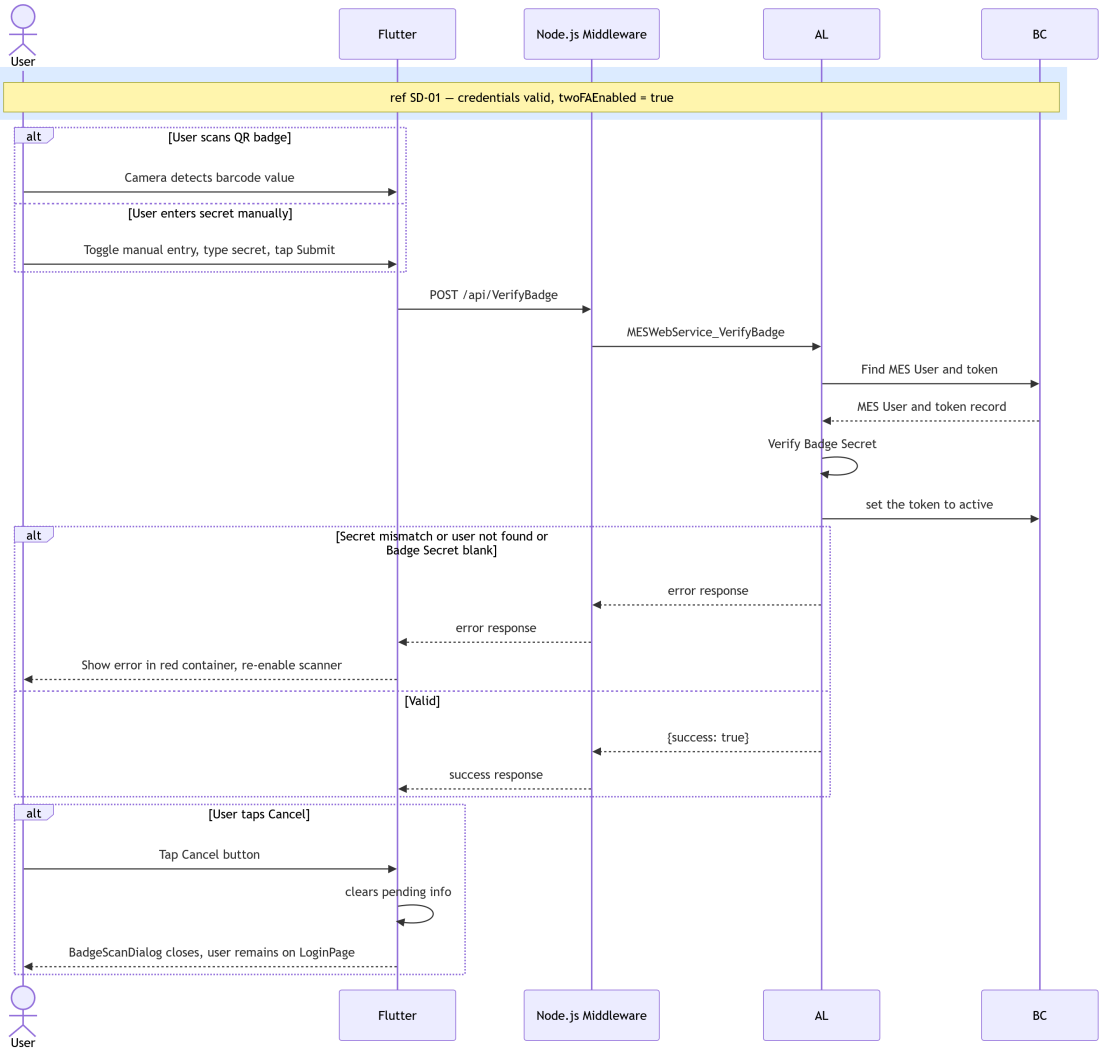

Figure 3.5. SD-2FA: Badge Scan.

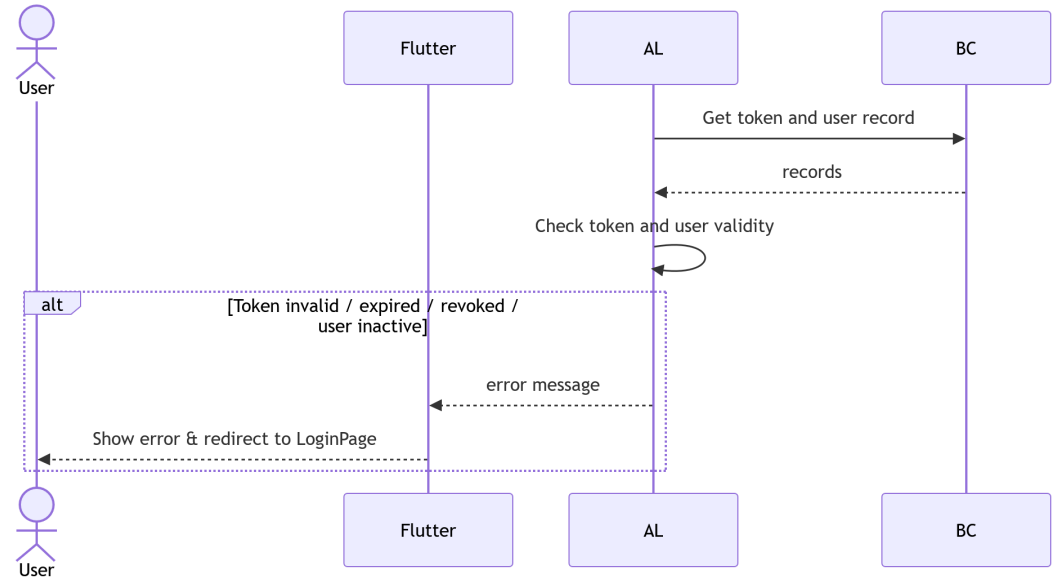

Figure 3.6. SD-02: Token Validation.

Figure 3.6 presents the token validation flow, which is invoked by the `AL` backend before processing any authenticated request. The backend retrieves the token record from `MES`, verifies its validity and expiry, confirms that the associated user account is still active, and either updates the `LastSeenAt` timestamp or returns an error that forces the `Flutter` client to redirect the user to the login page.

Figure 3.7 illustrates the logout flow, in which the `Flutter` client sends a logout request, the `AL` layer retrieves and revokes the current session token in `BusinessCentral`, and the client clears its local `AuthProvider` state before navigating back to the login page.

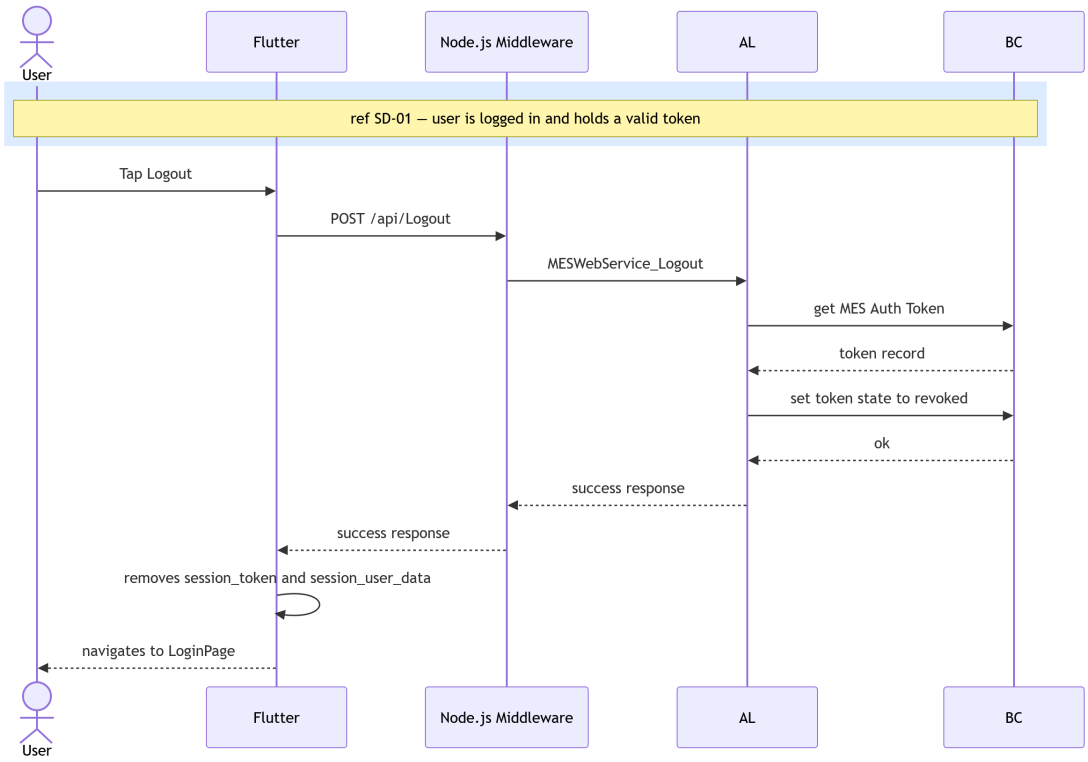

Figure 3.7. SD-03: Logout

Figure 3.8 describes the change-password flow. The `Flutter` client first validates locally that the fields, then forwards the request to the `AL` layer, which re-validates the token (SD-02), verifies the current password hash, checks the new password strength, and on success updates the credential record in `MES`.

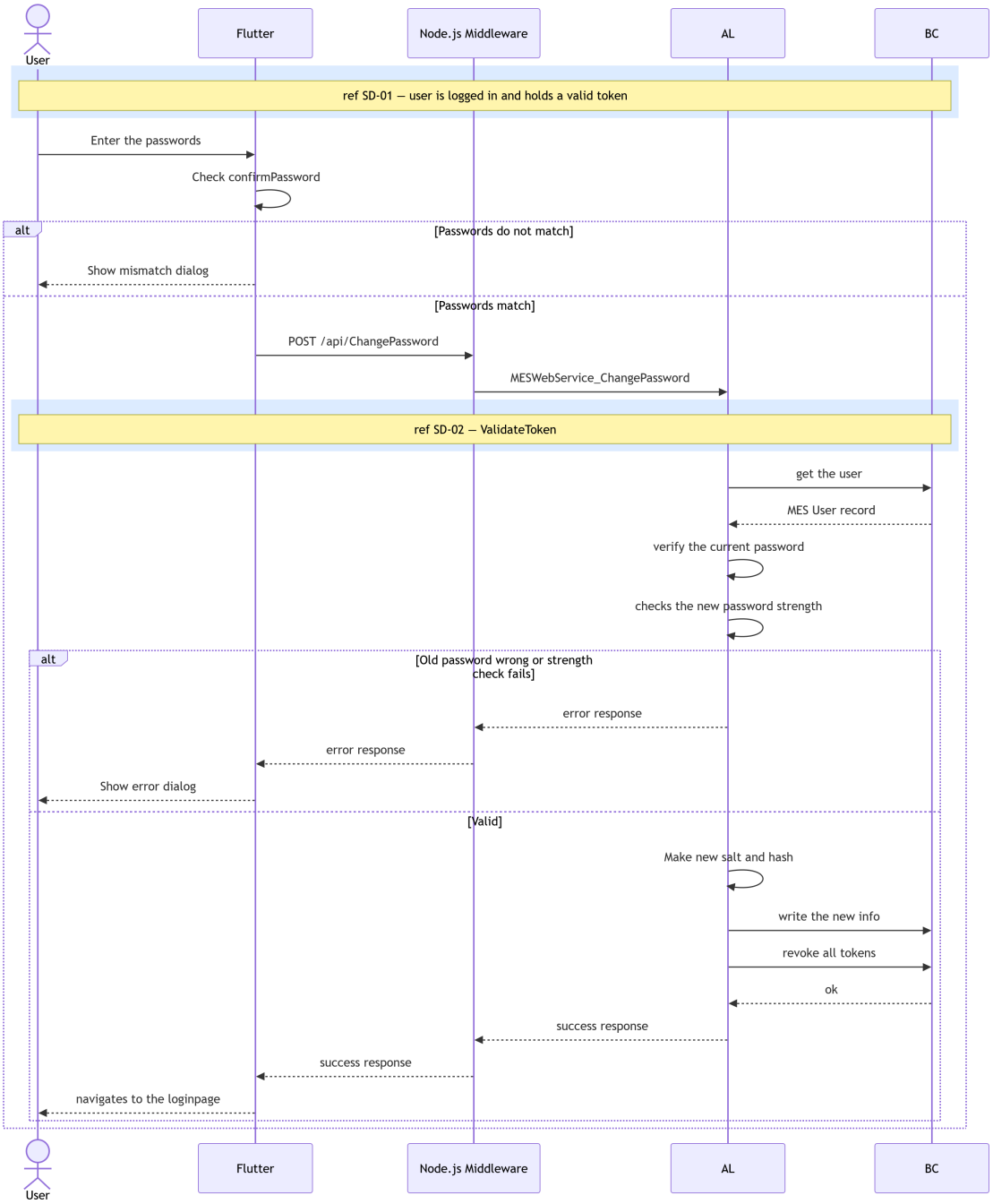

Figure 3.8. SD-04: Change Password

Figures 3.9 through 3.11 cover some of the administrator user-management operations. All follow the same pattern: the `Flutter` client sends an admin-scoped request, the `AL` layer validates the token and confirms that the caller holds the Admin role (SD-02), executes the requested `BusinessCentral` write operations, and triggers a reactive refresh so that the updated user list is reflected immediately in the UI.

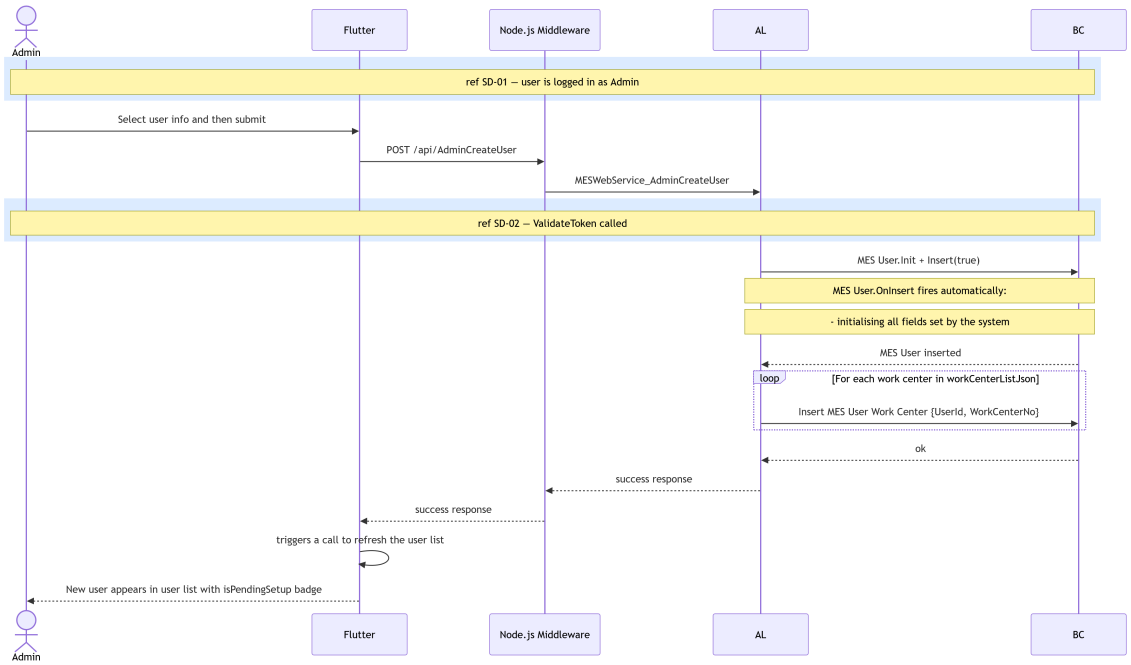

Figure 3.9. SD-05: Admin - Create User.

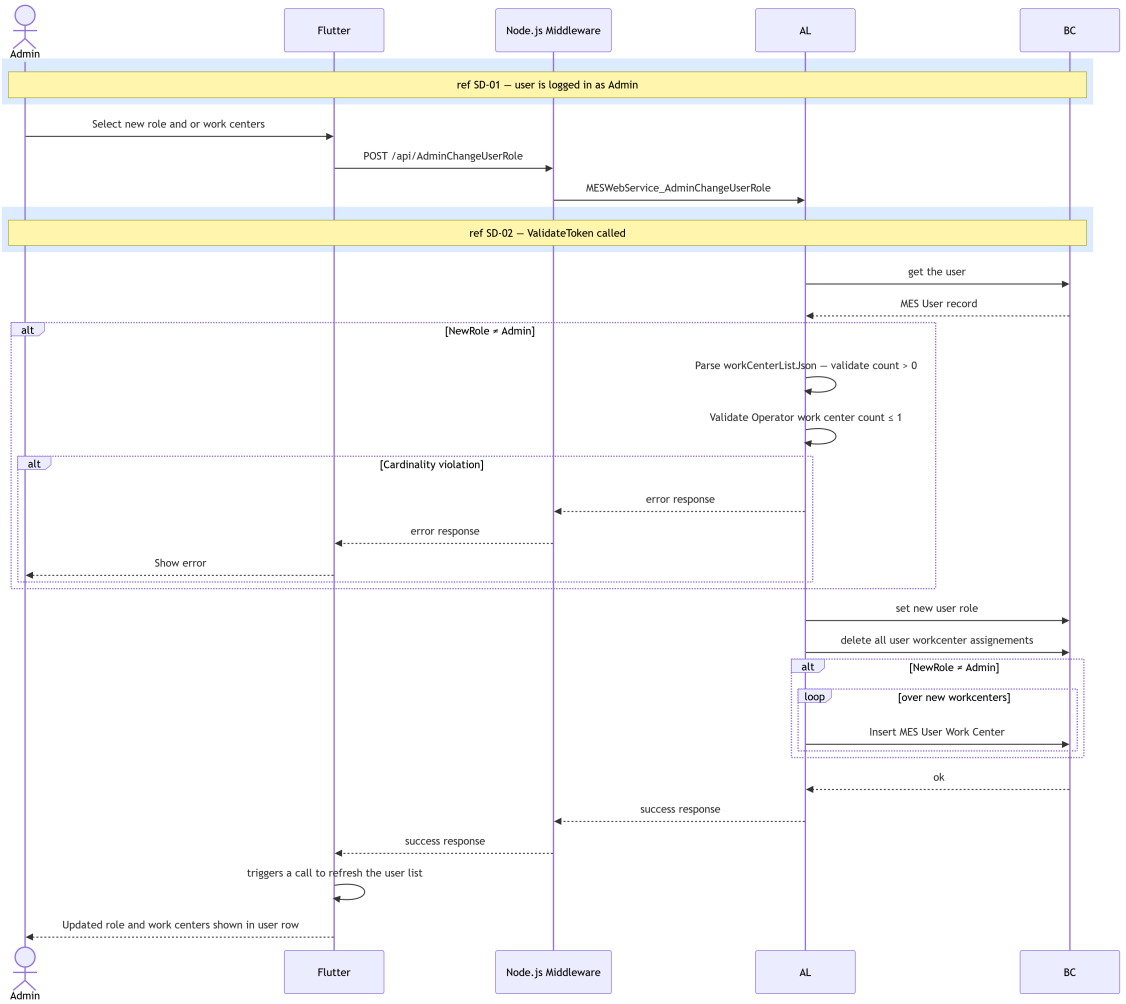

Figure 3.10. SD-07: Admin - Change Role / Work Centre.

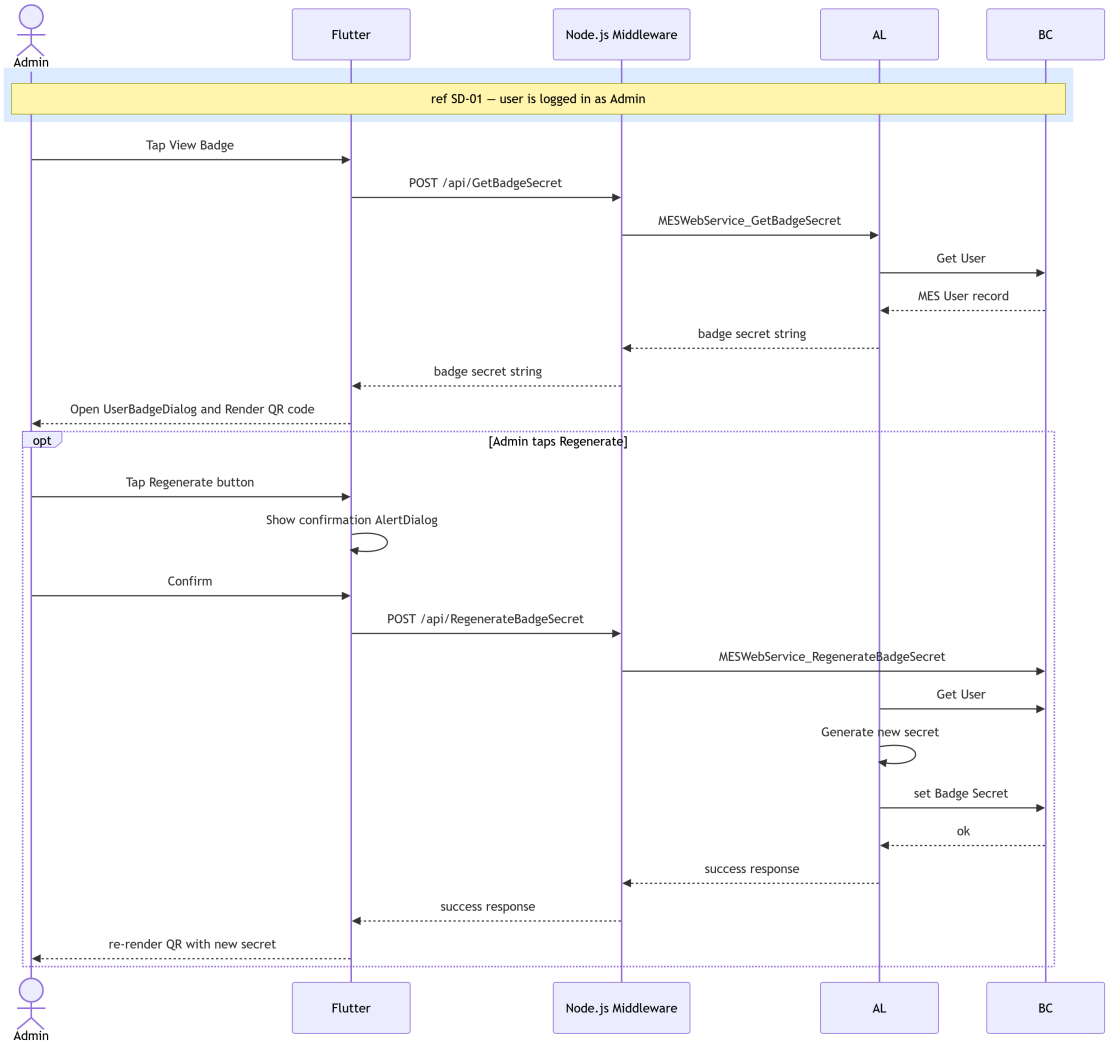

Figure 3.11. SD-ADMIN-BADGE: Admin - regenerate 2FA user badge.

3.2.4 Class Diagram

The UML Class Diagram of the MES *authentication domain, shown in Figure 3.12, depicts the* `MesUser`*,* `AuthToken`*, and* `settings` *classes with their attributes.*

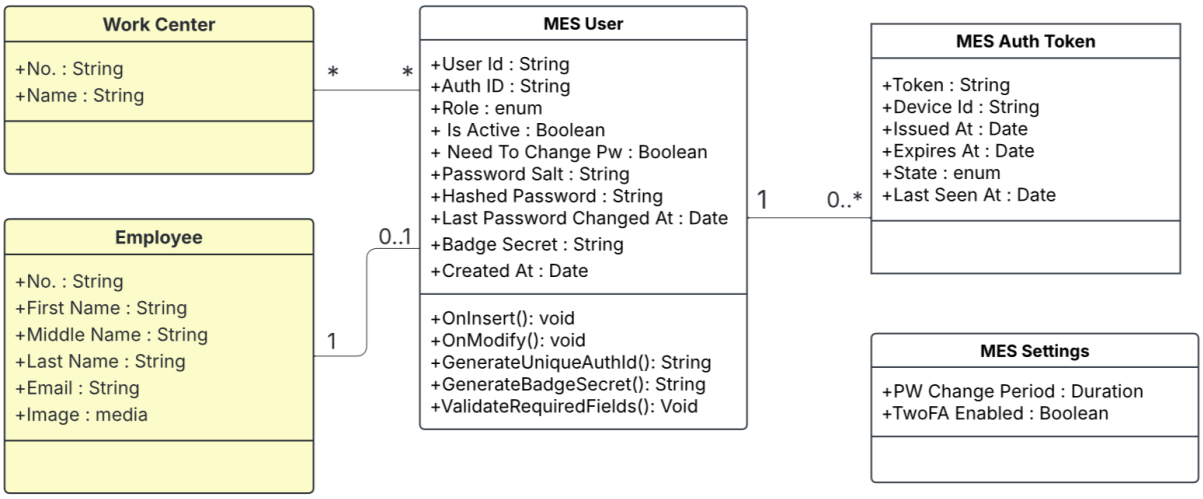

Figure 3.12. UML Class Diagram.

3.3 Implementation

3.3.1 Backend Implementation

Backend Structure Overview

The backend project structure is illustrated in Figure 3.13.

MES User Table Implementation

The `MES User` table is implemented to store user information, as described in Table 3.3.

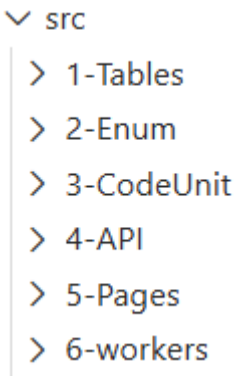

Figure 3.13. Backend project structure.

Name	*Type*	*Description*
User Id	*Code[50]*	*Primary key. Login identifier used at the MES login prompt (e.g., `JDOE`, `OP001`). If blank on insert, a GUID-derived value is assigned automatically.*
Employee ID	*Code[50]*	*`Employee."No."`. Required for every MES account and enforced as unique (one Employee maps to one MES User).*
Auth ID	*Text[100]*	*Auto-generated external identifier in the form `AUTH-XXXXXXXX`. Used as the login identifier at the MES login prompt. Uniqueness enforced.*

Continued on next page

Name	*Type*	*Description*
Role	*Enum (`MESUser-Role`)*	*Authorization role for the MES application (e.g., Operator / Supervisor / Admin). Used to control permissions and access.*
Is Active	*Boolean*	*Account enable/disable flag. If false, authentication is blocked and existing tokens are revoked when deactivated.*
Need To Change Pw	*Boolean*	*If true, the user must change their password before using the MES application (set at creation and on forced password resets).*
Password Salt	*Text[64]*	*Random hex salt used in password hashing so identical passwords produce different hashes across users.*
Hashed Password	*Text[128]*	*Hashed password value (hex string). Passwords are never stored in plaintext; hashing uses the password plus salt.*
Created At	*DateTime*	*UTC timestamp set on insert only (audit trail/reporting).*
Last Password Changed At	*DateTime*	*Updated whenever the password is modified. Used by the password expiry worker to determine whether a rotation is due.*
Badge Secret	*Text[64]*	*SHA-256-derived secret generated on user creation. Used to verify badge QR code scans during the two-factor authentication step.*

Table 3.3. MES User (Table 50101) — Field Dictionary

Authentication Token Table

The `MES Auth Token` table stores session information as described in Table 3.4.

Name	*Type*	*Description*
Token	*Guid*	*Primary key. Raw GUID token presented by the client as a Bearer credential. Generated at issue time (never reused). Used for all direct token lookups (validation/logout).*
User Id	*Code[50]*	*Foreign key to `MESUser."UserId"`. Identifies the MES user who owns this session/token. Enforces referential integrity via `TableRelation`.*

Continued on next page

Name	*Type*	*Description*
Device Id	*Text[100]*	*Optional identifier for the device that requested the token (e.g., `tablet-floor-02`, `scanner-01`). Stored for traceability/audit only; not used for validation.*
Issued At	*DateTime*	*UTC timestamp set when the token is created. Useful for audit trail and token age inspection.*
Expires At	*DateTime*	*UTC timestamp after which the token must be rejected even if not revoked. Set to `IssuedAt+TTL`. Token validation enforces `ExpiresAt>CurrentDateTime()`.*
Last Seen At	*DateTime*	*Updated on every successful token validation. Supports session monitoring (activity/idle detection). Not used for expiry enforcement (use `ExpiresAt`).*
State	*Enum (`MESAuthTokenState`)*	*The state of the authentication token (Pending, Active, Revoked) set to pending when 2FA is on, and revoked when logging out and bulk revocation (password change, account deactivation).*

Table 3.4. MES Auth Token (Table 50106) — Field Dictionary

Key points of the token design include a unique GUID *token, auto-expiry after 12 hours, last activity tracking, and soft revocation.*

MES Settings Table

The `MES Settings` singleton table (Table 50100) stores the two global security parameters configurable from the Settings page.

Name	*Type*	*Description*
PW change period	*Duration*	*Password rotation interval stored in milliseconds. A value of 0 does not disable the feature; only negative values do.*
TwoFA Enabled	*Boolean*	*Global flag. When true, all users must complete a badge QR code scan after entering their credentials.*

Table 3.5. MES Settings (Table 50100) — Field Dictionary

Password Management and Encryption

The password management codeunit ensures that plaintext is never persisted. A unique random salt *(64-character SHA-256 hex string) is generated per user and the password is hashed using single-pass* SHA-256 *before storage. Data classification attributes mark sensitive fields as* Customer Content *or* System Metadata*.*

Key points include: passwords are never stored in plaintext; a unique salt per user (64 characters) is generated; SHA-256 *hashing is applied; and data is classified as* Customer Content *or* System Metadata*.*

Authentication Logic — Login, Logout, and Change Password

Login

The login validates the user credentials against the `MES User` *table. If the credentials are correct and the account is active, and* 2FA *is disabled in* `MES Settings`*, a token is generated and stored in the* `MES Auth Token` *table and set as active. When* 2FA *is enabled, a token is generated and stored in the* `MES Auth Token` *table and set as pending, the backend returns the* `twoFAEnabled` *flag and the* `Flutter` *client suspends the session until the badge QR code is verified via the* `VerifyBadge` *endpoint at which point the token is set to active.*

Logout

On logout, the user's session authentication token is revoked. The token record matching the supplied session token is located in the `MES Auth Token` *table and its* `State` *is set to* `Revoked`*, immediately invalidating the session (soft revocation).*

Change Password

Changing password requires a valid authentication token, a correct current password, and a new password that complies with security rules.

Password Expiry Worker

The `MES Password Expiry Worker` *(Codeunit 50129) is a scheduled* `BusinessCentral` *Job Queue entry that runs every 24 hours. On each run it reads the* `PWchangeperiod` *value from* `MES Settings` *and computes a cutoff datetime equal to* `CurrentDateTime()minustheperiod`*. It then iterates all active MES Users whose* `LastPasswordChangedAt` *falls before the cutoff (or is zero, meaning the password has never been changed) and sets their* `NeedToChangePw` *flag to* `true`*. Affected users are prompted to change their password on their next login.*

3.3.2 REST API and Web Services Implementation

To enable communication between `Flutter` *and* `MicrosoftDynamicsBusinessCentral`*, a set of* RESTful APIs *were implemented using AL.*

OData Functions and Web Service Configuration

Exposed these functions in the `MESWebService` *codeunit (50126). Configured* `WebServices.xml` *to publish* `MESWebService` *as an ODataV4 web service under the service name* `MESWebService`.

3.3.3 Frontend Implementation

The frontend of this system was developed using `Flutter`. *The application is structured into four main layers: a* **UI Layer** *consisting of Screens and Widgets; a* **State Management Layer** *built on Providers; a* **Service Layer** *for API communication; and a* **Model Layer** *for Data Models.*

The data flow follows a unidirectional pattern from the UI Layer *through the* State Management Layer *(*`Provider` *pattern) and* Service Layer *to the* `BusinessCentral` *REST API backend, as illustrated in Figure 3.14.*

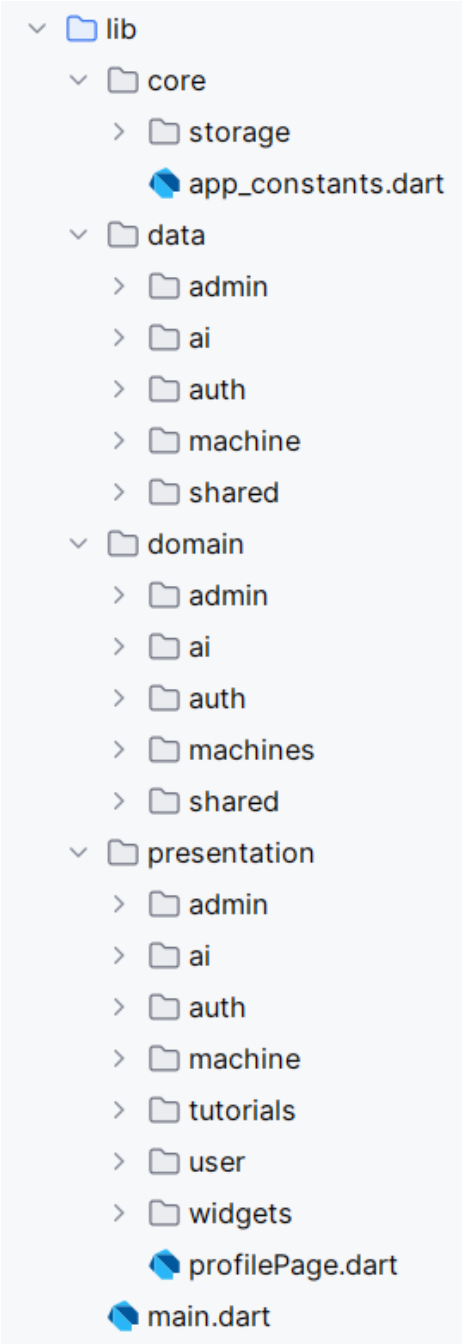

Figure 3.14. `Flutter` layered architecture.

Service Layer (API Communication)

The Service Layer *is responsible for handling all* HTTP *communication between* `Flutter` *and the* `BusinessCentral` *backend.*

Four services were implemented: the `api_service` *manages authentication requests such as login, logout, and password change, while the* `mes_user_service` *handles MES user operations including fetching and creating new users, A dedicated* `setting_service` *handles fetching and updating the MES global settings, And finaly* `export user Service` *handles the export user list to Excel feature.*

`api_service.dart`

The login method sends user credentials and device ID to the `/login` *endpoint and parses the returned session token, role,* needChangePassword *flag, and* twoFAEnabled *flag. When* 2FA *is enabled, the token is held in memory rather than persisted until badge verification succeeds. The logout method sends the active session token to the* `/logout` *endpoint to trigger server-side token revocation.*

The change password method submits the current password, new password, and session token to the `/changePassword` *endpoint. Shared error-handling and response-parsing utilities are used*

by all methods in the service class to propagate structured exceptions to the State Management Layer.

mes_user_service.dart

The class declaration and fetch users method send a request to the `/fetchAllMesUsers` *endpoint and parses the response into a list of* `MesUser` *model objects. The create user method sends a request to the* `/AdminCreateUser` *endpoint with the new user's role and employee reference in the request body. The generate password method handle the password generation endpoint and map the result into the* `GeneratedPassword` *model.*

State Management Layer using Provider

The Provider *pattern is implemented to ensure reactive UI updates and centralised data handling. Three providers were implemented:* `AuthProvider`*, which manages authentication state and the 2FA pending flow;* `MesUserProvider`*, which handles MES user data; and* `MesSettingsProvider`*, which manages the global settings fetch. Each provider wraps the corresponding service layer calls and exposes reactive state (authenticated user, user list, settings, loading flag, error message) consumed by the* UI Layer *via* `context.watch`*.*

User Interface

The Paring Page, shown in Figure 3.15, is the very first screen presented when the application has no saved host URL. The user enters the middleware URL manually or taps Scan QR Code to open the `QrScannerDialog` *and populate the field automatically. On submission the URL is validated and persisted via* `AppConstants.changeHost()`*.*

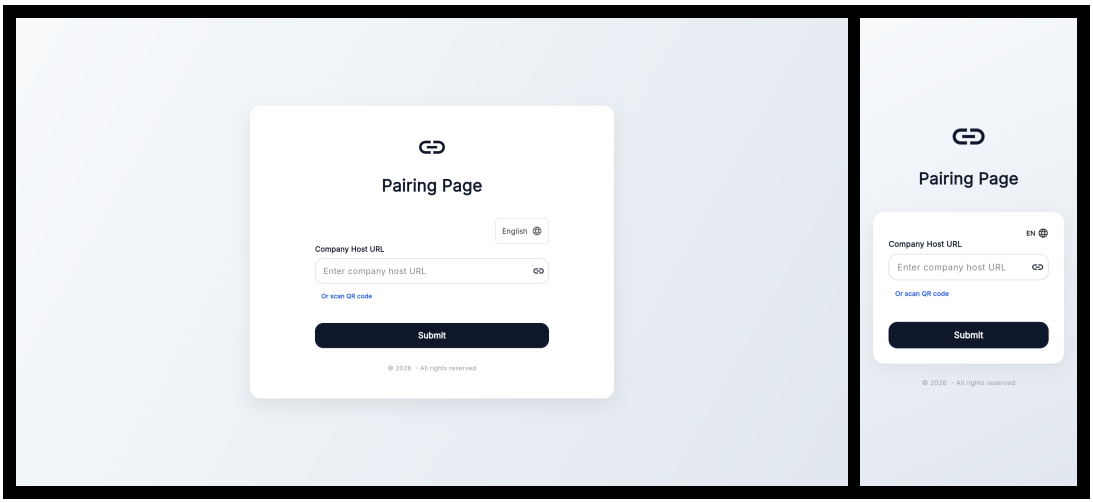

Figure 3.15. *Paring Page* — desktop and mobile views.

The Login Page, shown in Figure 3.16, is the application entry screen presented to all users on launch. It contains User ID and Password input fields and a Login button. Successful authentication redirects the user to the role-appropriate dashboard, or to the Badge Scan Dialog if 2FA

is enabled.

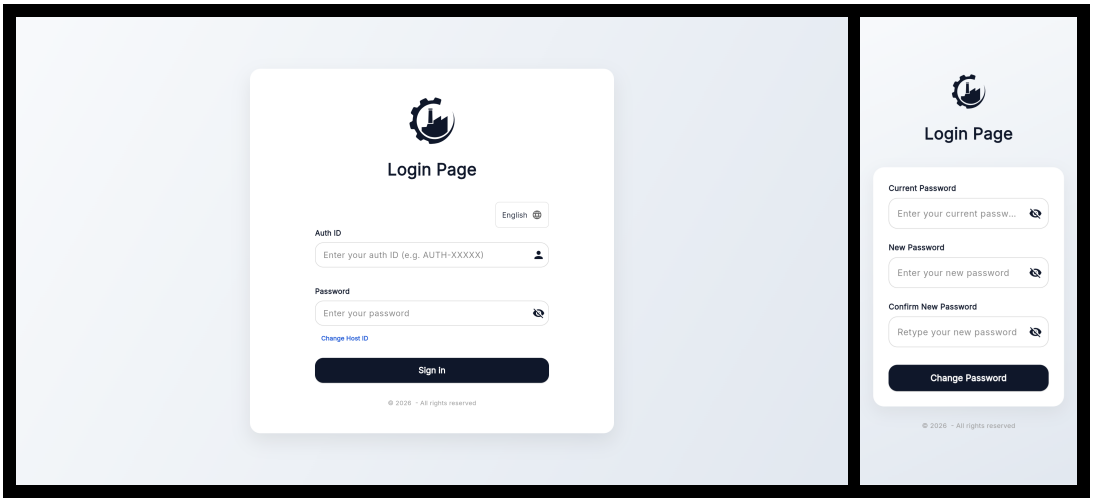

Figure 3.16. *Login Page* — desktop and mobile views.

The Badge Scan Dialog, shown in Figure 3.17, is displayed after a successful first-factor login when 2FA *is enabled. A live camera viewport reads the user's badge QR code; a manual entry field is available as a fallback. The dialog displays any verification error and re-enables the scanner on failure. The user can cancel at any time, which clears the pending session state and returns to the login page.*

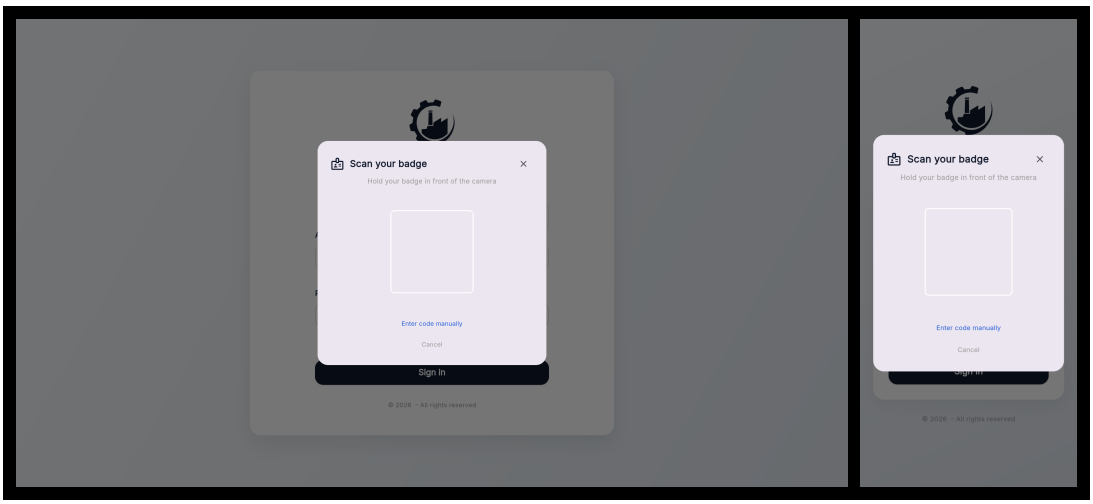

Figure 3.17. *Badge Scan Dialog* — desktop and mobile views.

The MES User List Page, shown in Figure 3.18, is the Administrator dashboard screen listing all registered MES *users with their assigned roles and active status, providing action buttons to assign roles, generate passwords, view or regenerate badges, and activate/deactivate each account.*

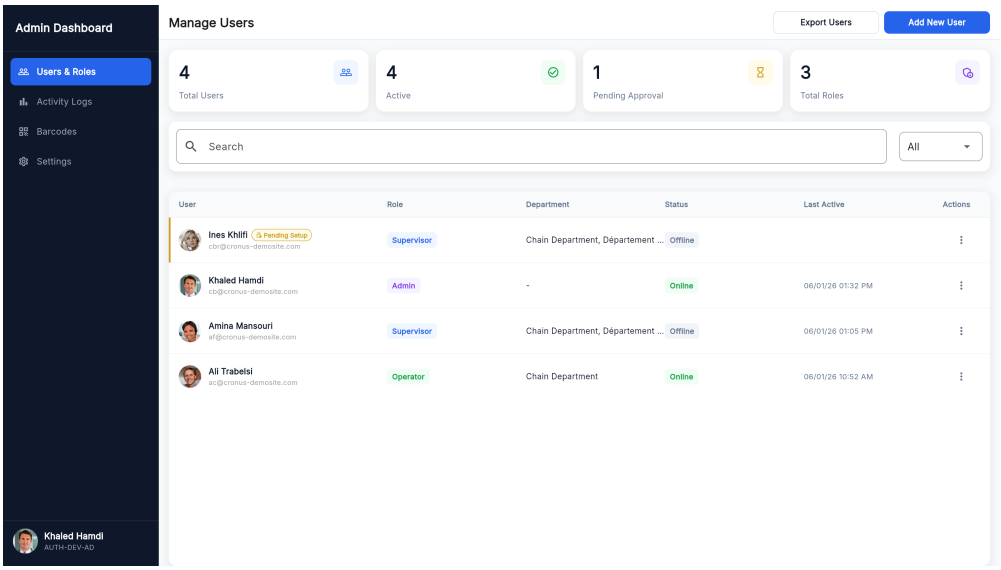

Figure 3.18. *MES User List Page.*

The Assign MES Role Dialog, shown in Figure 3.19, is a modal dialog allowing the Administrator to select a `BusinessCentral` *employee record and assign them an* MES *role (Operator, Supervisor, or Administrator), creating a new* `MES User` *record via the* `/createMesUser` *endpoint.*

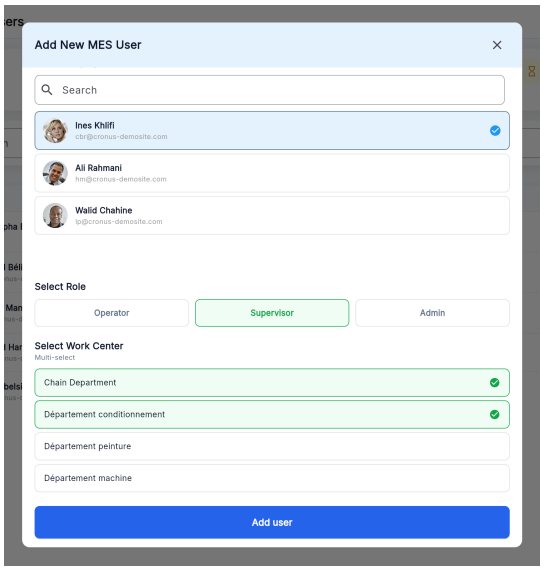

Figure 3.19. *Assign MES Role Dialog.*

The Generate Password Dialog, shown in Figure 3.20, is a modal dialog through which the Administrator triggers server-side secure password generation for a selected MES *user. The generated password is displayed once and must be communicated to the user out-of-band. The Password Generated Successfully Dialog, shown in Figure 3.21, is a confirmation dialog showing the plaintext password to the Administrator for one-time transmission to the user. The password is not stored in plaintext on the server after this point.*

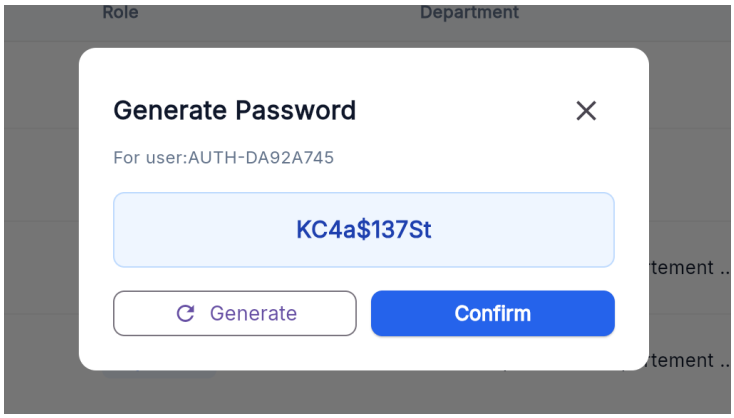

Figure 3.20. *Generate Password Dialog*

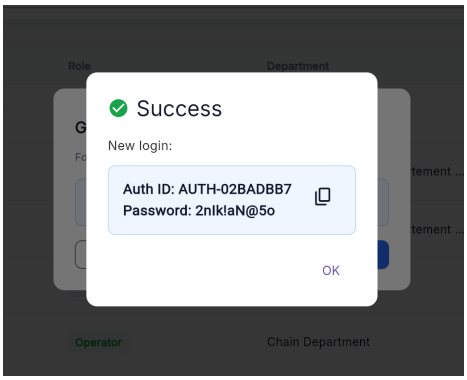

Figure 3.21. *Password Generated Successfully Dialog*

The Change Password Page, shown in Figure 3.22, is presented to users whose needChangePassword *flag is* `true` *after login, requiring them to enter their current password and choose a new password before accessing the main application.*

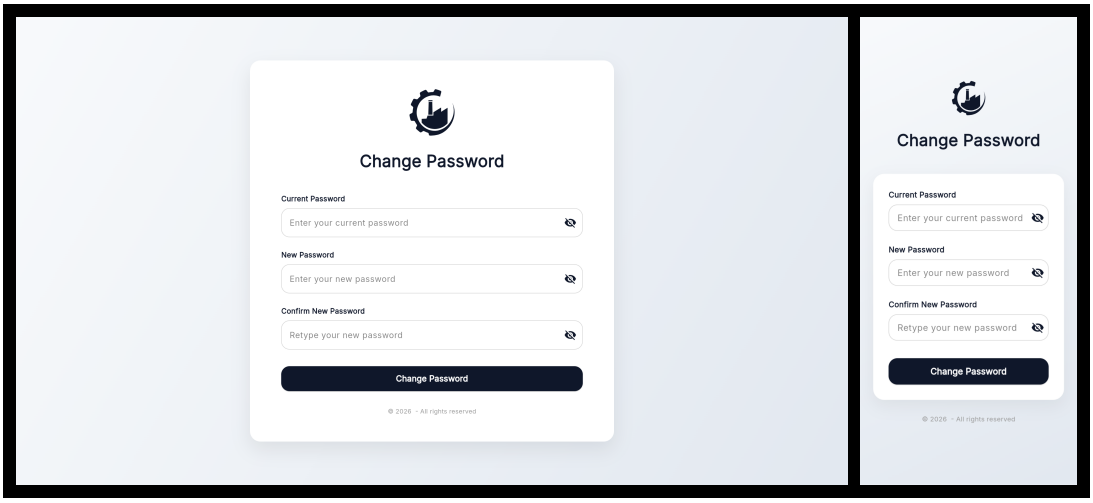

Figure 3.22. *Change Password Page* — desktop and mobile views.

The Change Role Dialog, shown in Figure 3.23, is a modal dialog through which the Administrator updates the role and work-centre assignment of an existing MES *user. The dialog pre-populates the current role and presents a three-button row (Operator / Supervisor / Admin); selecting Admin hides the work-centre list, selecting Operator shows a single-select list, and selecting Supervisor shows a multi-select list. An inline error banner displays any server-side validation failure. Tapping Save Changes submits the update and revokes all active tokens for the affected user.*

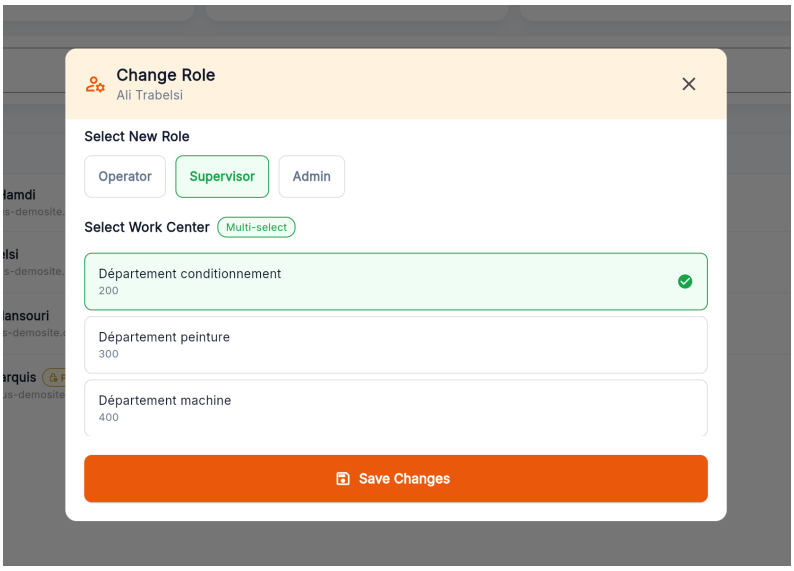

Figure 3.23. *Change Role Dialog.*

The User Badge Dialog, shown in Figure 3.24, allows the Administrator to view a user's badge QR code, print it as a PDF, and regenerate the underlying secret after confirmation. It is accessible via the View Badge action in the user row action menu and is available only for active accounts.

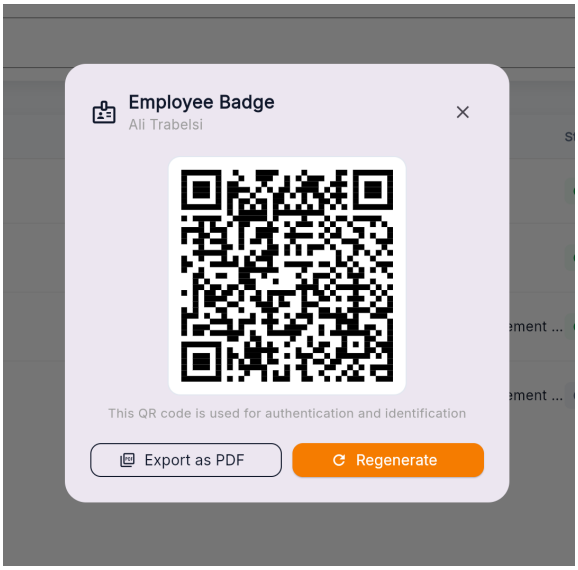

Figure 3.24. *User Badge Dialog.*

The Settings Page, shown in Figure 3.25, is accessible from the Administrator sidebar. It exposes two controls: a numeric field for the password rotation period (in days) and a toggle for global 2FA. *An animated action banner appears at the bottom of the page whenever the values differ from the last saved state, offering Discard and Save actions.*

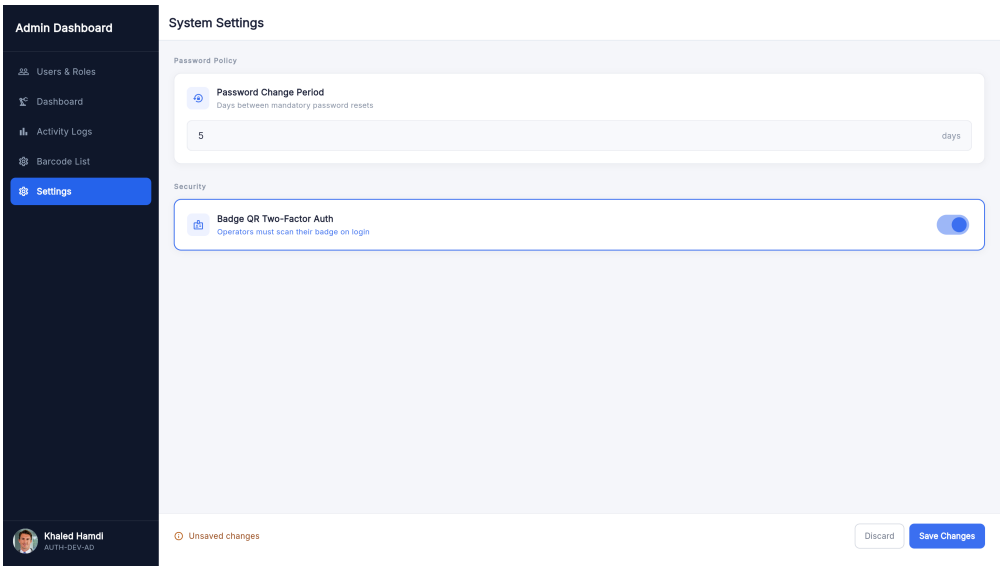

Figure 3.25. *Settings Page.*

3.4 Testing and Validation

Integration testing

For integration testing, we chose Postman, a powerful and user-friendly tool for API evaluation. Figure 3.26 shows a test case for the `/mesUsers` *endpoint.*

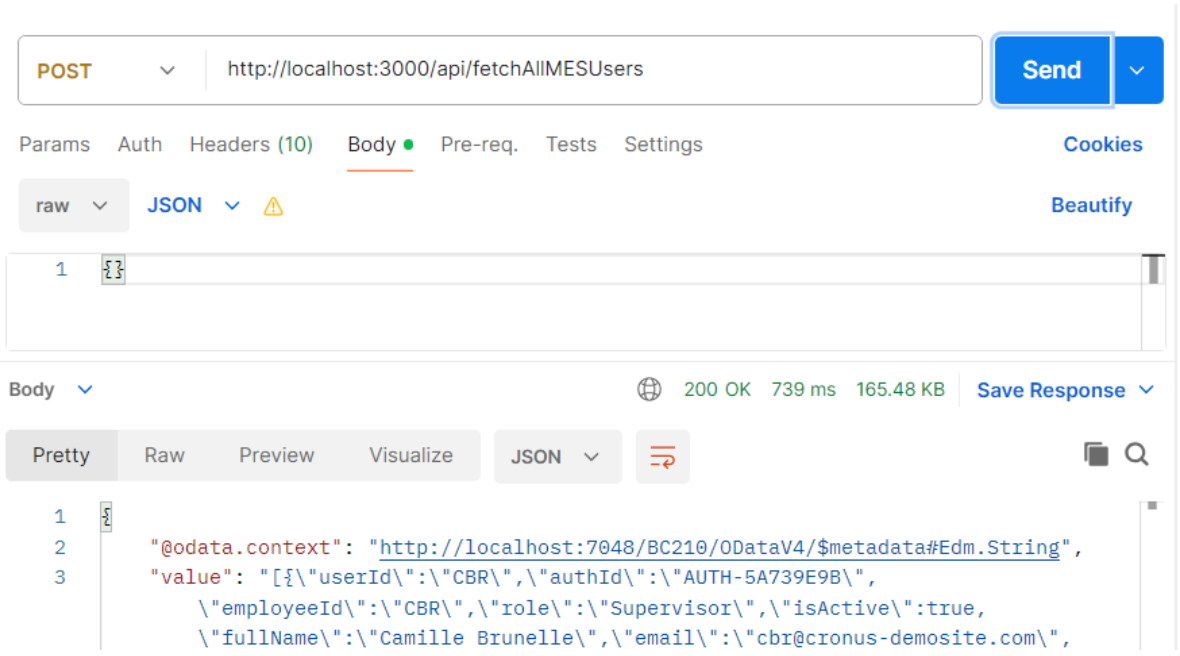

Figure 3.26. Postman endpoint test — fetchAllMesUsers.

System testing

Figures 3.27 and 3.28 illustrate the validation results of the sprint's implemented features, capturing both nominal execution paths and system responses to invalid or unauthorized actions.

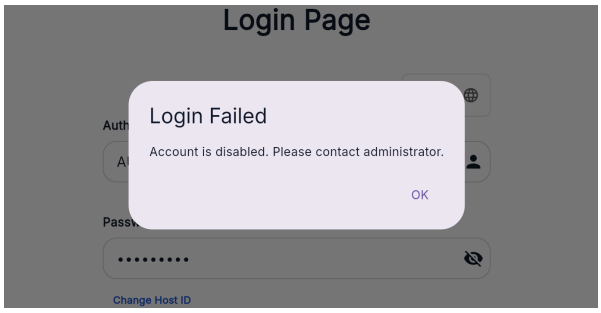

Figure 3.27. Validation dialog — Disabled Account.

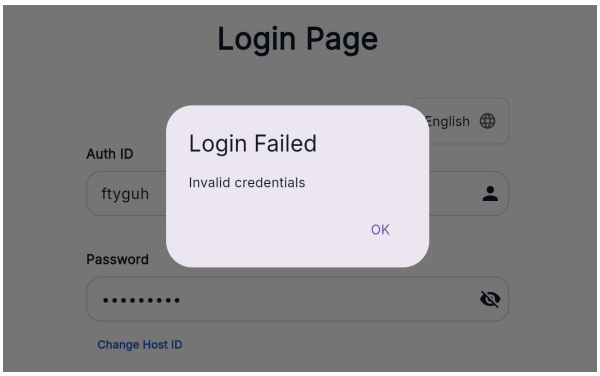

Figure 3.28. Validation dialog — Invalid Credentials.

Conclusion

This sprint successfully delivered a complete authentication and user management foundation for the MES *application. The backend, built in* `MicrosoftDynamicsBusinessCentral` *using* `AL`*, implements secure credential validation,* SHA-256 *password hashing, session token management with automatic expiry, optional two-factor authentication via badge QR code scanning, a configurable password expiry policy enforced by a scheduled background job, and a full set of* RESTful *API endpoints exposed through the Node middleware that forwards them to the* `MESWebService` *(codeunit 50126). The* `Flutter` *frontend provides a clean, role-aware user experience covering first-launch host pairing, login, badge QR verification, logout, forced password change, administrator-level user creation, badge management (view, print, regenerate), role and work-centre management, account activation control, and security policy configuration.*

Chapter 4

Sprint 2 Machine & Production Management

Contents

Introduction

*This sprint covers **Machine & Production Management** in the* MES *application, giving Operators and Supervisors real-time visibility into machine states and a full audit trail of completed operations.*

It addresses three related sub-domains: live machine status tracking from `BusinessCentral` *to the* `Flutter` *frontend, production order management with operation lifecycle initiation, and a history view showing all completed,interrupted and cancelled operations per machine.*

4.1 Sprint Backlog

*This sprint covers **Domain 2:*** **Machine & Production Management**.

Table 4.1. Sprint Backlog

ID	*User Story*	*Tasks*
US-2.1	*As an Operator or Supervisor, I need to see the list of machines in my work centre with their current status so that I can see which machines are currently active.*	*Business Central (AL):* • *Create table* `MESMachineStatus`. • *Create enum* `MESMachineStatus`. • *Create function* `FetchMachines()` *in codeunit* `MESMachineFetch`. • *Expose* `FetchMachines()` *through codeunit* `MESWebService`. • *Create page* `MESMachineList`. *Flutter:* • *Create model* `MachineModel`. • *Create service* `MESMachineListService` *to consume the* `FetchMachines()` *endpoint.* • *Create provider* `MesMachinesProvider`. • *Create page* `MachineListPage`. • *Create widget* `MachineCard`. • *Implement automatic machine list refresh in* `MESMachineListService`.
US-2.2	*As an Operator or Supervisor, I need to view the operations of the production orders assigned to a machine so that I know what work needs to be done.*	*Business Central (AL):* • *Create function* `getMachineOrders()` *in codeunit* `MESMachineFetch`. • *Expose* `getMachineOrders()` *in codeunit* `MESWebService`.

Continued on next page

ID	User Story	Tasks
		Flutter: • *Create model* `MachineOrderModel`. • *Create service* `ErpMachineOrdersService` *to consume the* `getMachineOrders()` *endpoint.* • *Create provider* `MachineOrdersProvider`. • *Create page* `MachineOrderPage`.
US-2.3	*As an Operator or Supervisor, I need to start a production operation so that the system records that the order has begun.*	*Business Central (AL):* • *Create table* `MESOperationState`. • *Create enum* `MESOperationStatus`. • *Create function* `TryStartOperation()` *in codeunit* `MESMachineValidation`. • *Create function* `InsertMESOperation()` *in codeunit* `MESMachineInsert`. • *Expose* `startOperation()` *through codeunit* `MESWebService`. • *Create page* `MESOperations`. *Flutter:* • *Implement* `ErpMachineOrdersService` *that Consumes the* `startOperation` *endpoint.* • *Implement* `MachineOrdersProvider` *to manage its state.* • *Implement start operation action handling in* `ActionButtons`. • *Implement automatic navigation to* `Orders-ProgressionPage`
US-2.4	*As a Supervisor or Operator, I need to view all active operations on a machine so that I can follow the progress of each order.*	*Business Central (AL):* • *Create function* `fetchOperationsStatusAndProgress()` *in codeunit* `MESMachineFetch`. • *Expose* `fetchOngoingOperationsState()` *through codeunit* `MESWebService`. *Flutter:* • *Create model* `OperationStatusAndProgressModel`. • *Update* `ErpMachineOrdersService` *to Consume the* `fetchOngoingOperationsState()`.

Continued on next page

ID	User Story	Tasks
		• *Create model* `OperationStatusStyle`. • *Create page* `OrdersProgressionPage`. • *Create the widgets* `OperationCard`, `OperationStatusBadge`, *widget* `OperationInfoGrid`, *and* `OperationProgressBar`. • *Implement automatic refresh for operation status monitoring.*
US-2.5	*As an Operator or Supervisor, I need a tabbed view for a machine so that I can navigate between pending orders, active operation progress, and the operation history of the machine without leaving the machine context.*	*Flutter:* • *Create page* `MachineMainPage`. • *Create the widgets* `TopNavigationBar` *and* `NavButton`. • *Implement tab navigation in* `MachineMainPage`. • *Implement navigation from* `MachineCard` *to* `MachineMainPage` *which displays the selected machine's operations.*
US-2.6	*As an Operator or Supervisor, I need to filter and sort production orders so that I can quickly find the most relevant work.*	*Flutter:* • *Create widget* `GlobalSearchBar`. • *Implement order search, filtering, and sorting in* `MachineOrderPage`.
US-2.7	*As a Supervisor or Operator, I need to view the history of completed operations for a machine so that I can audit past production.*	*Business Central (AL):* • *Exposed the function* `fetchOperationsHistory` *that calls the function* `fetchOperationsStatusAndProgress` *in codeunit* `MESWebService`. *Flutter:* • *Update* `ErpMachineOrdersService` *to consume the* `fetchOperationsHistory` *endpoint.* • *Update* `MachineordersProvider` *to manage its state.* • *Create page* `MachineHistoryPage`. • *Create the widgets* `HistoryCard`, `HistoryInfoGrid`, *and* `HistoryBadgeAndId`. • *Implement history search and sorting in* `MachineHistoryPage`.

Continued on next page

ID	*User Story*	*Tasks*
		• *Implement navigation from* `HistoryCard` *to* `OperationDetailPage`.

4.2 Design

4.2.1 Use Case Diagram

The UML Use Case Diagram (Figure 4.1) shows the two primary actors (Operator and Supervisor) and their interactions with production order management, and history subsystem. In addition to viewing machines, browsing orders, and managing operation lifecycle transitions, the diagram includes the View Operation History use case, which allows both actors to access the finished, interrupted and cancelled operation log for any machine in their work centre.

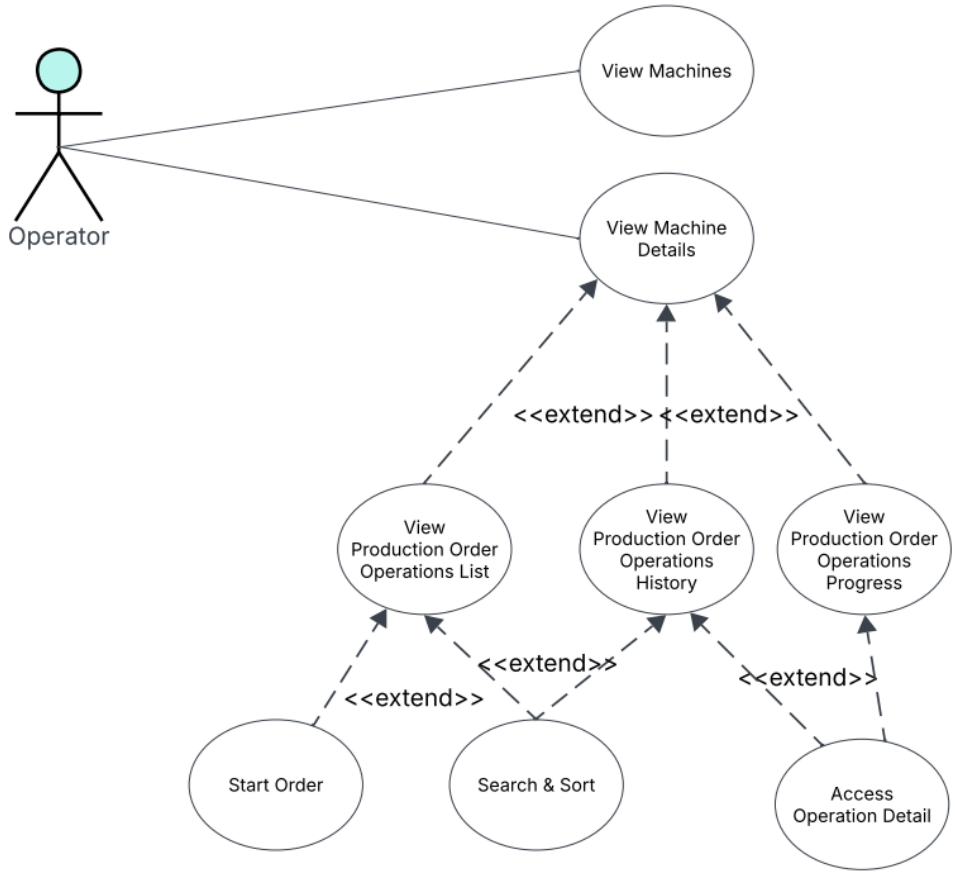

Figure 4.1. UML Use Case Diagram — Machine & Production Management.

4.2.2 Textual Use Case Description

The following table (Table 4.2) provides the detailed textual description of the Start Operation use case.

Table 4.2. Textual Use Case Description — Start Operation

Use Case Name	*Start Operation*
Primary Actor	*Operator*
Preconditions	• *The Operator is authenticated and has a valid session token.* • *The target routing line has status* Released. • *No other operation is currently* Running *on the same machine.* • *The previous operation in the same production order (if any) must be Finished, Interrupted, or currently Running with enough send-ahead quantity*
Postconditions	• *A new* `MES Operation State` *record is inserted with status* Running *and the current* UTC *timestamp.* • *The Operations Progress tab reflects the new running operation within the next polling cycle.* • *If any precondition fails, a descriptive error dialog is shown to the operator and no record is inserted.*
Main Scenario	1. *The Operator navigates to the Machine List Page and selects a machine.* 2. *The system displays the Machine Orders Page listing all applicable production orders.* 3. *The Operator locates a* Released *order and taps Start Order.* 4. *The frontend sends a* POST *request to* `MESWebService_startOperation`. 5. *The backend validates all preconditions.* 6. *On success, a* `MES Operation State` *record is inserted.* 7. *The frontend navigates to the Orders Progression Page.*

4.2.3 Sequence Diagrams

The following sequence diagrams illustrate the communication flow between the `Flutter` *frontend, the* `BusinessCentral` *OData layer, and the* `AL` *business logic for the machine-listing and order-management flows introduced in Sprint 2.*

Figure 4.2 depicts the machine-list polling flow. The `Flutter` *client issues a* `FetchMachines` *request for each work centre resolved from the authenticated user's profile every twenty seconds. For every machine returned by* `BusinessCentral`*, the* `AL` *layer fetches the latest* MES Machine Status *row and the work-centre name before assembling the response, which the client uses to*

rebuild the MachineCard grid.

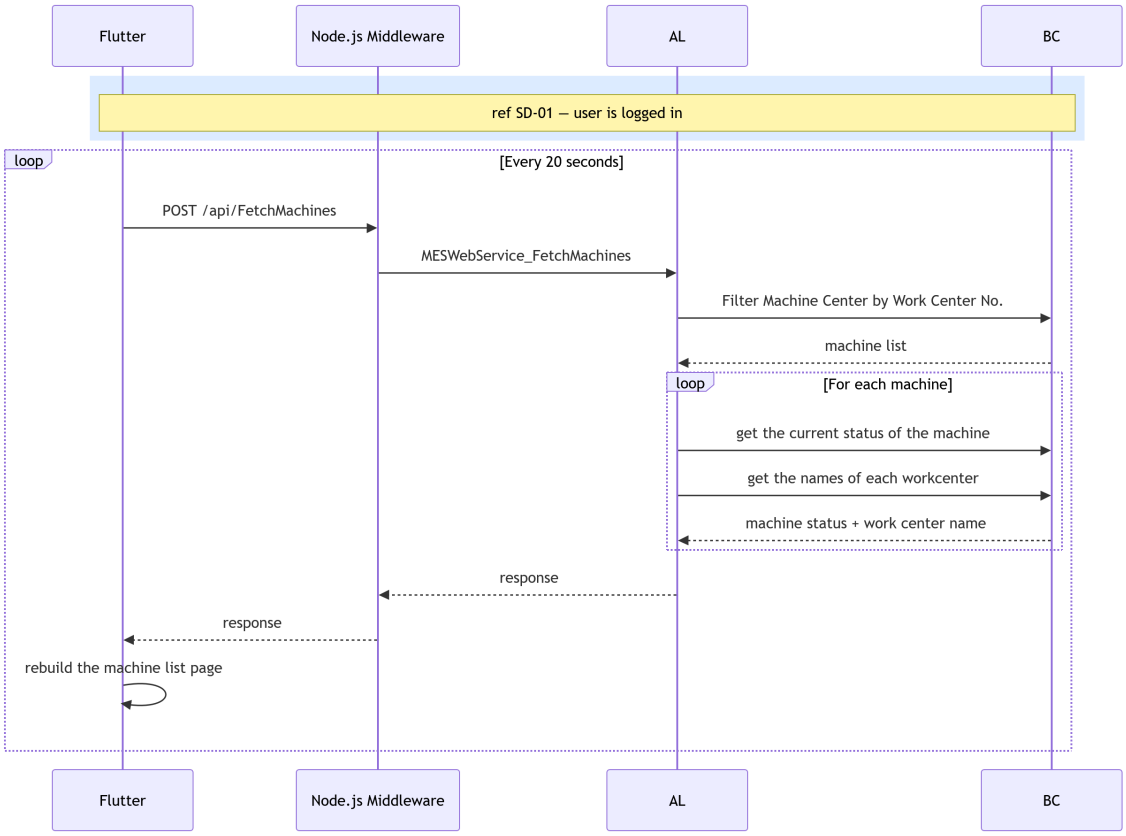

Figure 4.2. SD-09: Fetch Machines.

Figure 4.3 presents the machine-orders retrieval flow, triggered when the user opens the Orders tab. The `AL` *layer filters* ProdOrderRoutingLine *records for the selected machine, skips any operation that has already been started, and enriches each remaining routing line with the corresponding* ProdOrderLine *details before returning the list to the* `Flutter` *client as a set of OrderCard entries.*

Figures 4.4 and 4.5 present the two flows that supply real-time and historical data to the MachineDetailPage. Figure 4.4 describes the ongoing-operations state polling flow, in which the `AL` *layer retrieves all active operations for the machine, resolves their current state and cumulative produced quantity from* `BusinessCentral`*, and returns the aggregated data to the* `Flutter` *client. Figure 4.5 presents the operation history retrieval flow, triggered when the user opens the MachineDetailPage, whereby the* `AL` *layer queries* `BusinessCentral` *for finished and cancelled operations associated with the machine, enriches each record with production quantities and start and end times, and returns the assembled list so the* `Flutter` *client can display the operations history.*

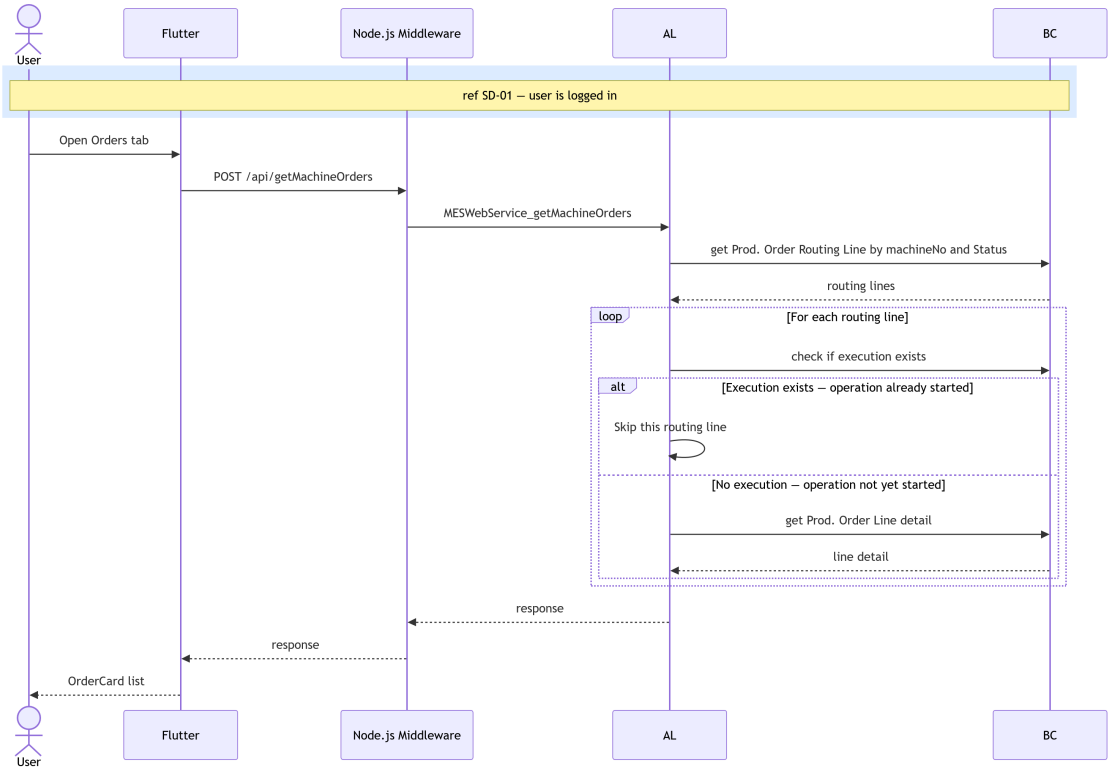

Figure 4.3. SD-10: Get Machine Orders.

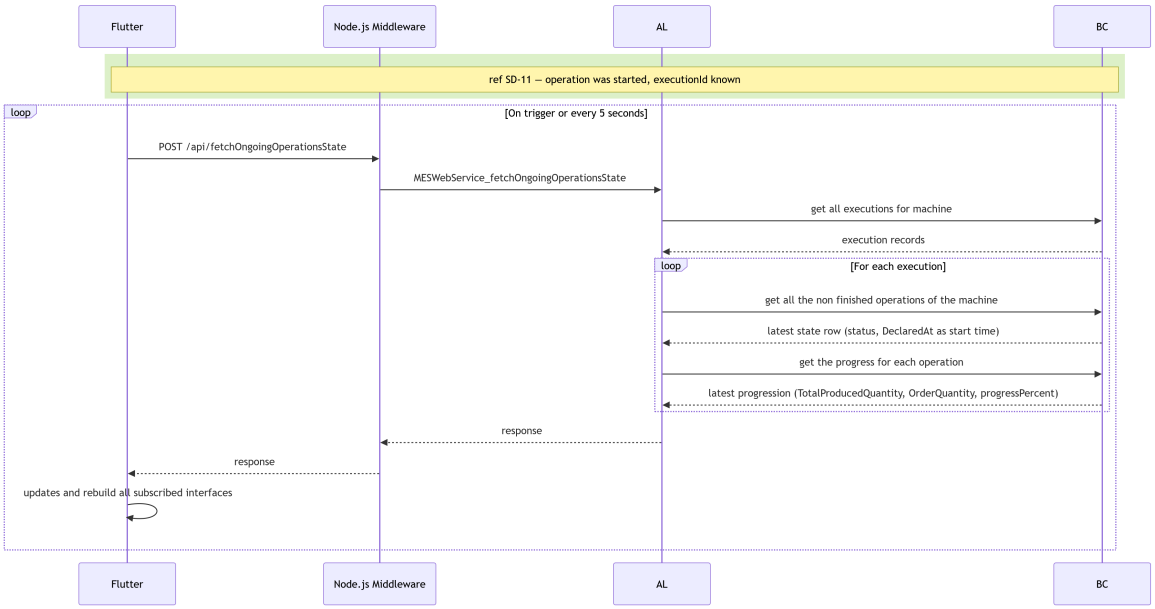

Figure 4.4. SD-23: Fetch Operation progress.

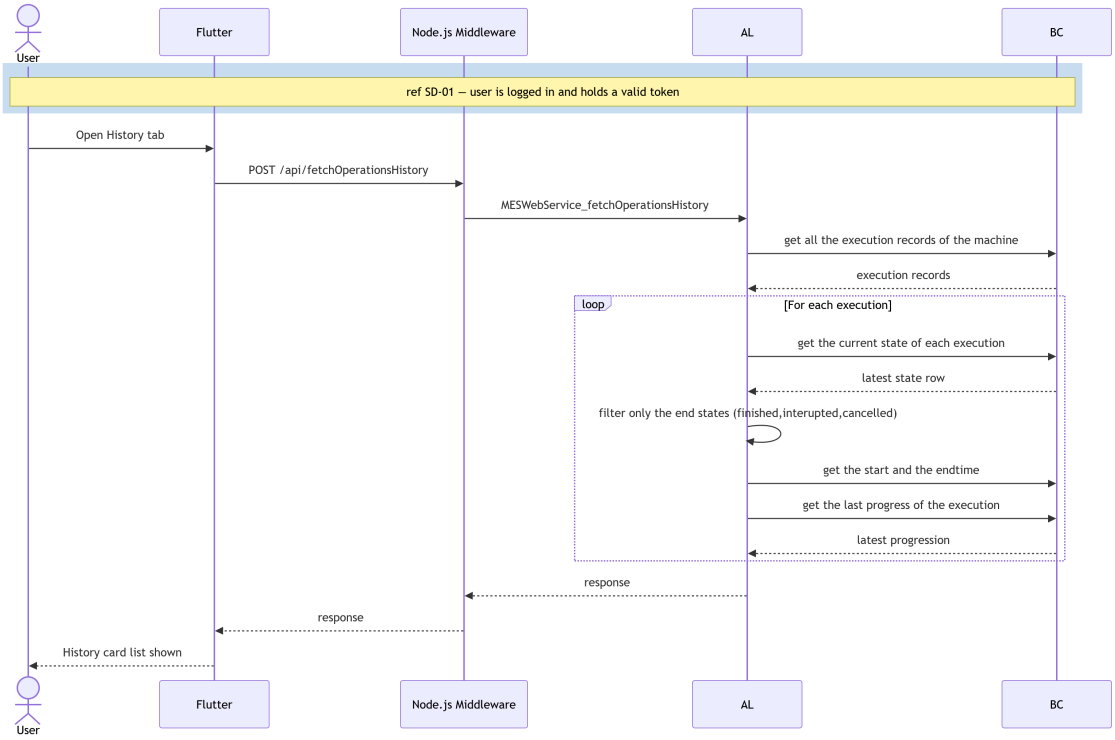

Figure 4.5. SD-24: Fetch Operation History

4.2.4 Class Diagram

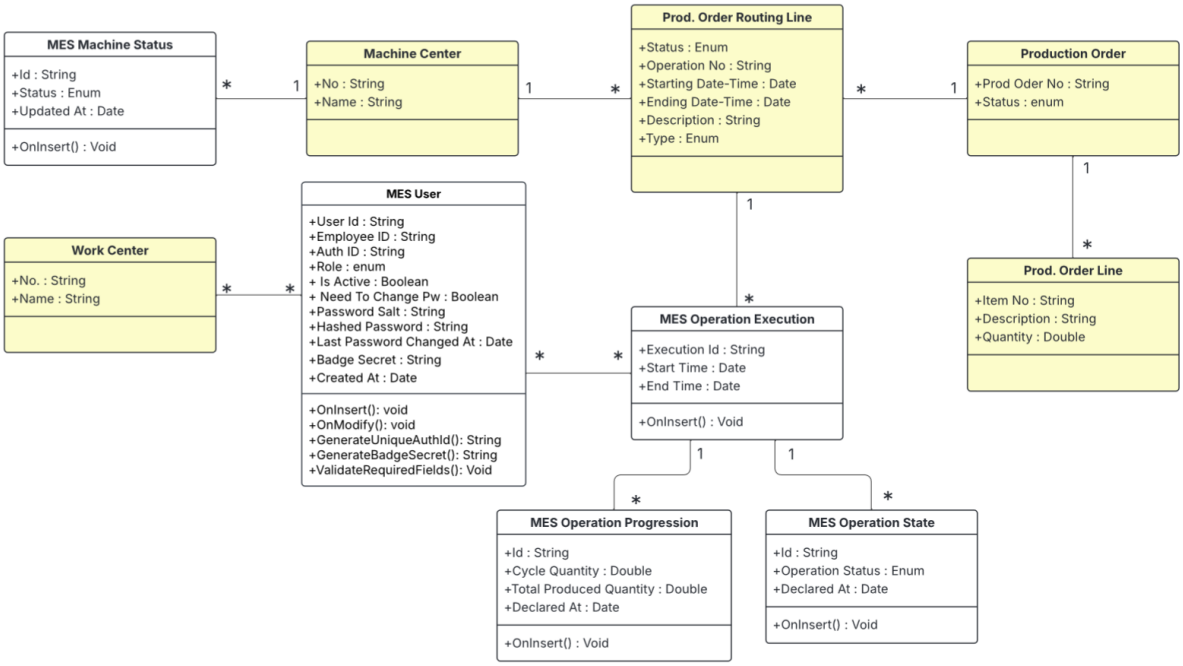

Figure 4.6. UML Class Diagram — Machine & Production Management.

The UML Class Diagram (Figure 4.6) of the machine and production management domain shows the `MES Machine Status` *and* `MES Operation State` *tables and their relationships*

with the `BusinessCentral` standard tables `Machine Center`, `Work Center` and `Prod. Order Routing Line`.

4.3 Implementation

4.3.1 Backend Implementation

Backend Structure Overview

The machines domain follows the same layered structure established in Sprint 1. Two new subfolders were added under `src/1-Tables/machines/` and `src/2-Enum/machines/`, and a new codeunit was placed under `src/3-CodeUnit/machines/`. The OData web service is published through the existing `WebServices.xml` mechanism under a new service name.

MES Machine Status Table

The `MES Machine Status` table (50107) stores the full history of machine state changes as an append-only log. Each insert creates a new record; the most recent record per machine represents the current state. Table 4.3 lists the fields and their descriptions.

Table 4.3. MES Machine Status (Table 50107) — Field Dictionary

Name	***Type***	***Description***
Id	*Code[50]*	*Primary key. GUID-derived value assigned automatically on insert if blank.*
Machine No.	*Code[20]*	*Foreign key to `MachineCenter."No."`. Identifies the machine this status record belongs to.*
Status	*Enum (`MES-Machine-Status`)*	*Current machine state:* Idle *or* Working.
Current Prod. Order No.	*Code[20]*	*Records which production order the machine is currently executing, if any.*
Updated At	*DateTime*	*UTC timestamp set automatically on insert. Used to find the most recent status record for a given machine.*

MES Operation State Table

The `MES Operation Status` table (50108) implements the same append-only pattern for operation lifecycle events. Every state transition (start, pause, resume, finish) inserts a new record; queries always sort by `Last Updated At` descending to obtain the current state. Table 4.4 provides the full field dictionary.

Table 4.4. MES Operation State (Table 50108) — Field Dictionary

Name	***Type***	***Description***
Id	*Code[50]*	*Primary key. GUID-derived identifier assigned automatically on insert.*
Execution Id	*Code[50]*	*Foreign key to* `MESOperationExecution`.
Operator Id	*Code[50]*	*The operator who triggered the state transition.*
Operation Status	*Enum (*`MES-Operation-Status`*)*	*Lifecycle state at the time of this record:* Running, Paused, Finished, Cancelled, *or* Interrupted.
Declared At	*DateTime*	*UTC timestamp set automatically on insert. Together with the execution index, this field is used to retrieve the current status via descending sort.*

Key points:

- *Both tables use an append-only design: no* `Modify` *is called on existing records.*
- *The* secondary index `ExecutionTimeline("ExecutionId","DeclaredAt")` *on* `MES Operation State` *supports efficient retrieval of the latest event per group.*

Enumerations

Two enumerations support the domain. `MES Machine Status` *defines two values:* Idle *and* Working. `MES Operation Status` *defines five values:* Running, Paused, Finished, Cancelled, *and* Interrupted. *Both enumerations are declared* `Extensible=true` *to allow future additions without modifying the base objects.*

Machine Actions Business Logic

Machine and operation business logic is encapsulated across three codeunits: `MES Machine Fetch` *for read operations and* `MES Machine Write` *and* `MES Machine Insert` *for write operations. Their public procedures are exposed by the OData web service layer.*

FetchMachines. *Accepts an array of work centres and returns their machines enriched with each machine's current status and active production order, defaulting to* Idle *when no status history exists.*

getMachineOrders. *Accepts a machine number and returns only the production routing lines that are eligible for assignment, those in a* Planned, Firm Planned, *or* Released *state, of type* Machine Center, *and not yet linked to an execution record. Each result is joined with its parent order line for item description and quantity.*

startOperation. *Accepts an order,token, operation, and machine number. Before inserting a new* Running *execution record, the procedure enforces three guards: the routing line must be* Released*, no running record may already exist for that operation, the state of the previous operation in that order allows it and the machine must be free. Any guard failure is returned as a structured* JSON *error.*

fetchOngoingOperationsState. *Returns the active operations for a machine, those whose most recent status entry is* Running *or* Paused*, deduplicated to one entry per (order, operation) pair.*

4.3.2 REST API and Web Services Implementation

Communication between `Flutter` *and* `BusinessCentral` *for the machines domain uses the same* OData V4 Unbound Actions *throught the middleware proxy. The procedures are exposed through the existing* `MESWebService` *registered in* `WebServices.xml`*. After deployment, the four procedures are reachable at the following endpoints:*

- `POST .../ODataV4/MESWebService_FetchMachines`
- `POST .../ODataV4/MESWebService_getMachineOrders`
- `POST .../ODataV4/MESWebService_startOperation`
- `POST .../ODataV4/MESWebService_fetchOngoingOperationsState`

All endpoints follow the same JSON *request/response convention used by the authentication layer: the request body carries named parameters as top-level* JSON *fields, and the response wraps the* `AL` *return value inside a* `"value"` *field which the* `Flutter` *service layer double-decodes.*

4.3.3 Frontend Implementation

*The machines domain retains the same four-layer call stack introduced in Sprint 1 (*Model → Service → Provider → UI*).*

Model Layer

Two models were introduced. `MachineModel` *(*`mes_machine_model.dart`*) holds the info displayed in the machine cards in machineList page, populated from the* `FetchMachines` *response.* `MachineOrderModel` *(*`erp_order_model.dart`*) holds the full set of production order fields returned by* `getMachineOrders`*.*

Service Layer

Two service classes handle all network communication for the machines domain. MESMachineListService (mes_MachineList.dart) exposes fetchMachines for retrieving and deserialising the machine list for a given work centre, and streamMachines for polling that endpoint every 20 seconds, emitting an empty list on error to keep the StreamBuilder stable. ErpMachineOrdersService (erp_order_service.dart) covers the order and operation lifecycle through

three methods: fetching the order list, triggering and validating a start-operation request, and retrieving active operation records. A companion streamMachines generator applies the same polling pattern at a 5-second interval for the operations progress view.

State Management Layer

Two providers were introduced. `MesMachinesProvider` *wraps the* `MESMachineListService` *stream and exposes it directly to the UI, no* `ChangeNotifier` *is needed since the stream itself carries all state.* `MachineOrdersProvider` *follows the standard* `ChangeNotifier` *pattern and manages both machine order retrieval and order start logic, exposing the operation status polling stream to the UI layer.*

User Interface

The Machine List Page is the entry point for operators after login. It displays a live-updating grid (tablet/desktop) or list (mobile) of machine cards scoped to the authenticated work centre, refreshing every 20 seconds. Each card shows the machine name, its current status with a colour-coded badge, and the active production order number. The layout adapts across breakpoints. As illustrated in Figure 4.7, the interface is consistent across both desktop and mobile platforms.

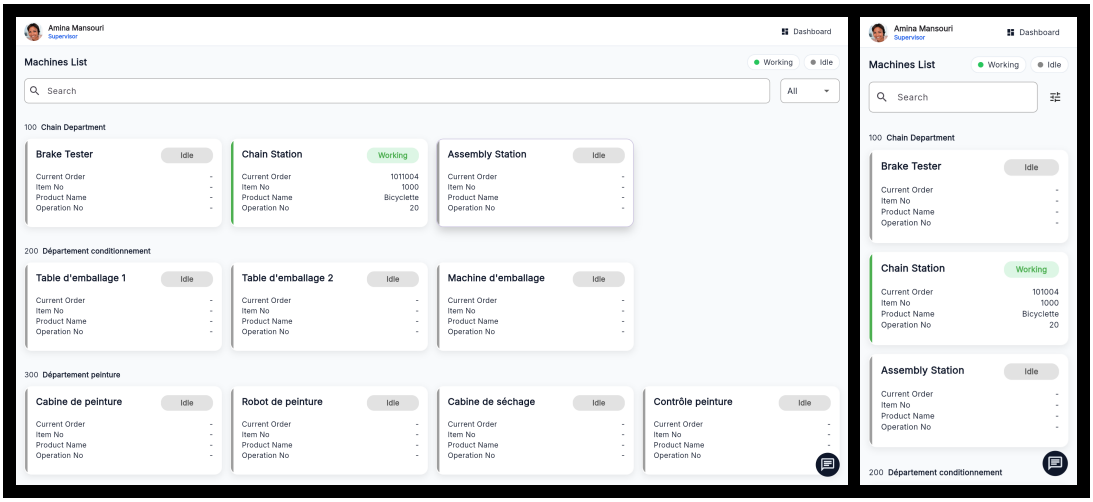

Figure 4.7. *Machine List Page* — desktop and mobile views.

The user interface for the machines domain is split across two top-level pages accessible via a shared Top Navigation Bar with tabs for Orders, Progress, and History.

The Machine Orders Page displays the production routing lines assigned to the selected machine. A Global Search Bar allows filtering by free text (order number, item description, or date), by status, and sorting by planned start date. Order cards use a responsive layout that switches between a stacked narrow layout *and a side-by-side* wide layout*. Only* Released *orders display an active Start Order button;* Planned *and* Firm Planned *orders show it disabled. Figure 4.8 illustrates both the desktop and mobile views.*

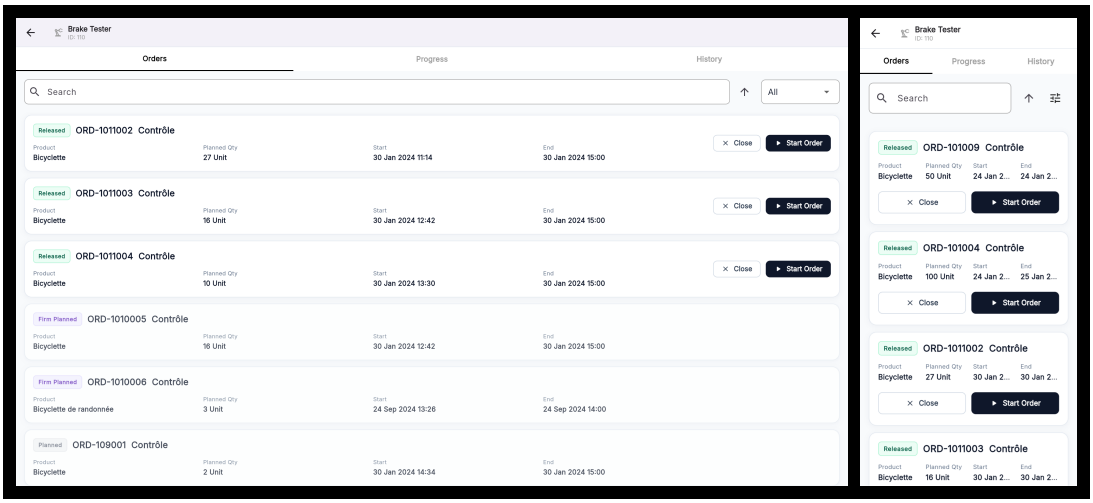

Figure 4.8. *Machine Orders Page* — desktop and mobile views.

The Orders Progression Page displays the currently active operations on a machine, polling every 5 seconds. Each operation card shows a colour-coded status badge (green for Running*, amber for* Paused*), the order and operation identifiers, the last updated timestamp, and a progress bar. The card layout also switches between narrow and wide variants. Pause and Resume action buttons adapt their colour and label to the current operation status, as shown in Figure 4.9.*

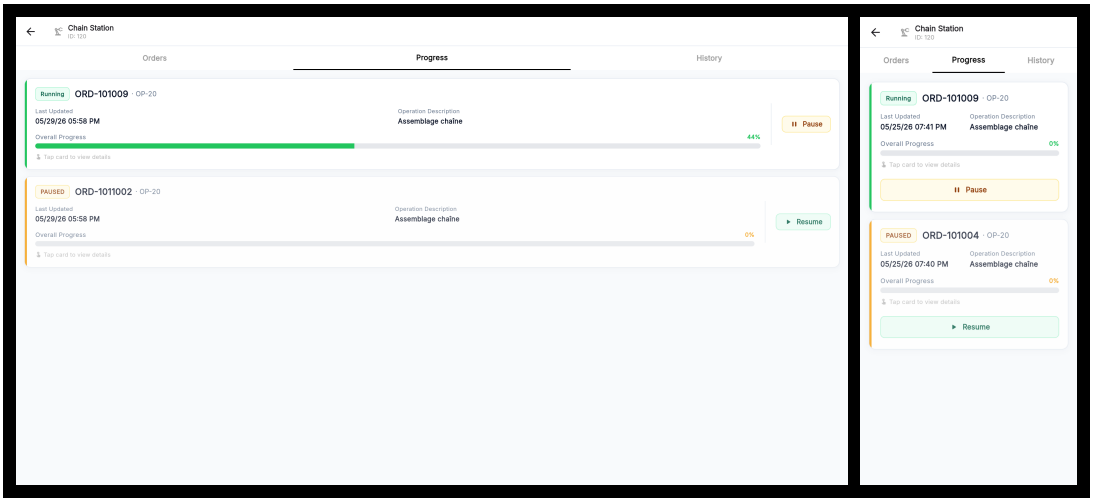

Figure 4.9. *Orders Progression Page* — desktop and mobile views.

The Machine History Page (Figure 4.10) displays completed operations assigned to the selected machine, limited to Finished*,* Interrupted*, and* Cancelled *statuses. It shares the same layout and filtering behaviour as the Machine Orders Page, including free-text search, and sort controls — but omits the Start Order button. Tapping an operation card navigates to its Operation Detail Page for review. The Machine History Page is accessible via the* History *tab in the* `TopNavigationBar`*. Figure 4.10 shows the desktop and mobile views.*

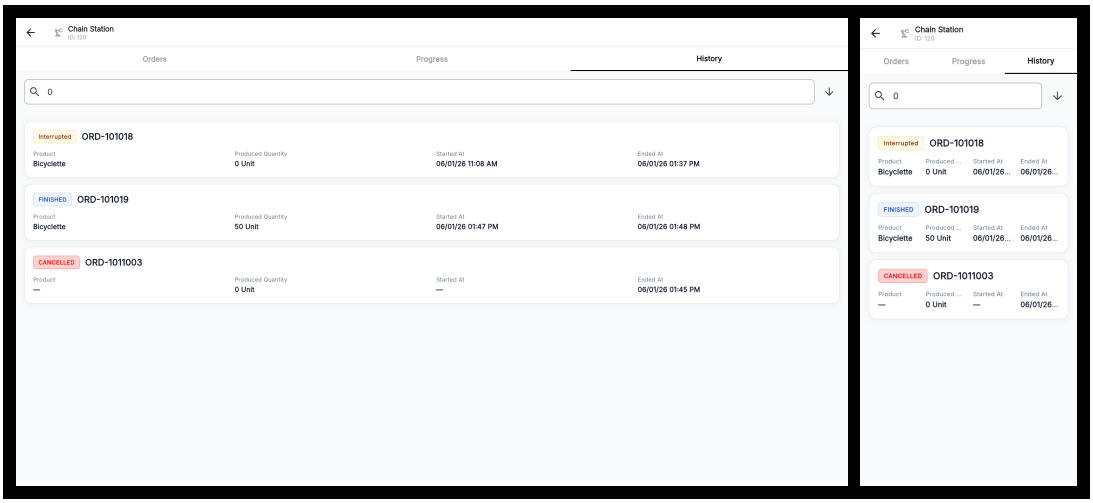

Figure 4.10. *Machine History Page* — desktop and mobile views.

4.4 Testing and Validation

Integrations tests

For integration testing, we chose Postman, a powerful and user-friendly tool for endpoint evaluation. Figure 4.11 shows a test case for the `/fetchMachines` *endpoint.*

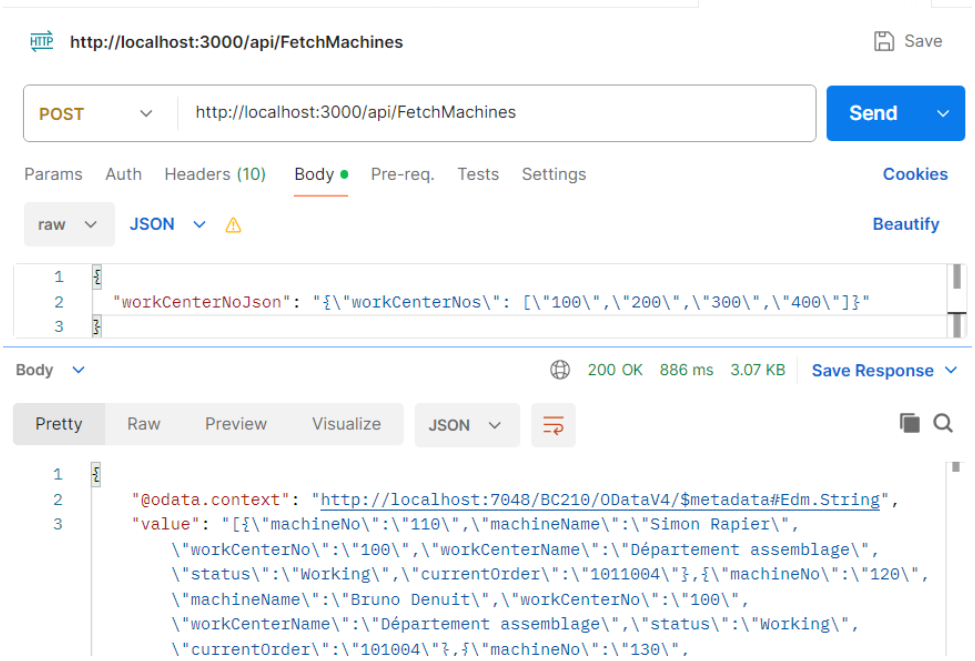

Figure 4.11. Postman endpoint test — fetchMachines.

Figure 4.12 shows a test case for the `/getMachinesOrders` *endpoint.*

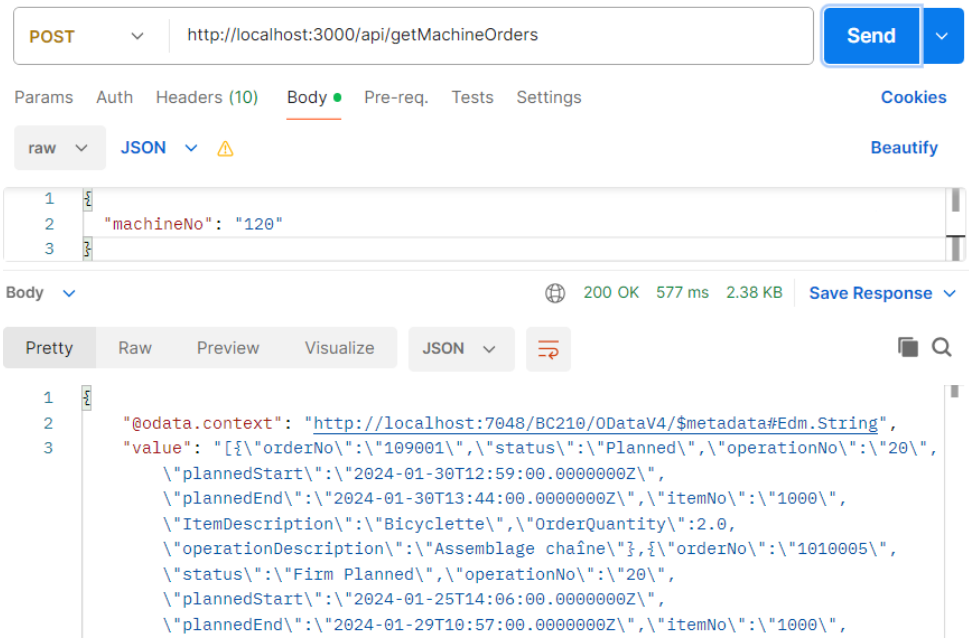

Figure 4.12. Postman endpoint test — getMachinesOrders.

System testing

Figures 4.13 and 4.14 illustrate the validation results of the sprint's implemented features, capturing both nominal execution paths and system responses to invalid or unauthorized actions.

Figure 4.13. Validation dialog — Already Running.

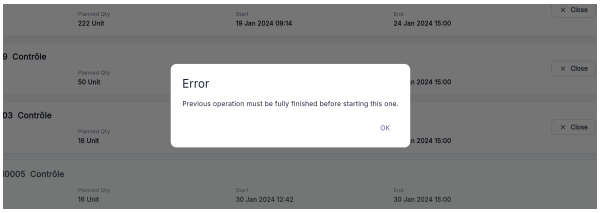

Figure 4.14. Validation dialog — Previous Operation.

Conclusion

This sprint successfully delivered the machine monitoring and production order management foundation of the MES *application.On the backend, two new append-only tables (`MES Machine Status` and `MES Operation State`) and dedicated codeunits (`MES Machine Fetch`, `MES Machine Validation` and `MES Machine Write`) were implemented in `BusinessCentralAL`, exposing five* OData V4 *endpoints that cover the full machine and operation lifecycle. The append-only design ensures a complete, tamper-evident audit trail of every state transition without requiring any record modification.*

Chapter 5

Sprint 3 Production Execution & Traceability

Contents

Introduction

*This sprint report covers **Domain 3: Production Execution & Traceability**. Building upon the authentication and machine monitoring infrastructure established in the previous sprints, this sprint delivers the core production execution features; declaring production output, pausing and resuming operations, finishing, Interrupting or cancelling orders, tracking production declarations over time, monitoring Bill of Materials consumption, component barcode scanning, scrap declaration, supervisor proxy declaration, and label printing. The result is a fully operational shop-floor execution loop from order assignment through to order closure, material traceability, and printed part identification.*

5.1 Sprint Backlog

*This sprint covers **Domain 3:** **Production Execution & Traceability**.*

Table 5.1. Sprint Backlog

ID	*User Story*	*Tasks*
US-3.1	*As an Operator or Supervisor, I need to declare the produced quantity in a cycle so that the system records my production progress.*	*Business Central (AL):* • *Create table `MESOperationProgression`.* • *Create the functions:* – *`TryDeclareProduction()` in codeunit `MESMachineValidation`.* – *`InsertNewProgressionCycle()` in codeunit `MESMachineInsert`.* – *`declareProduction()` in codeunit `MESMachineWrite`.* • *Expose `declareProduction()` through the codeunit `MESWebService`.* • *Create page `MESOperationProgression` list page.* *Flutter:* • *Updated `ErpMachineOrdersService` to consume the `declareProduction` endpoint and `MachineOrdersProvider` to manage it.* • *Create widget `DeclareProduction-Dialog`.* • *Add the `DeclareProduction` action in `ActionButtonsContainer`.*

Continued on next page

ID	***User Story***	***Tasks***
US-3.2	*As an Operator or Supervisor, I need to pause and resume an active operation so that production interruptions are tracked accurately.*	*Business Central (AL):* • *Create the functions:* – *`TryPauseOperation()` and `TryResumeOperation()` in codeunit `MESMachineValidation`.* – *`ExecuteOperationTransition()`, `pauseOperation()`, and `resumeOperation()` in codeunit `MESMachineWrite`.* • *Expose `pauseOperation()` and `resumeOperation()` throught codeunit `MESWebService`.* *Flutter:* • *Implement `ErpMachineOrdersService` to consume the new endpoints* • *Implement `MachineOrdersProvider` to manage its state.* • *Create widget `OperationActionButtons`.* • *Implement pause/resume operation toggle handling in `OrdersProgressionPage`.*
US-3.3	*As a Supervisor, I need to finish, interrupt or cancel a production operation so that the machine is released and the order is closed in the system.*	*Business Central (AL):* • *Create the functions:* – *`TryCloseOperation()` in codeunit `MESMachineValidation`.* – *`InsertOperationStatus()`, `SetErpOrderToFinish()`, `InsertStartMESMachineStatus()` and `InsertIdleMachineStatus()` in codeunit `MESMachineInsert`.* – *`finishOperation()` and `cancelOperation()` in codeunit `MESMachineWrite`.* – *`ExecuteCancelUnstartedOperation()` in codeunit `MESMachineWrite`.*

Continued on next page

ID	User Story	Tasks
		• *Expose `finishOperation()` and `cancelOperation()` in codeunit `MESWebService`.* *Flutter:* • *Update `ErpMachineOrdersService` to consume the new endpoints.* • *Update `MachineOrdersProvider` to manage its state.* • *Update `ActionButtonsContainer` to handle Finish/Interrupt operation flow.*
US-3.4	*As an Operator or Supervisor, I need a dedicated operation detail page so that I can see all real-time information in one place.*	*Business Central (AL):* • *Create table `MESOperationExecution`.* • *Create function `InsertMESOperationExecution()` in codeunit `MESMachineInsert`.* • *Create function `fetchOperationLiveData()` in codeunit `MESMachineFetch`.* • *Expose `fetchOperationLiveData()` through codeunit `MESWebService`.* • *Create page `MESOperationExecution`.* *Flutter:* • *Update `ErpMachineOrdersService` to consume the new endpoints.* • *Update `MachineOrdersProvider` to manage its state.* • *Create page `OperationDetailPage`.* • *Create widget `OperationAppBar`.* • *Create widget `CurrentOrderInfoContainer`.*
US-3.5	*As an Operator or Supervisor, I need to see all production cycles declared for an operation so that I can review each declaration's information.*	*Business Central (AL):* • *Create function `fetchProductionCycles()` in codeunit `MESMachineFetch`.* • *Expose `fetchProductionCycles()` throught codeunit `MESWebService`.* *Flutter:* • *Create model `ProductionCycleModel`.*

Continued on next page

ID	User Story	Tasks
		• *Update `ErpMachineOrdersService` to consume the new endpoints.* • *Update `MachineOrdersProvider` to manage its state.* • *Create widget `ProductionCycle`.*
US-3.6	*As an Operator or Supervisor, I need a visual chart of declarations over time to identify performance trends.*	*Flutter:* • *Create widget `ProductionChart`.* • *create a function to ensure that the chart is always centered* • *implemented a scrollable view into the chart*
US-3.7	*As an Operator or Supervisor, I need to see the Bill of Materials for a production operation so that I know which components are required, how much has already been consumed, and the amount available in the inventory.*	*Business Central (AL):* • *Create function `fetchBom()` in codeunit `MESMachineFetch`.* • *Expose `fetchBom()` through codeunit `MESWebService`.* *Flutter:* • *Create `MesComponentConsumption-Service` to consume the `fetchBom()` endpoint.* • *Create `MesComponentConsumption-Provider` to manage BOM stream and refresh state.* • *Create widget `RequiredComponent`.* • *Implement BOM streaming and refresh flow in `MesComponentConsumptionService` and `MesComponentConsumption-Provider`.* • *Implement component status display and status filtering in `RequiredComponent` widget.*
US-3.8	*As an Operator or Supervisor, I need to report scrapped units with a reason code so that waste is accurately tracked.*	*Business Central (AL):* • *Create table `MESOperationScrap`.* • *Create function `declareScrap()` in codeunit `MESMachineWrite`.*

Continued on next page

ID	User Story	Tasks
		• *Expose `declareScrap()` through codeunit `MESWebService`.* • *Create `MESScrapCodeAPI` to fetch scrap codes from the standard `BusinessCentral Scrap Code`.* *Flutter:* • *Create model `MesScrapCode`.* • *Create `MesScrapService` to consume the scrap code fetch and `declareScrap()` endpoints.* • *Create `MesScrapProvider` to manage scrap code and scrap declaration state.* • *Create dialog `DeclareScrapDialog`.*
US-3.9	*As a Supervisor, I need to declare production or scrap on behalf of an operator so that output is attributed to the correct person.*	*Business Central (AL):* • *Add field `DeclaredBy` to the tables `MESOperationProgression` and `MESOperationScan`.* • *Created the function `fetchMesUsersByWC` to retrieve users with the Operator role assigned in the current user's work center.* *Flutter:* • *Create widget `OperatorSelector`.* • *Update `DeclareProductionDialog` and `DeclareScrapDialog` to support supervisor declaration on behalf of an operator.* • *Updated the `MesUserService` and provider to consume the new endpoints.* • *Connect `OperatorSelector` to `MesUserProvider` and pass `onBehalfOfUserId` to `MachineOrdersProvider.declareProduction()`.*
US-3.10	*As an Operator or Supervisor, I need to scan components into an operation so that consumption is recorded.*	*Business Central (AL):* • *Create table `MESComponentScan`.* • *Create the function `insertScans()` in codeunit `MESMachineWrite`.*

Continued on next page

ID	User Story	Tasks
		• *Create the functions* `resolveBarcode()` *in codeunit* `MESMachineFetch`. • *Expose the three function through codeunit* `MESWebService`. • *Create page* `MESComponentConsumption`. *Flutter:* • *Create model* `ItemBarcodeModel`. • *Create* `MesBarcodeService` *to consume the endpoints* `insertScans()`, *and* `resolveBarcode()`. • *Create* `MesBarcodeProvider` *to manage barcode state and endpoint calls.* • *Create widget* `ScannerWidget`. • *Update widget* `RequiredComponent` *to integrate item scanning, search, filtering, and BOM refresh after scan submission.* • *Implement ondevice camera scanning via* `mobile_scanner` *in* `ScannerWidget`
US-3.11	*As an Operator or Supervisor, I need to print a label for the finished production operations (100% progress) so that produced parts are correctly identified.*	*Flutter:* • *Create page* `PrintLabelPage`. • *Update* `ActionButtonsContainer` *to integrate the Print Label action.* • *Implement label generation, preview, print flow, and orientation toggle in* `PrintLabelPage`.
US-3.12	*As an Operator or Supervisor, I need a label printed for each declared unit so that individual pieces carry traceable information.*	*Flutter:* • *Create page* `DeclarationLabelPage`. • *Implement declaration label generation, preview, orientation toggle, and per-unit PDF export in* `DeclarationLabelPage`.

5.2 Design

5.2.1 Use Case Diagram

This sprint adds production execution actors and interactions centred on the Operator and Supervisor roles. The primary use cases are: Declare Production, Pause Operation, Resume Operation, Finish Operation, Interrupt Operation, Cancel Operation, View BOM Components, View Operation Detail, View declaration History. The Supervisor role is a superset of Operator: both roles can declare production, pause, resume, and view detail, only the Supervisor may close Finish, Interrupt or Cancel a production order in the intended business flow. The resulting use case diagram is shown in Figures 5.2 and 5.1.

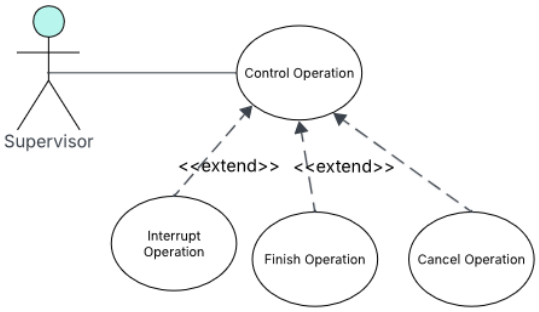

Figure 5.1. UML Use Case Diagram — Sprint 3 (control operations)

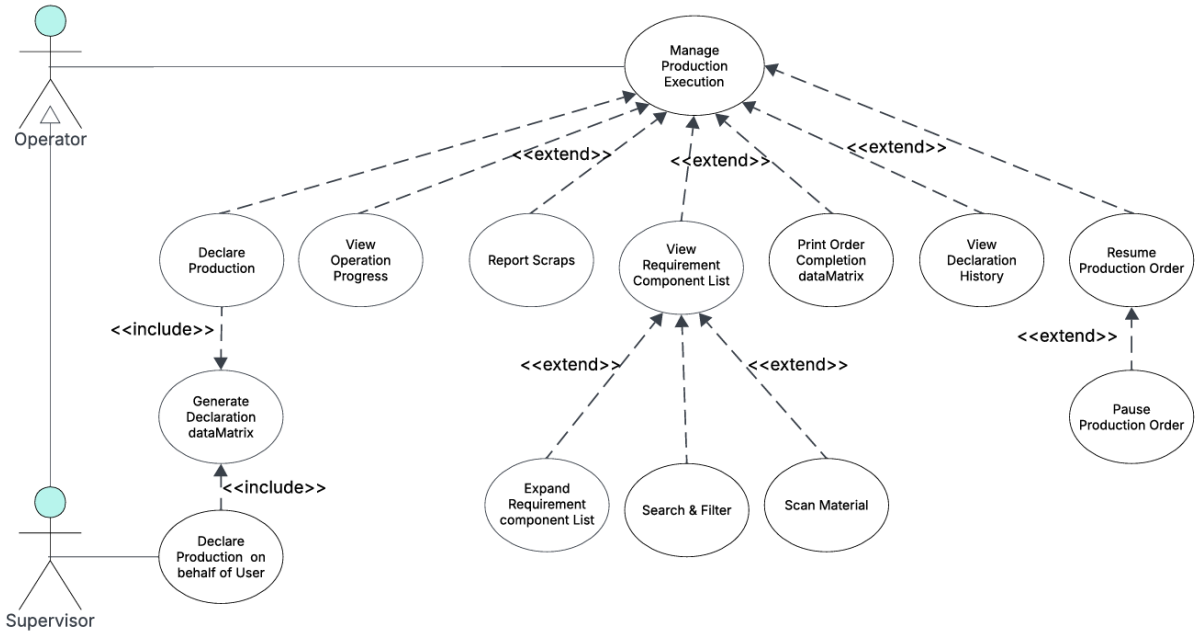

Figure 5.2. UML Use Case Diagram — Sprint 3 (Production Execution)

5.2.2 Textual Use Case Description

Table 5.2. Textual Use Case Description — Declare Production

Use Case Name	*Declare Production*

Continued on next page

Primary Actor	*Operator / Supervisor*
Preconditions	• *The user is authenticated and navigated to an Operation Detail page.* • *The operation status is* Running. • *The quantity produced is below the requested quantity.*
Postconditions	• *A new* `MESOperationProgression` *record is inserted.* • *The production chart and cycle table update via the refresh stream or trigger.* • *The progress percentage and produced quantity in the info container update.*
Main Scenario	1. *The user taps Declare Production in the Quick Actions panel.* 2. *The* `DeclareProductionDialog` *opens showing the remaining quantity.* 3. *The user (if Supervisor) optionally selects an operator from the dropdown to declare on behalf of.* 4. *The user must enter a positive integer less than or equal to the remaining requested quantity, and the remaining scanned component quantity must be sufficient to support the entered production quantity.* 5. *The user taps Submit.* 6. *The ERP validates the quantity and inserts the progression record.* 7. *The user is redirected to the declaration label print page* 8. *The refresh stream triggers an update.*
Alternative Flow	• *Step 3, invalid quantity: client-side validation shows an inline error; submission is blocked.* • *Step 6, ERP error: the server error message is displayed in the dialog; the record is not inserted.*

Table 5.3. Textual Use Case Description — Finish / Cancel / Interrupt Operation

Use Case Name	*Finish / Cancel / Interrupt Operation*
Primary Actor	*Supervisor*
Preconditions	• *The user is authenticated with an active session as a Supervisor.* • *The operation is in* Running, Paused, *or* Released and unstarted *state.*

Continued on next page

Postconditions	• *A* Finished, Interrupted, *or* Cancelled *status record is inserted.* • *The execution end time is stamped.* • *An Idle machine status record is inserted if the operation was started.*
Main Scenario	1. *The user clickes in the (Interrupt, Finish, Close) button based on the operation state.* 2. *A confirmation dialog is displayed; the user confirms.* 3. *The system POSTs to* `/MESWebService_finishOperation` *or* `cancelOperation`*.* 4. *The ERP validates, inserts the status record, stamps the end time, and inserts an Idle machine status.*

Table 5.4. Textual Use Case Description — Scan Component Consumption

Use Case Name	*Scan Component Consumption*
Primary Actor	*Operator / Supervisor*
Preconditions	• *The user is authenticated and holds a valid session token.* • *The operation is running.* • *The user is on the Operation Detail page with the* `ScannerWidget` *open.*
Postconditions	• *One* `MESOperationScan` *record is inserted per scanned item line.* • *One* `ItemJournalLine` *(consumption) is posted to BC inventory per scanned item line.* • *The* `RequiredComponent` *widget refreshes to reflect the updated consumed quantities and inventory.*
Main Scenario	1. *The user taps Scan Item and points the camera or scanner at a component barcode.* 2. *Flutter POSTs to* `/api/resolveBarcode`*; the middleware forwards the call to* `MESWebService_resolveBarcode` *in AL.* 3. *AL resolves the item: if the barcode carries an* `ItemNumber:` *prefix, AL parses the item number directly and retrieves the item record from BC; otherwise AL resolves the identifier via the* `ItemIdentifier` *table.*

Continued on next page

	4. *AL fetches the unit-of-measure conversion factor via `ItemUnitofMeasure.Get` and returns `itemNo`, `itemDescription`, `unitOfMeasure`, and `quantityPerUnitOfMeasure` to Flutter.* 5. *Flutter validates the response and adds the item to the local scanned-items list (or increments its quantity if already present); the item appears in the scan list.* 6. *The user repeats steps 1–5 for each additional component, then taps Confirm Scans.* 7. *Flutter POSTs the full scan list to `/api/insertScans`; the middleware forwards to `MESWebService_insertScans` in AL.* 8. *AL for each scan inserts an `MESOperationScan` record and an `ItemJournalLine`.* 9. *AL posts all journal lines to BC inventory in a single batch; BC confirms success.* 10. *Flutter pops the `ScannerWidget` and triggers a data refresh; the `RequiredComponent` widget displays the updated consumed quantities.*
Alternative Flow	• *Step 3, unresolvable barcode: AL returns an error; Flutter displays an inline error message and does not add the item to the scan list.* • *Step 5, client validation failure: Flutter blocks the addition and shows an inline error.* • *Step 9, BC posting error: AL returns the server error to Flutter; no consumption or journal records are committed, and the dialog remains open with the error displayed.*

5.2.3 Sequence Diagrams

The following sequence diagrams illustrate the communication flow between the `Flutter` frontend, the `BusinessCentral` OData layer, and the `AL` business logic for the production and operation-lifecycle flows introduced in Sprint 3.

Figures 5.3 and 5.4 present the two supplementary data-retrieval flows that run concurrently alongside the main operation detail view. Both are polling streams that re-emit on demand or when a state-change trigger is received from the `Flutter` client.

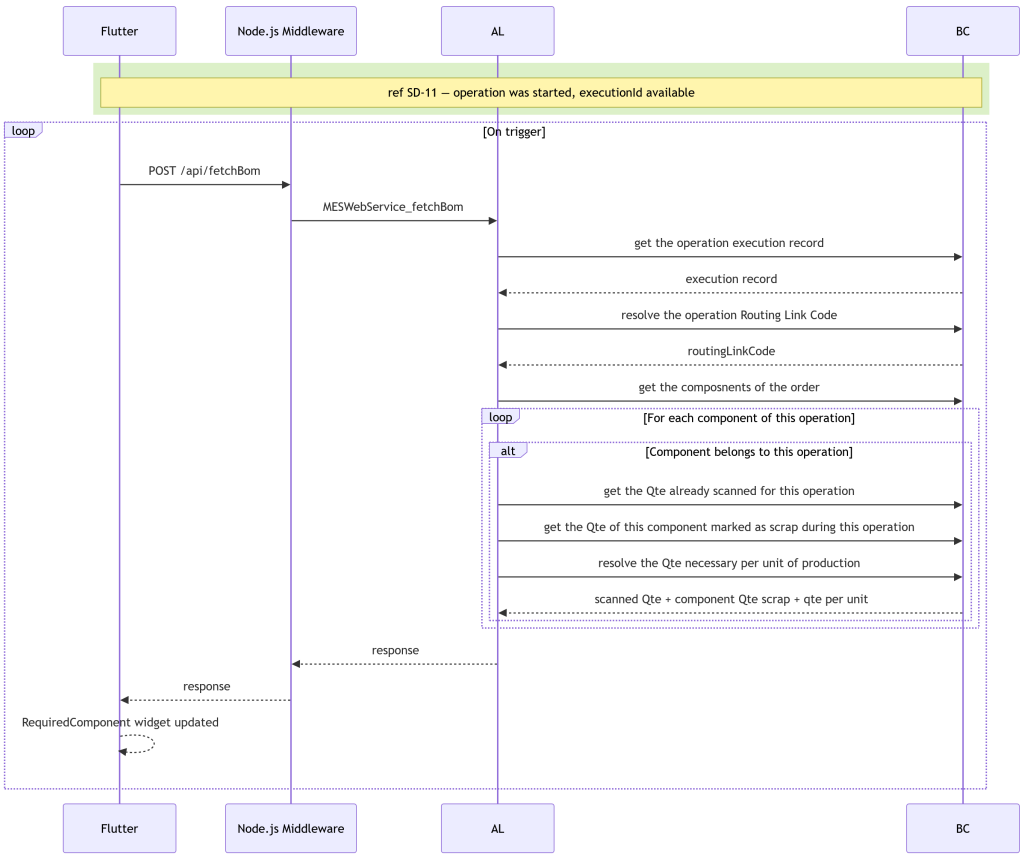

Figure 5.3. SD-21: Fetch Bill of Materials.

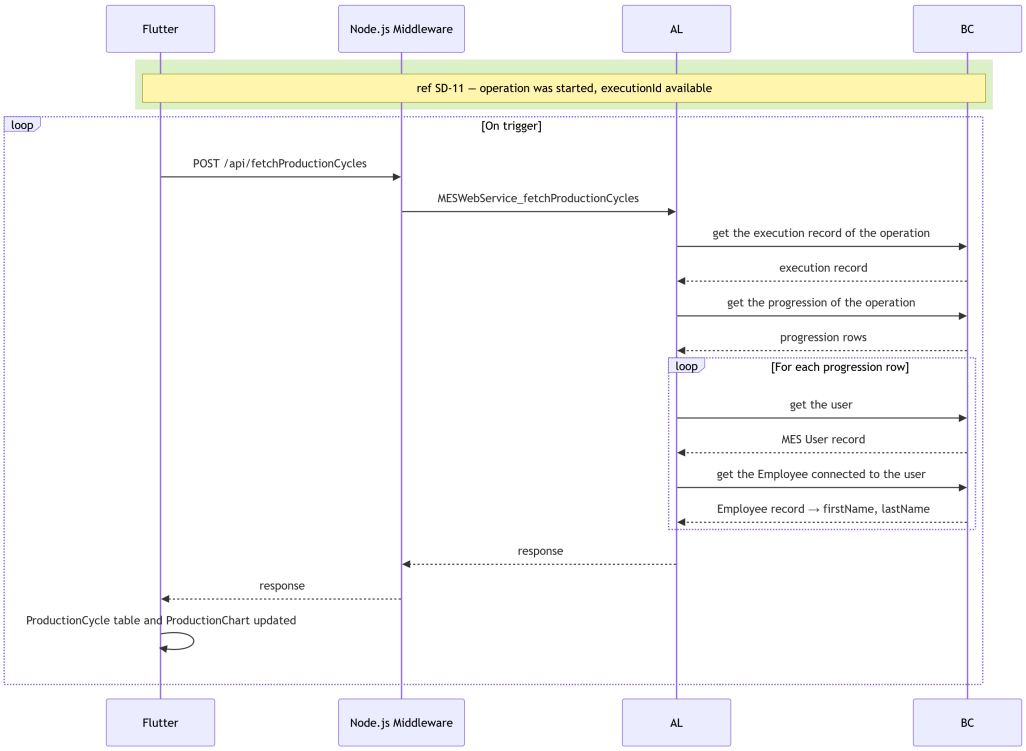

Figure 5.4. SD-22: Fetch Production Cycles.

Figure 5.5 describes the pause flow. The `AL` *layer confirms that the operation is currently in the* Running *state before inserting a* Paused MES Operation State *record and setting the machine status back to* Idle.

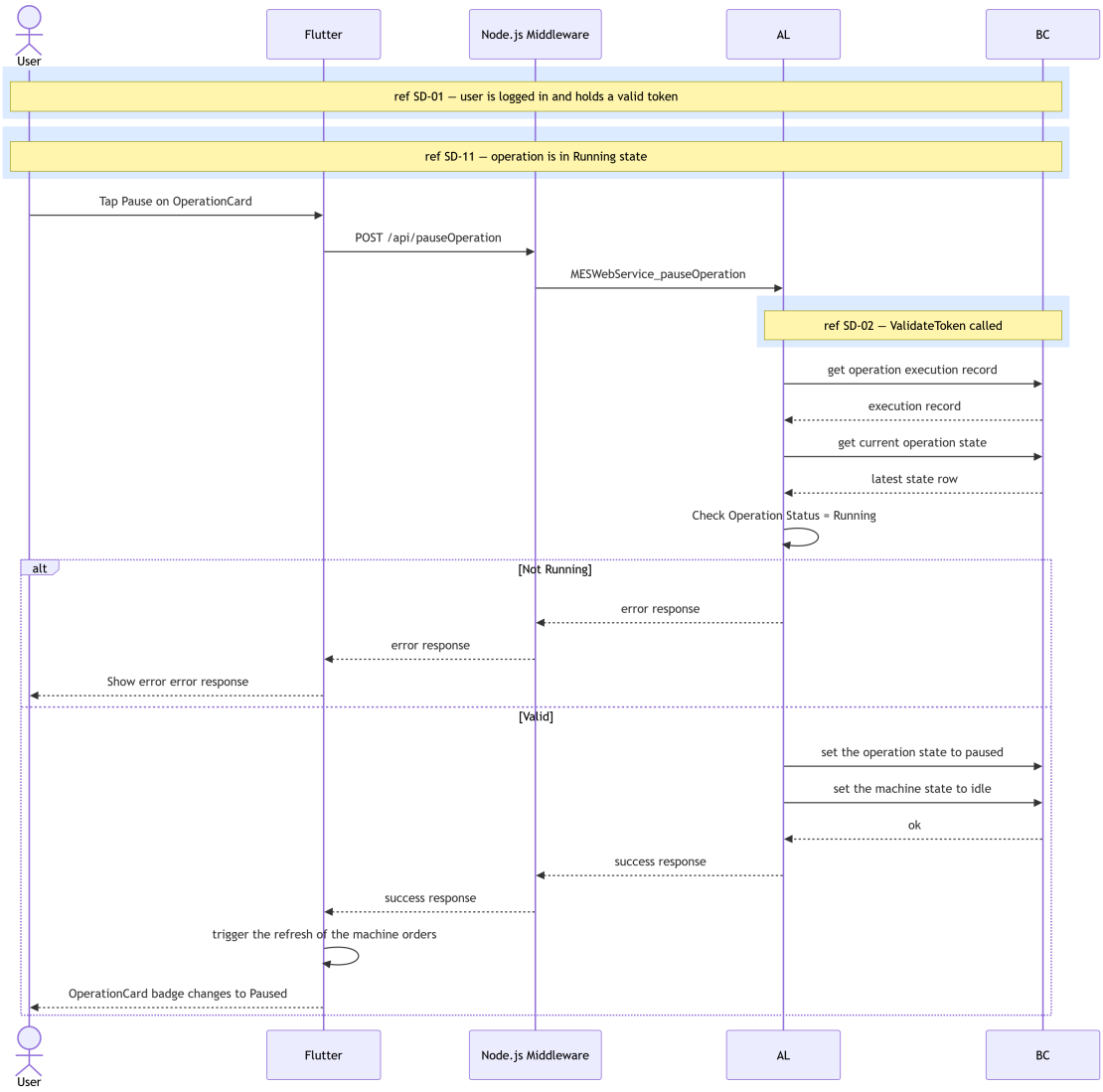

Figure 5.5. SD-13: Pause Operation.

Figure 5.6 describes the scrap declaration flow. Before inserting the scrap record, the `AL` *layer verifies that the operation is running, and that the referenced scrap code exists in* `Business-Central`*. On success, it inserts a* MES Operation Scrap *entry, records user interaction, and appends a new* MES Operation Progression *row so that the scrap quantity is reflected in the live data stream.*

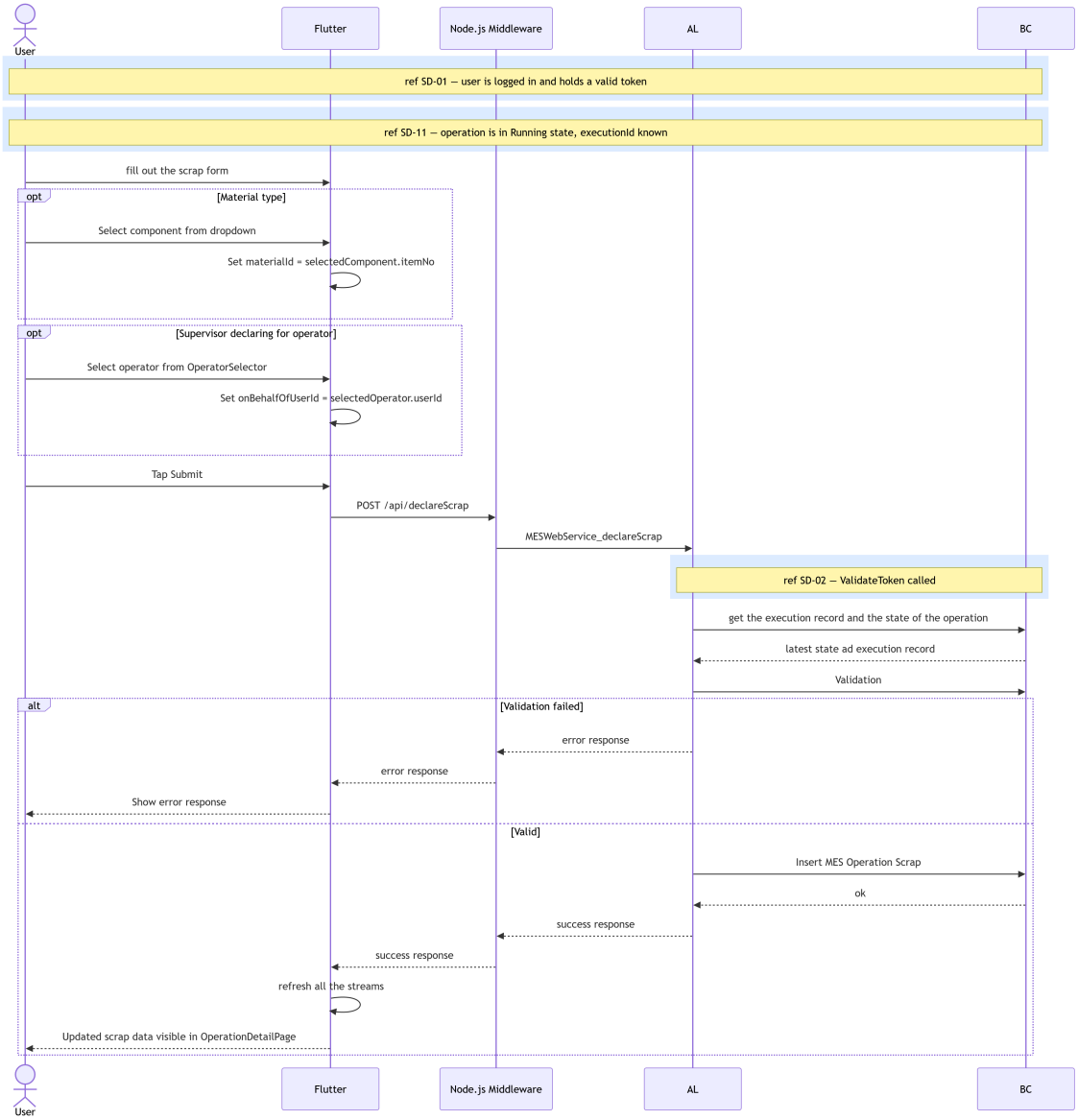

Figure 5.6. SD-16: declare scrap.

Figure 5.7 presents the live operation data retrieval sequence. When the user navigates to the OperationDetailPage, three `StreamBuilder` *instances mount simultaneously: one polling the current operation status and quantities from* `BusinessCentral` *(SD-18), one consuming the production-cycle stream (SD-22), and one consuming the BOM stream (SD-21). The* `Flutter` *client merges all three data sources and renders the full detail view. If the operation is finished, cancelled, or interrupted, the request returns an empty response and the client falls back to the history view.*

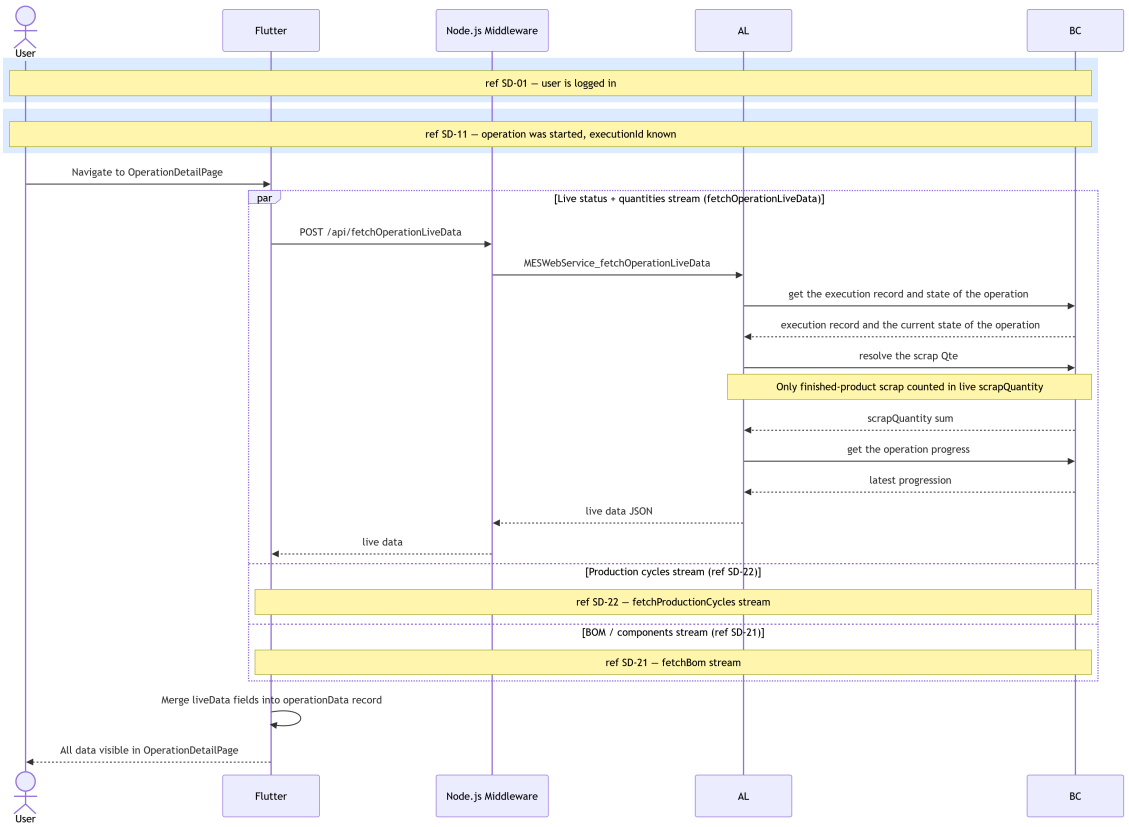

Figure 5.7. SD-18: Fetch Operation Live Data.

5.2.4 Class Diagram

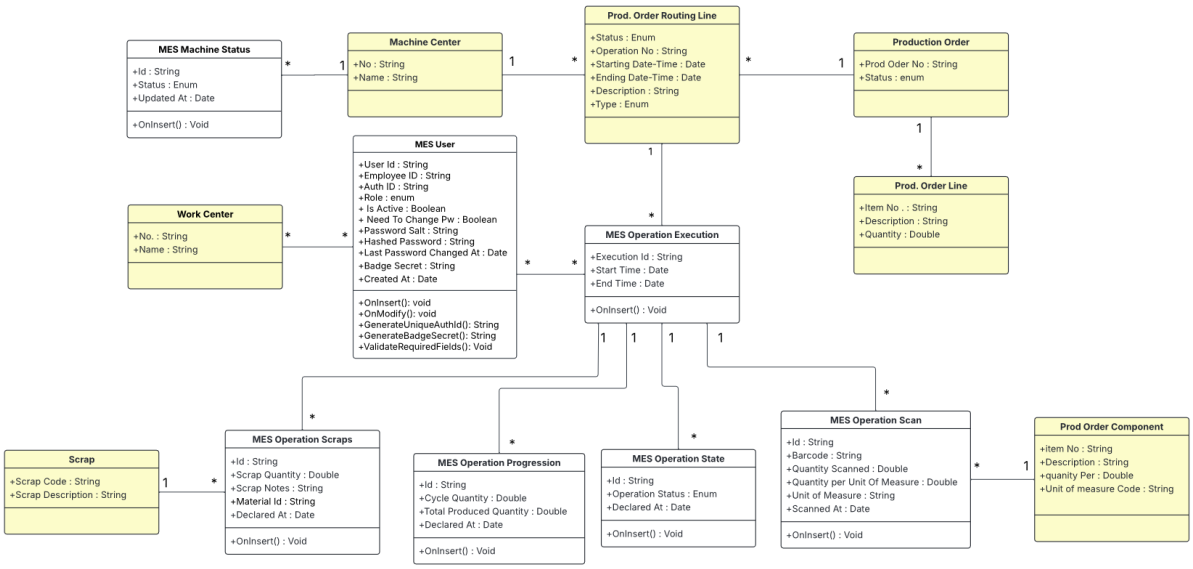

Figure 5.8. UML Class Diagram — Production Execution & Traceability.

The UML Class Diagram (Figure 5.8) extends the Sprint 2 domain model with three new tables. `MES Operation Execution` *(50110) is the central anchor record linking a machine, production order, and operation to its full lifecycle.* `MES Operation Progression` *(50109) is an*

append-only table of production cycle declarations, each referencing an execution and accumulating `TotalProducedQuantity` from the previous cycle. `MES Component Consumption` (50111) records scrap and barcode scan events, also referencing an execution. Both progression and consumption records are child tables of the execution record, forming a hub-and-spoke traceability model.

5.3 Implementation

5.3.1 Backend Implementation

Tables and Entities

Table 5.5. MES Operation Execution (Table 50110) — Field Dictionary

Name	***Type***	***Description***
Execution Id	*Code[50]*	*GUID-based primary key; auto-generated on insert.*
Machine No	*Code[20]*	*References Machine Center (no referential constraint enforced in the current implementation).*
Prod Order No	*Code[20]*	*Foreign key to Prod. Order Routing Line.*
Operation No	*Code[10]*	*Operation identifier from the routing line.*
Item No	*Code[20]*	*Copied from Prod. Order Line on execution creation.*
Item Description	*Text[100]*	*Copied from Prod. Order Line for denormalized display.*
Order Quantity	*Decimal*	*Copied from Prod. Order Line; used for progress calculation.*
Start Time	*DateTime*	*Set by `OnInsert` trigger to `CurrentDateTime()`.*
End Time	*DateTime*	*Stamped when status transitions to Finished or Cancelled.*

Table 5.6. MES Operation Progression (Table 50109) — Field Dictionary

Name	***Type***	***Description***
Id	*Code[50]*	*GUID-based primary key; auto-generated on insert.*
Execution Id	*Code[50]*	*Foreign key to MES Operation Execution.*
Cycle Quantity	*Decimal*	*Units declared in this single production cycle.*

Continued on next page

Name	*Type*	*Description*
Total Produced Quantity	*Decimal*	*Running total: previous total + Cycle Quantity.*
Operator Id	*Code[50]*	*BC session UserId of the user who produced the unites during this cycle.*
Declared At	*DateTime*	*Set by `OnInsert` trigger to `CurrentDateTime()`.*
Declared By	*Code[50]*	*BC session UserId of the user who declared the production cycle*

Table 5.7. MES Operation Scan (Table 50111) — Field Dictionary

Name	*Type*	*Description*
Id	*Code[50]*	*GUID-based primary key; auto-generated on insert.*
Execution Id	*Code[50]*	*Foreign key to MES Operation Execution.*
Prod Order No	*Code[50]*	*Prod. Order Component.*
Item No	*Code[50]*	*Item identifier of the consumed component.*
Barcode	*Text[500]*	*Barcode scanned to register the component.*
Unit of Measure	*Code[50]*	*Unit of measure code from the component definition.*
Quantity Scanned	*Decimal*	*Raw quantity read from the barcode scan.*
Quantity per Unit of Measure	*Decimal*	*Unit-of-measure conversion factor; the effective consumed quantity equals Quantity Scanned multiplied by this value.*
Operator Id	*Code[50]*	*BC session UserId of the operator who scanned.*
Scanned At	*DateTime*	*Set by `OnInsert` trigger to `CurrentDateTime()`.*
Declared By	*Code[50]*	*BC session UserId of the user who declared the production cycle*

Table 5.8. MES User Execution Interaction (Table 50113) — Field Dictionary

Name	*Type*	*Description*
Execution Id	*Code[50]*	*Foreign key to MES Operation Execution.*

Continued on next page

Name	*Type*	*Description*
User Id	*Code[50]*	*BC session UserId of the user who interacted with the execution.*

Table 5.9. MES Operation Scrap (Table 50112) — Field Dictionary

Name	*Type*	*Description*
Id	*Code[50]*	*GUID-based primary key; auto-generated on insert.*
Execution Id	*Code[50]*	*Foreign key to* MES User Execution Interaction*; identifies the operation to which this scrap record belongs.*
Scrap Quantity	*Decimal*	*Number of units scrapped.*
Scrap Code	*Code[10]*	*Scrap reason code sourced from the* `Business-Central` Scrap *table.*
Scrap Description	*Text[100]*	*Description of scrap code.*
Scrap Notes	*Text[256]*	*Optional free-text remarks supplied by the operator to further qualify the scrap event.*
Operator Id	*Code[50]*	*BC session* UserId *of the operator who declared the scrap.*
Material Id	*Code[20]*	*Identifies whether the scrapped material is an input or output component of the operation.*
Declared At	*DateTime*	*Set by* `OnInsert` *trigger to* `CurrentDateTime()`*.*
Declared By	*Code[50]*	*BC session* UserId *of the user who declared the production cycle.*

REST API and Web Services Implementation

All endpoints are exposed as ODataV4 unbound actions through `MESWebService` *(codeunit 50126) and are served by the middleware at:*

`middlewareHostURL/api/<endpointName>`

Each request is forwarded to the `AL` *layer as an ODataV4 action call of the form:*

`POST <baseUrl>/ODataV4/MESWebService_<ProcedureName>?company=<companyId>`

Table 5.10 lists all endpoints introduced in this sprint.

Table 5.10. New API Endpoints — Sprint 3

Endpoint	***Endpoint***
`fetchOngoingOperationsState`	`fetchOperationsHistory`
`declareProduction`	`fetchOperationLiveData`
`pauseOperation`	`fetchProductionCycles`
`resumeOperation`	`fetchBom`
`finishOperation`	`declareScrap`
`cancelOperation`	`insertScans`
`resolveBarcode`	

5.3.2 Frontend Implementation

This sprint introduces a reactive pattern: a shared `StreamController<void>.broadcast()` *in* `MachineOrdersProvider` *and* `MesComponentConsumptionProvider`*. When any write action (declare, pause, resume, finish, cancel) succeeds,* `triggerRefresh()` *adds an event to the controller. All live data streams (*`fetchOperationLiveDataStream`*,* `streamProductionCycles`*,* `streamBom`*,* `streamMachinesOngoingOperationsState`*) are implemented as* `async*` *generators that yield on both the initial fetch and on each event from the shared broadcast stream, enabling coordinated, push-driven UI updates without polling.*

Provider Architecture

Table 5.11. Provider Responsibilities — Sprint 3 Additions

Provider	***Responsibility***
`MachineOrdersProvider`	*Owns write operations (start, declare, pause, resume, interrupt, finish, cancel) and their refresh controller; exposes all live data streams.*
`MesComponent-ConsumptionProvider`	*Exposes* `getBomStream()` *combining service fetch with an independent refresh controller.*

User Interface

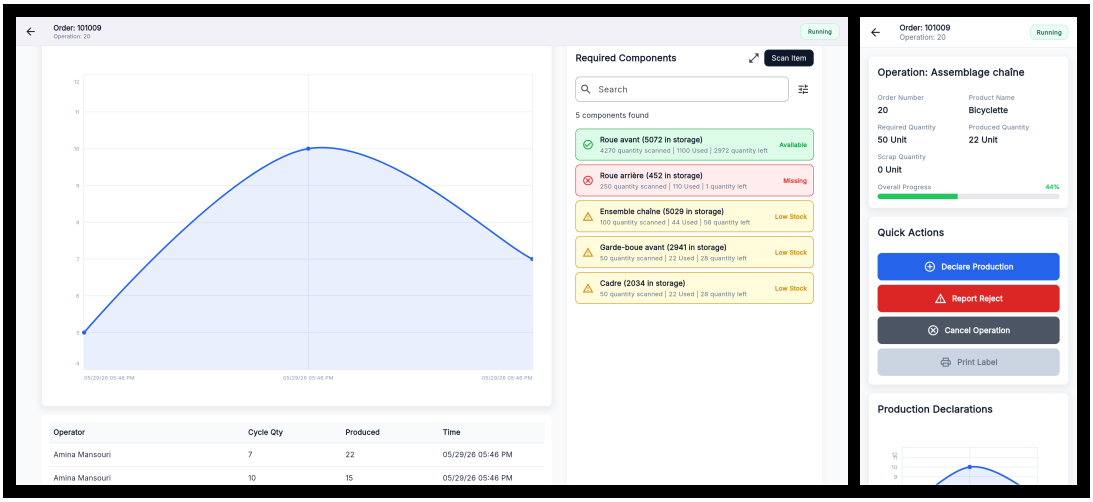

Figure 5.9. *Operation Detail Page* (Running state) — desktop and mobile views.

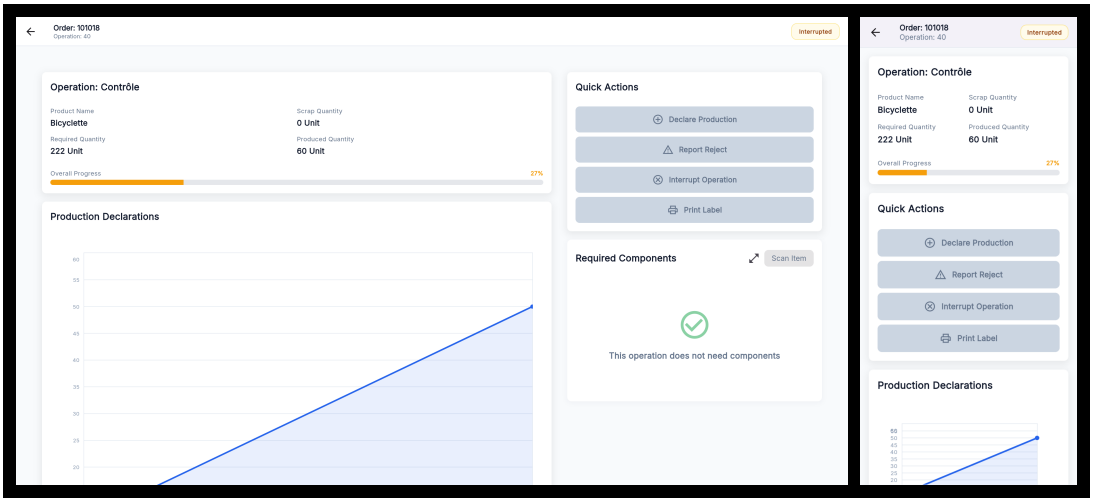

Figure 5.10. *Operation Detail Page* (Interrupted state) — desktop and mobile views.

Figures 5.9, 5.10, and 5.11 illustrate the Operation Detail Page across some of its possible states, as well as the dialog that allows the user to scan components for consumption. Each view of the Operation Detail Page adapts its action buttons and status badge to the current state, while the production chart, cycle table, and BOM list remain consistently visible. On desktop, the page uses a two-column layout with order information and production data on the left, and the Quick Actions panel on the right; on mobile, these sections stack vertically. Action and Scan button availability is state-dependent. All action buttons are disabled when the operation is in the Paused, Finished, Canceled, or Interrupted state. The only exception is the Print Label button, which remains available when the operation is Finished. The scan dialog can be opened from the Operation Detail Page when the operation is in the Running state, by tapping the Scan Item button. The dialog displays a live-updating list of scanned items, and allows the user to confirm or cancel the scan batch.

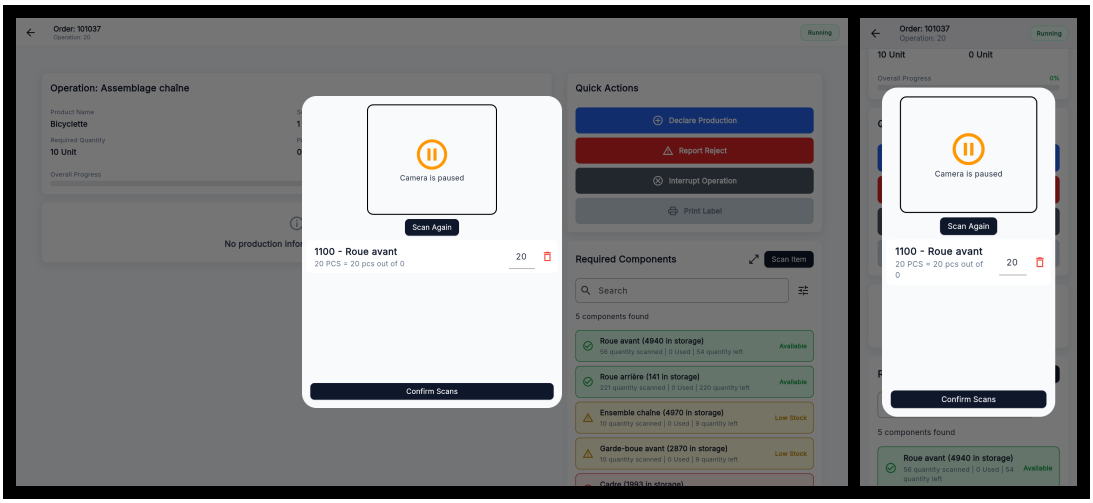

Figure 5.11. *Scan Component Dialog*— desktop and mobile views.

5.4 Test and Validation

System testing

Figures 5.12 and 5.13 illustrate the validation results of the sprint's implemented features, capturing both nominal execution paths and system responses to invalid or unauthorized actions.

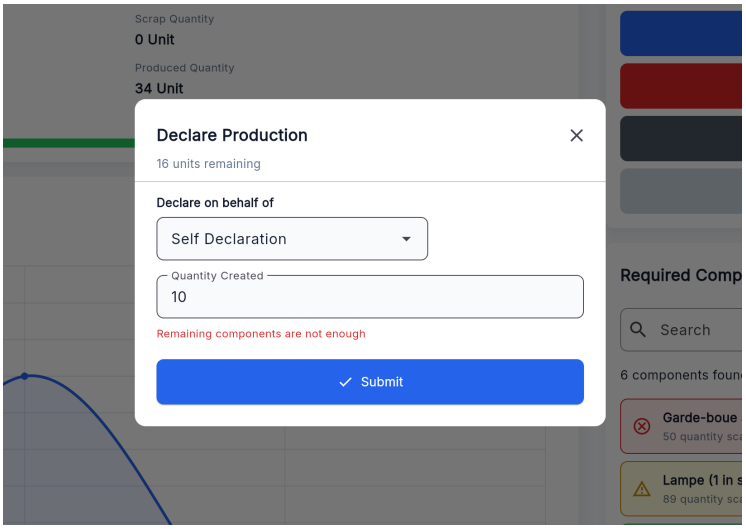

Figure 5.12. Validation dialog — Insufficient Components.

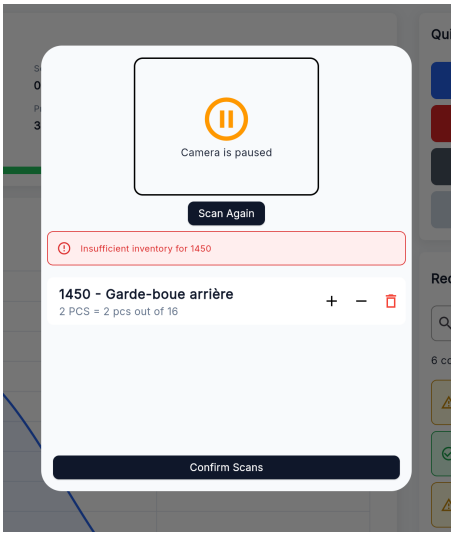

Figure 5.13. Validation dialog — Insufficient Inventory.

Integration test

For integration testing, we chose Postman, a powerful and user-friendly tool for API evaluation. Figure 5.14 shows a test case for the `/fetchBom` *endpoint.*

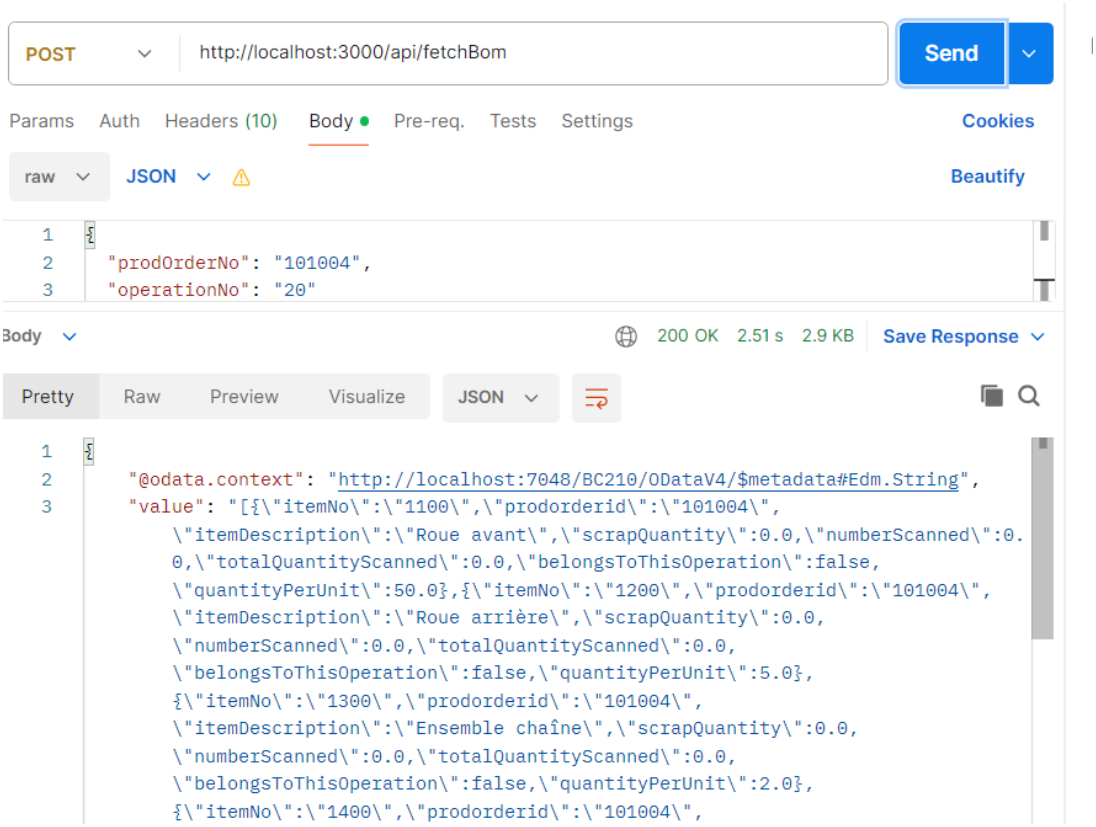

Figure 5.14. Postman endpoint test — fetchBom.

Conclusion

This sprint delivered the complete shop-floor execution loop for the MES system. Operators can now pause, resume, interrupt and close production operations, declare production output cycle by cycle, track cumulative progress through real-time charts and paginated tables, inspect the Bill of Materials, scan physical components via barcodes, declare scrapped units with reason codes, and print traceable labels for both production orders and individual declared units. Supervisors gain the ability to declare production and scrap on behalf of a specific operator, with a full attribution audit trail. The reactive stream architecture ensures that all UI panels update in concert after every write action, providing a consistent and accurate view of the production floor without manual refreshes.

Chapter 6

Sprint 4 Administration & System Monitoring

Contents

Introduction

This sprint delivers the administration layer, multi-language support, and in-app onboarding tutorials that complete the MES *solution. Administrators receives access to an activity log, barcode list, and a user role management interface. Quality-of-life features include full internationalisation with English, French, and Arabic support, automatic right-to-left layout for Arabic, and guided coach-mark tutorials shown once in the main screens on first use. This sprint also delivers machine performance dashboard for the Supervisor. No new* `BusinessCentral` *tables are introduced in this sprint; all data is derived by querying and aggregating the tables created in Sprints 1 through 3.*

6.1 Sprint Backlog

This sprint covers ***Domain 4:*** **Administration & Monitoring**

Table 6.1. Sprint 4 Story Backlog

ID	*User Story*	*Tasks*
US-4.1	*As an Supervisor, I need a performance dashboard for all machines so that I can monitor production health across the facility.*	*Business Central (AL):* • *Create function* `fetchMachineDashboard()` *in codeunit* `MESMachineFetch`. • *Expose the function through the codeunit* `MESWebService`. *Flutter:* • *Create model* `MachineDashboardModel`. • *Create* `LogService` *to consume the* `fetchMachineDashboard` *endpoint and* `LogProvider` *to manage its state.* • *Create page* `MachineDashboardPage`. • *Implement machine dashboard search and time-range filtering in* `MachineDashboardPage`.
US-4.2	*As an Administrator, I need a filterable log of all system actions so that I can audit operator and machine activity.*	*Business Central (AL):* • *Create function* `fetchActivityLog()` *in codeunit* `MESMachineFetch`. • *Add the wrapper of the function to* `MESWebService`. *Flutter:* • *Create model* `ActivityLogModel`. • *Create* `LogService` *to consume the* `fetchActivityLog()` *endpoint*

Continued on next page

ID	*User Story*	*Tasks*
		• *Create* `LogProvider` *to manage its state.* • *Create page* `ActivityLogPage`. • *Implement activity log search, type filtering, time-range filtering, and pagination in* `ActivityLogPage`.
US-4.3	*As any user, I need the application available in English, French, and Arabic so that I can use it in my preferred language.*	*Flutter:* • *Integrate the* `easy_localization` *package.* • *Create translation JSON files for* `en`, `fr`, *and* `ar`. • *Create widget* `LanguageSelector`. • *Implement language selection in* `ProfilePage`. • *Add the* `EasyLocalization` *wrapper in* `main()`. • *Implement right-to-left layout support through* `MaterialApp`.
US-4.4	*As a first-time user, I need guided coach-mark tutorials so that I can learn the interface without external documentation.*	*Flutter:* • *Integrate the* `tutorial_coach_mark` *package.* • *Implement tutorial persistence using* `SharedPreferences`. • *Create* `MachineListTutorial`, `MachineDetailTabsTutorial`, `OperationDetailTutorial`, *and* `AdminDashboardTutorial`. • *Implement tutorial target styling and localisation.*
US-4.4	*As an Administrator, I need a searchable barcode page so that I have a source of the mes barcodes labels.*	*Business Central (AL):* • *Create function* `fetchAllItemBarcodes()` *in codeunit* `MESMachineFetch`. • *Add the wrapper of the function to* `MESWebService`. *Flutter:* • *created the page* `BarcodeListPage.dart`. • *updated the povider mes_barcode_provider.dart and mes_barcode_service.dart to consume the new endpoints.* • *created the widget dataMatrix_card.dart.*

6.2 Design

6.2.1 Use Case Diagram

The UML Use Case Diagram (Figure 6.1) covers the actors and interactions introduced in this sprint. The Administrator gains the interaction View Activity Log and view item barcodes.The Supervisor gains the interaction View Machine Dashboard. All three actors (Operator, Supervisor, Administrator) share the Select Language and View Tutorial use cases, which are cross-cutting accessibility features available on every screen.

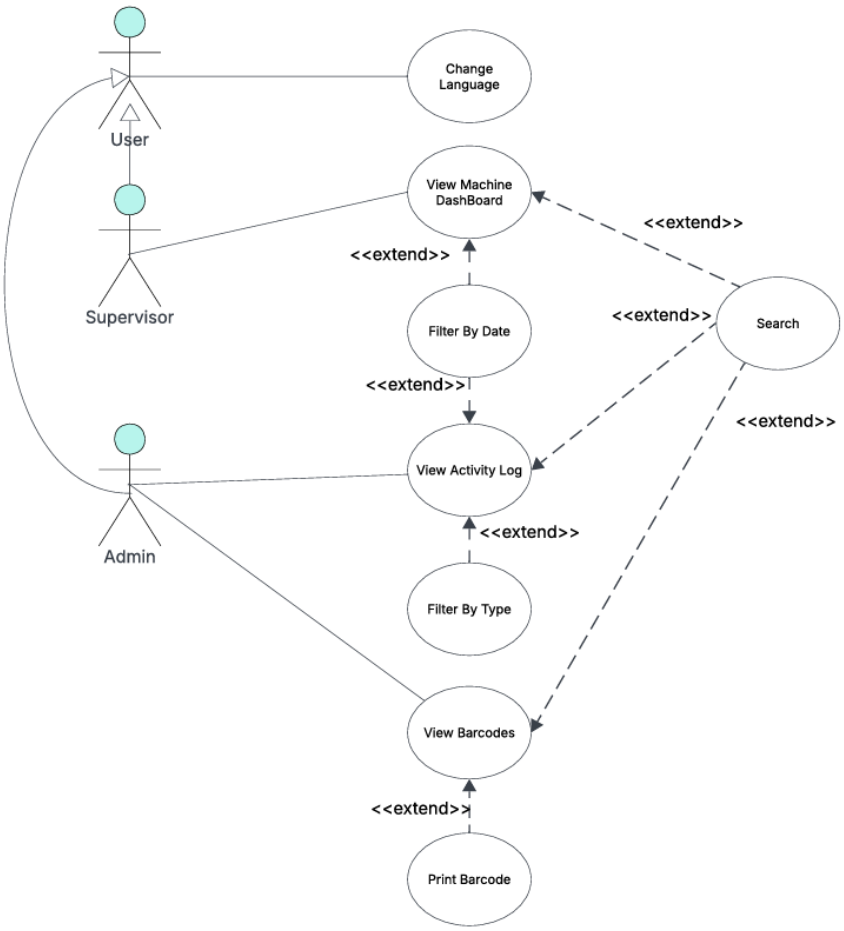

Figure 6.1. UML Use Case Diagram — Administration & UX.

6.2.2 Textual Use Case Description

The following table (Table 6.2) provides the detailed textual description of the View Machine Dashboard use case, which represents the most significant new administrator workflow in this sprint.

Table 6.2. Textual Use Case Description — View Machine Dashboard

Use Case Name	*View Machine Dashboard*
Primary Actor	*Supervisor*
Preconditions	• *The user is authenticated with the* Supervisor *role.*
Postconditions	• *A grid of machine cards is displayed, each showing uptime, production totals, finished operation and concelled/interrupted operations and scrap for the selected time window.* • *Changing the time-range selector refreshes all card metrics.*
Main Scenario	1. *The Supervisor logs in and is routed to* `machineList`*.* 2. *The Supervisor selects Machine Dashboard in the topBar.* 3. *The system POSTs to* `/MESWebService_fetchMachineDashboard` *with the default* `hoursBack=24`*.* 4. *The backend aggregates execution and progression records and returns per-machine metrics.* 5. *The Supervisor changes the time range to 48 h via the AppBar dropdown; the page re-fetches and updates.*

6.2.3 Sequence Diagrams

The following sequence diagrams illustrate the communication flow between the `Flutter` *frontend, the* `BusinessCentral` *OData layer, and the* `AL` *business logic for the administrator analytics flows introduced in Sprint 4.*

Figure 6.2 depicts the machine dashboard retrieval flow. The `AL` *layer iterates over all* Machine Center *records and, for each machine, computes the total finished operation count, cancelled/Interrupted operations, total production and scrap quantities, and uptime percentage from the corresponding* `BusinessCentral` *records before assembling the per-machine KPI rows returned to the* `Flutter` *client.*

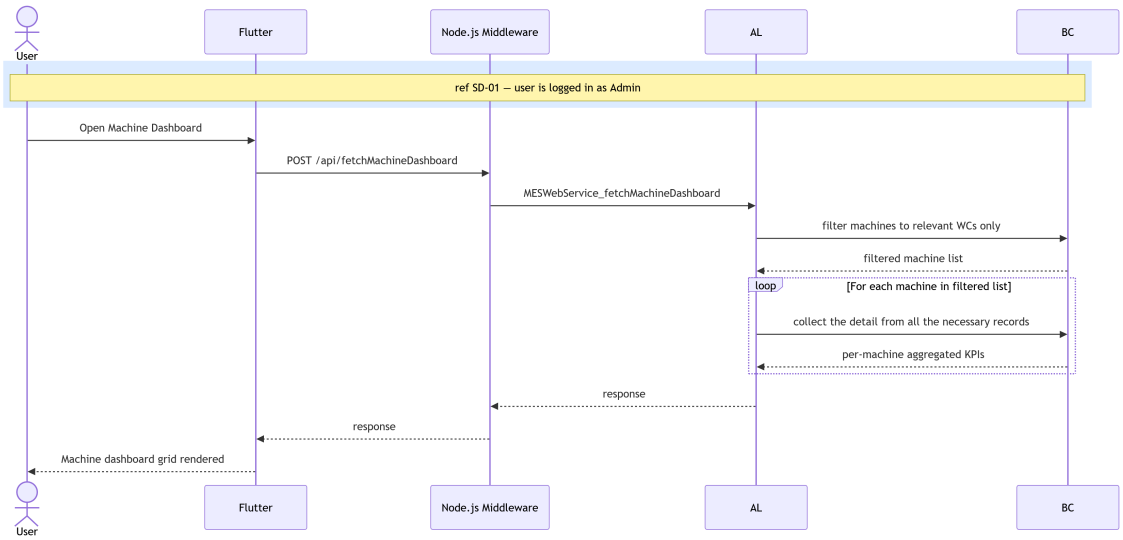

Figure 6.2. SD-19: Fetch Machine Dashboard.

Figure 6.3 presents the activity log retrieval flow. The `AL` *layer queries four event tables in parallel;* MES Operation State*,* MES Operation Progression*,* MES Operation Scrap*, and* MES Operation Scan *, filtering each by the requested time window. For every event row it joins the associated* MES User *and* MES Operation Execution *records to enrich the entry with operator identity and execution context. The resulting flat event list is returned to the* `Flutter` *client, which renders it as a chronological activity table.*

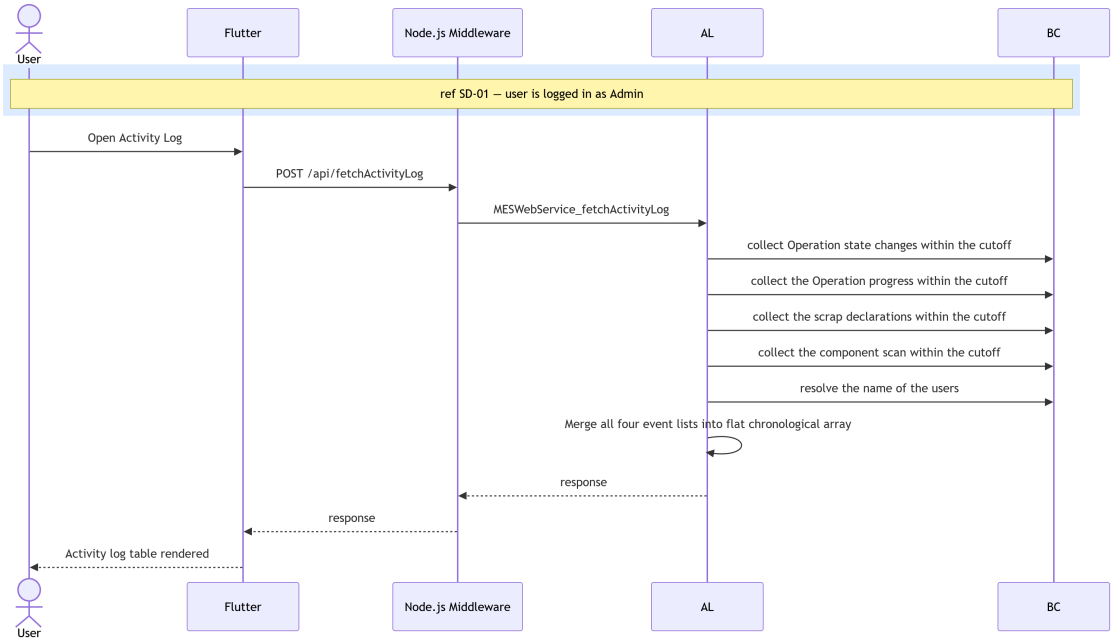

Figure 6.3. SD-20: Fetch Activity Log.

6.2.4 Class Diagram

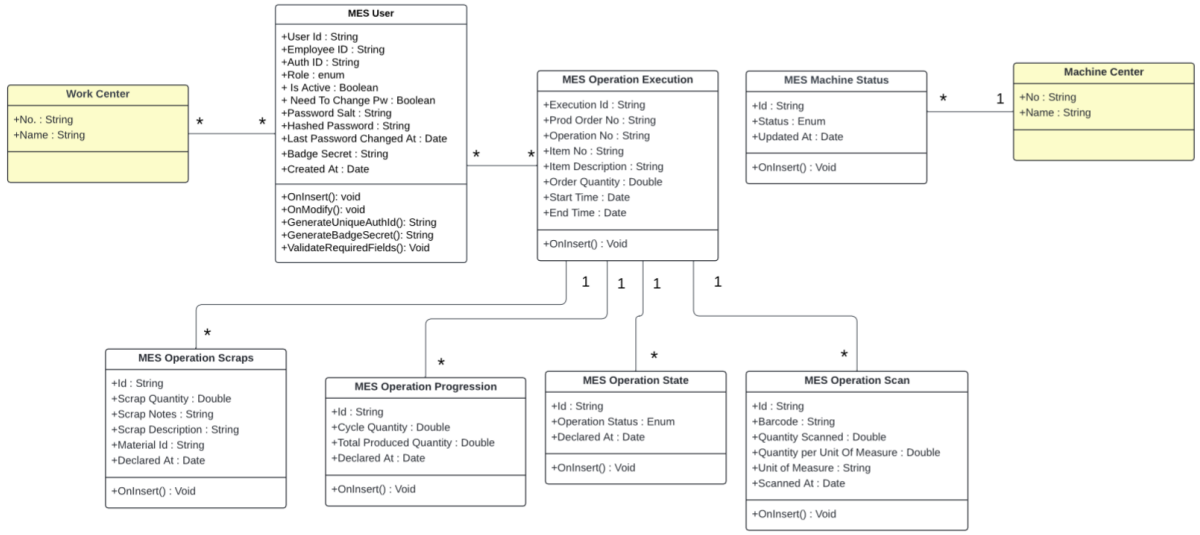

Figure 6.4. UML Class Diagram.

6.3 Implementation

6.3.1 Backend Implementation

No new `BusinessCentral` tables are introduced in this sprint. All backend work consists of new query and aggregation procedures added to `MESWebService` (codeunit 50126), reading from tables defined in Sprints 1 through 3.

Dashboard Aggregation

`fetchMachineDashboard()` accepts a `hoursBack` window and an optional `workCenterNoJson` filter. For each Machine Center, it aggregates execution data, counts finished and cancelled operations, calculates produced and scrap quantities within the selected time window, and computes uptime from machine status records as the percentage of time spent in the Working state.

Activity Log Aggregation

`fetchActivityLog()` queries four tables within the `hoursBack` window `MES Operation State`, `MES Operation Progression`, `MES Operation Scrap`, and `MES Component Consumption` resolves operator names via the `Employee` table, and merges the results into a single descending-timestamp JSON array.

API Endpoints Added in Sprint 4

All endpoints are exposed as ODataV4 unbound actions through `MESWebService` (codeunit 50126) and are served by the middleware at:

`middlewareHostURL/api/<endpointName>`

Each request is forwarded to the AL layer as an ODataV4 action call of the form:

`POST <baseUrl>/ODataV4/MESWebService_<ProcedureName>?company=<companyId>`

Table 6.3. New API Endpoints — Sprint 4

Endpoint	***Endpoint***
`fetchMachineDashboard`	`fetchActivityLog`

6.3.2 Frontend Implementation

Shared HTTP Infrastructure

All sprint services (including retroactively migrated ones) share a `HttpClient` and `HttpResponseParser` layer, replacing the inline `http.post` calls from Sprint 1. `HttpClient.post()` is a centralised wrapper that injects JSON headers and encodes the request body. `HttpResponseParser` exposes four static helpers: `parseList()` and `parseObject()` decode OData list and single-object responses respectively; `parseWriteResult()` decodes write responses and surfaces backend errors; `_assertSuccess()` validates HTTP status codes and throws a structured exception on non-2xx responses.

Provider Architecture

Table 6.4. Provider Responsibilities — Sprint 4 Additions

Provider	***Responsibility***
`LogProvider`	*Owns dashboard and activity log fetch calls; exposes `selectedHours`, `hourOptions`, and `setHours()` shared between the two admin pages.*
`MesBarcodeProvider`	*Exposes `fetchAllBarcodes()`; resolves the session token automatically from `AuthProvider`.*

User Interface

Figures 6.5, 6.7, and 6.6 illustrate the three pages. The Machine Dashboard Page renders one card per machine with a circular uptime indicator and four metric rows (finished operations, Cancelled and Interrupted operations, produced, scrap, uptime), and an AppBar time-range dropdown that re-fetches on change. The Activity Log Page presents a paginated, newest-first event list where each row shows a type icon, operator, declared by, machine, time, order, operation number and action ; AppBar dropdowns filter by type and time range. The Barcode Page presents a searchable list of all items with their associated barcodes rendered as DataMatrix codes.

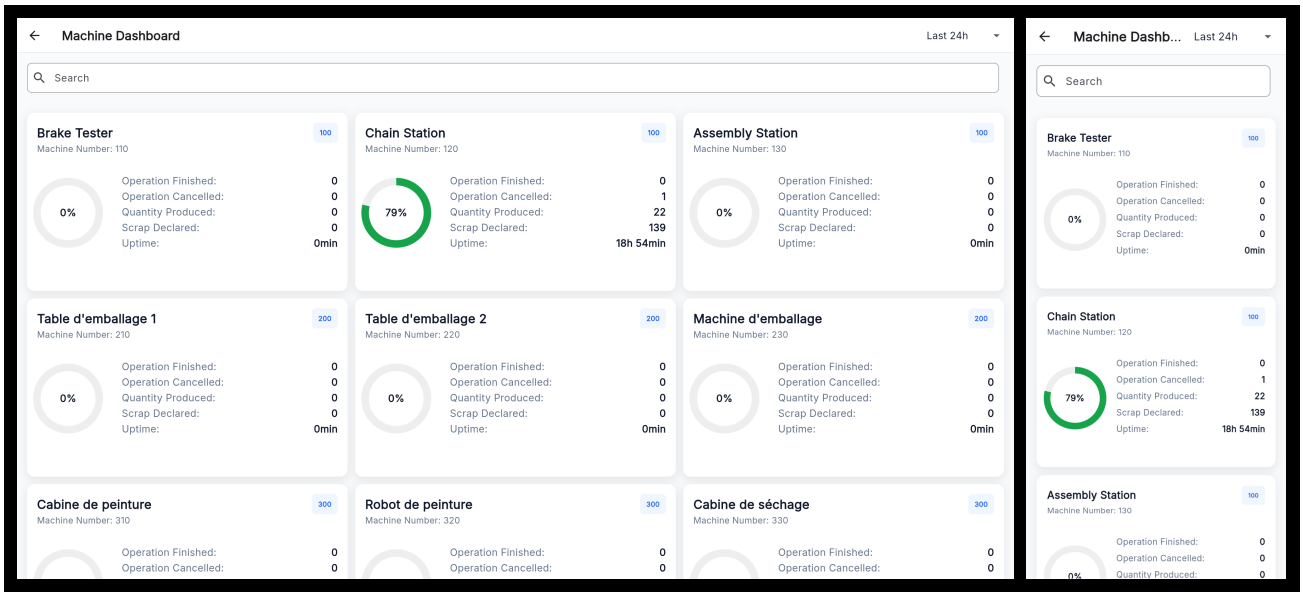

Figure 6.5. *Machine Dashboard Page* — desktop and mobile views.

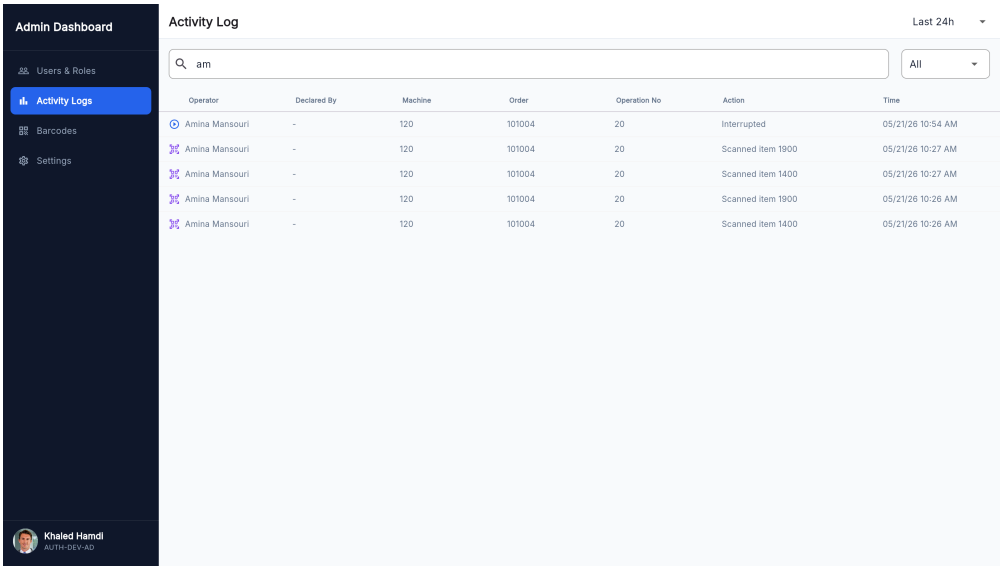

Figure 6.6. *Activity Log Page* — desktop and mobile views.

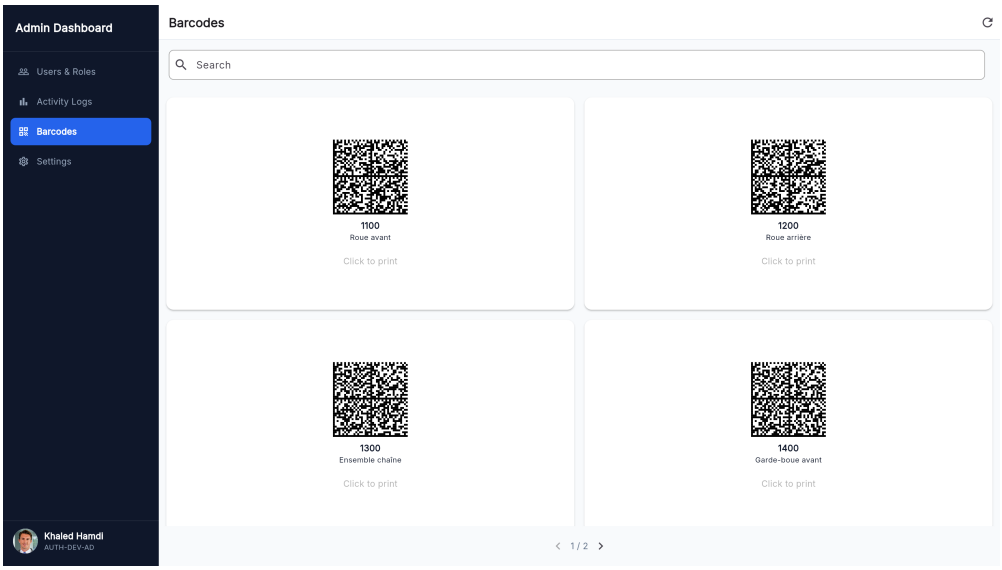

Figure 6.7. *Barcode Page* — desktop and mobile views.

6.4 Test and validation

Integration test

For integration testing, we chose Postman, a powerful and user-friendly tool for API evaluation. Figure 6.8 shows a test case for the `/activityLog` *endpoint.*

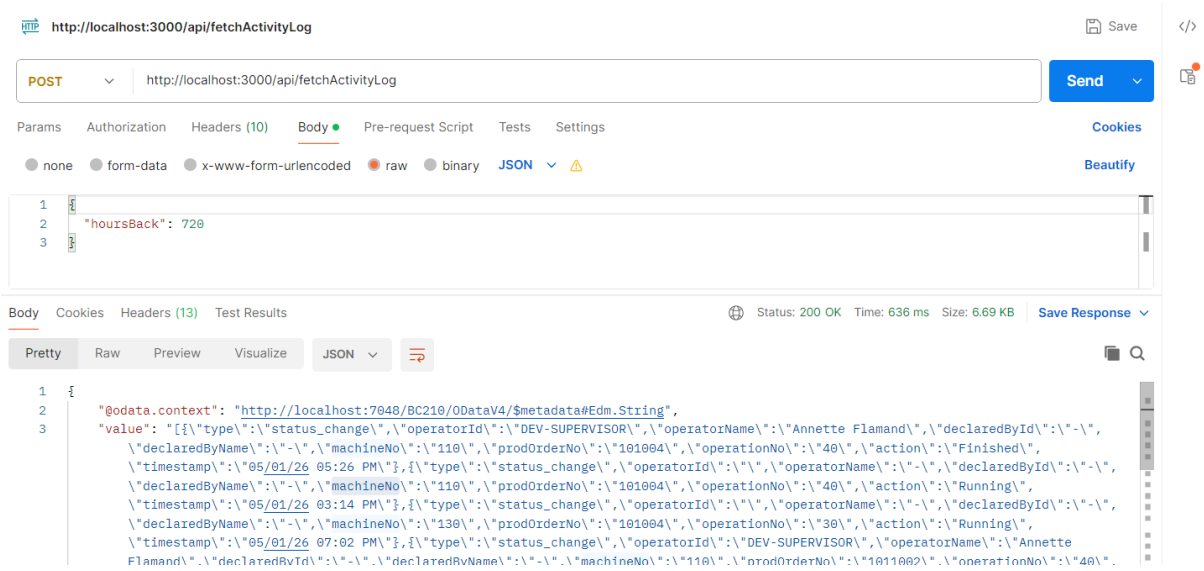

Figure 6.8. Postman endpoint test — fetchActivityLog.

Figure 6.9 shows a test case for the `/MachineDashboard` *endpoint.*

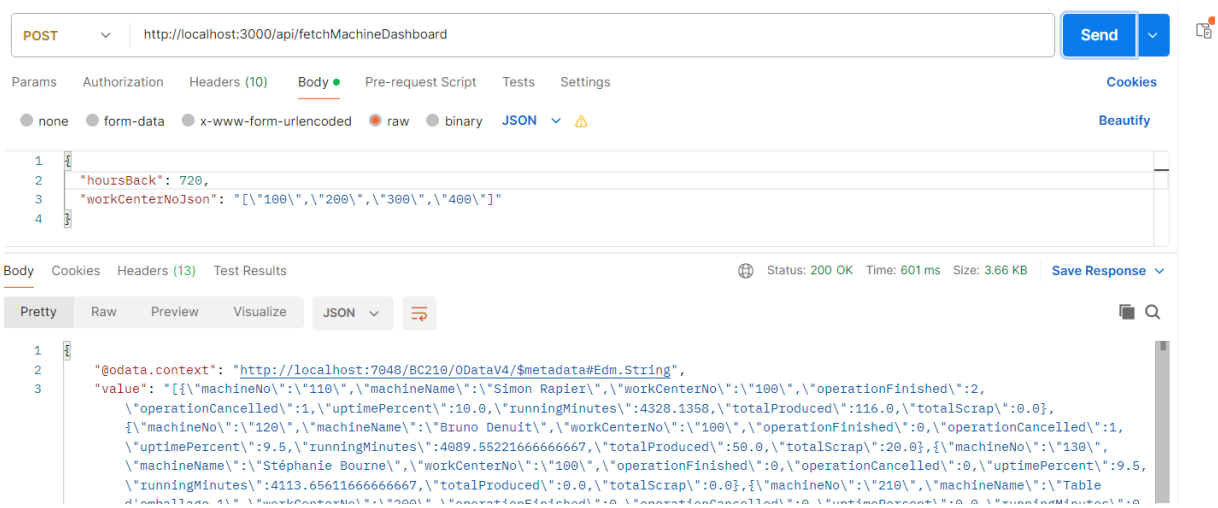

Figure 6.9. Postman endpoint test — fetchMachineDashboard.

Conclusion

This sprint completed the MES *solution by delivering the administration layer and cross-cutting quality-of-life features. Supervisor can now monitor the production health through a configurable machine performance dashboard, and the Administrator can audit all operator and machine actions through a paginated activity log, view the barcodes of all the items, and manage users. All users benefit from full internationalisation covering English, French, and Arabic with automatic right-to-left layout, and from guided coach-mark tutorials that ease onboarding for first-time users.*

Chapter 7

Sprint 5 AI Assistant & Analytical Intelligence.

Contents

Introduction

This sprint delivers the conversational intelligence layer of the MES *solution. Building upon the production execution and administration infrastructure established in the previous sprints, this sprint introduces an embedded natural-language chat assistant that allows operators and supervisors to query machine status, production progress, scrap data, Bill of Materials, and operation history without navigating the full interface. The assistant resolves ambiguous machine references, answers fleet-wide questions through parallel fan-out queries, proactively surfaces operational anomalies, and provides contextual navigation shortcuts directly within the chat reply.*

7.1 Sprint Backlog

This sprint covers ***Domain 5:*** **AI Chat Assistant & Analytical Intelligence**.

Table 7.1. Sprint 5 Story Backlog

ID	*User Story*	*Tasks*
US-5.1	*As an Operator or Supervisor, I need a natural-language chat assistant so that I can query shop floor status without navigating the full UI.*	*Python AI Agent* • *Implement* `FastAPI` *application (*`main.py`*) with* `/health` *and* `POST/chat` *endpoints and lifespan context manager.* • *Implement* `agent/models.py`*:* `ChatRequest`*,* `ChatResponse`*,* `UserContext`*,* `ConversationTurn`*,* `ToolResult`*,* `RedirectAction`*,* `ActionType`*.* • *Implement* `agent/config.py` *with all environment variable constants (LLM provider, model, timeouts, middleware URL).* • *Implement* `agent/llm_client.py`*:* `LLMClient` *abstraction and* `build_llm_client()` *factory supporting Ollama [13], OpenAI [14], OpenAI-compatible, and Anthropic [15] providers.* • *Implement* `tools/mes_tools.py`*: all 17 BC web service tool classes,* `TOOL_MAP` *registry,* `MACHINE_SCOPED_TOOLS` *set,* `_post()` *HTTP helper, and* `_extract_value()` *response normaliser.*

Continued on next page

ID	User Story	Tasks
		• *Implement* `agent/llm_intent.py`: `LLMIntentIdentifier` *with* `_llm_classify()`, `_llm_classify_retry()` *(error-context retry on first failure),* `_validate_plan()`, `_plan_to_tool_chain()`, *and* `ThinkingBlock` *reasoning trace.* *Flutter* • *Create model* `AiChatResponse`, `ConversationTurn`, *and* `AiRedirectAction`. • *Create* `AiChatService` *consuming* `POST/api/ai/chat`. • *Create* `AiChatProvider` *managing conversation history, loading state, and last response.* • *Create page* `AiChatPage` *rendering as side overlay (width* $\geq$ *600 dp) or dialog (mobile).* • *Integrate chat floating action button and overlay toggle in* `Machinelistpage`.
US-5.2	*As an Operator or Supervisor, I need the assistant to answer fleet-wide questions so that I get a consolidated view without visiting each machine individually.*	*Python AI Agent* • *Implement* `agent/composite.py`: `is_composite()` *detection,* `CompositeResult` *dataclass,* `run_composite()` *with status filters (running/idle/overdue/scrap), fan-out capped at 12 machines in batches of 4.* • *Implement* `build_composite_context()` *context builder.* • *Integrate composite path in* `Orchestrator.run()` *and* `Orchestrator._run_composite()`. • *Add* `is_composite` *flag detection to both LLM and regex classifiers.*

Continued on next page

ID	User Story	Tasks
US-5.3	*As an Operator or Supervisor, I need the assistant to provide contextual navigation shortcuts so that I can open a machine, operation, or dashboard directly from the chat response.*	*Python AI Agent* • *Define `ActionType` enum and `RedirectAction` schema in `agent/models.py`.* • *Implement `Orchestrator._parse_actions()` to extract `actions` JSON block from LLM reply (up to 4 actions).* • *Document supported action types and payload shapes in `prompts/system_prompts.py` (`SYNTHESIS_USER_TEMPLATE`).* *Flutter* • *Implement `AiChatPage._handleAction()` routing: `redirect_machine` → `MachineMainPage`, `redirect_machine_list` → `popUntil`, `redirect_operation` → `OperationDetailPage`.* • *Render action buttons only on the last assistant message.*
US-5.4	*As a Supervisor, I need the assistant to proactively highlight anomalies so that I can act before problems escalate.*	*Python AI Agent* • *Integrate enrichment into `Orchestrator._enrich()` for all result key types (machines, orders, live_data, cycles, activity, dashboard, history, bom, etc.).* • *Implement `prompts/system_prompts.py`: `SYNTHESIS_SYSTEM` and `SYNTHESIS_USER_TEMPLATE` with anomaly alert rules and action block instructions.*

Continued on next page

ID	*User Story*	*Tasks*
		• *Implement* `agent/data_analysis.py`: `enrich_deadlines()` *(hours until deadline, overdue flag, risk level),* `analyse_scrap()` *(scrap rate, severity, spike detection at* $\times 2.5$ *baseline),* `analyse_velocity()` *(units per hour, projected end, velocity status),* `summarise_machines()`, `summarise_dashboard()`, `analyse_work_center_summary()` *(fleet utilisation across work centres),* `analyse_operator_summary()` *(active vs idle operator counts, top producers),* `analyse_scrap_summary()` *(top machines/orders/reason codes by scrap quantity), and* `analyse_delay_report()` *(overdue count, abnormal-pause count, worst offenders by delay minutes).* • *Implement* `agent/orchestrator.py`: *full 7-stage pipeline (*`Orchestrator.run()`, `_resolve_machine_ref()`, `_execute_step()`, `_fetch_all_machines()`, `_enrich()`, `_build_context()`, `_synthesise()`, `_parse_actions()`*).* • *Wire all 17 tools to the* `fetchSupervisorOverview` *and* `fetchDelayReport` *BC endpoints used in anomaly detection.*

7.2 Design

7.2.1 Use Case Diagram

The UML Use Case Diagram (Figure 7.1) shows the two primary actors (Operator and Supervisor) and their interactions with the AI chat assistant. The Supervisor role has access to anomaly-oriented queries (overdue orders, scrap spikes, idle operators) in addition to the machine and operation queries shared with Operator. Both roles benefit from the fuzzy machine resolution and redirect action use cases.

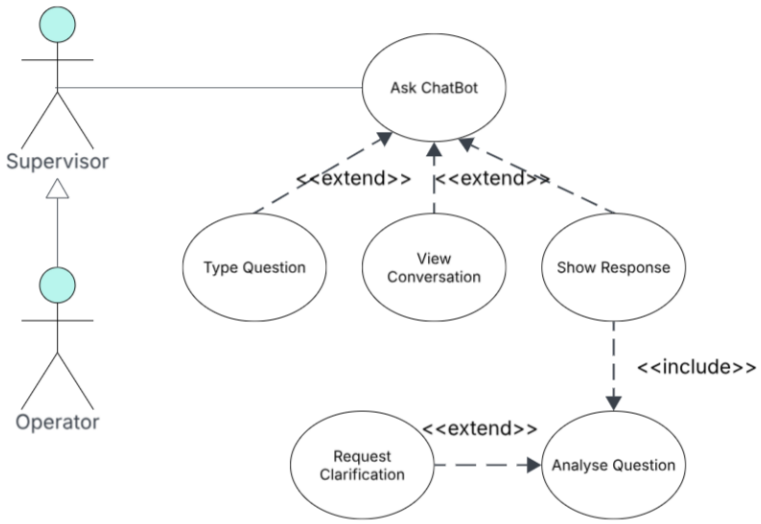

Figure 7.1. UML Use Case Diagram — Sprint 5

7.2.2 Textual Use Case Description

The following table (Table 7.2) provides the detailed textual description of the Ask Question via Chat Assistant use case, which represents the central interaction delivered in this sprint.

Table 7.2. Textual Use Case Description — Ask Question

Use Case Name	*Ask Question via Chat Assistant*
Primary Actors	*Operator, Supervisor*
Preconditions	• *The user is authenticated and holds a valid session token.* • *The* `Node.js` *middleware and* `Python` *AI agent are running and reachable.* • *At least one work centre is assigned to the authenticated user.*
Postconditions	• *A natural-language answer is displayed in the chat panel.* • *Up to 4 contextual navigation action buttons may appear below the reply.* • *The conversation turn is appended to the session history for multi-turn context.*
Main Scenario	1. *The user opens the chat panel from the Machine List Page.* 2. *The user types a question in natural language (e.g., "What is MC-001 producing?").*

Continued on next page

	3. *The* `Flutter` *client sends a* `POST/api/ai/chat` *request to the* `Node.js` *middleware with the message, user context, and conversation history.* 4. *The middleware forwards the request to the* `Python` *AI agent.* 5. *The AI agent classifies intent via the* LLM *classifier and builds a* `ToolChain`*.* 6. *If the machine reference is ambiguous, the agent resolves it using the five-tier fuzzy resolver.* 7. *The agent executes each* `ToolStep` *in sequence, calling the corresponding* `BC` *web service via the* `Node.js` *middleware.* 8. *The agent enriches the raw results with pure-Python analytics (deadline enrichment, scrap analysis, velocity computation).* 9. *The agent synthesises a natural-language answer via a second* LLM *call using the enriched context block.* 10. *Any contextual redirect actions are extracted from the reply and returned as structured* `RedirectAction` *objects.* 11. *The* `Flutter` *client displays the answer and optional action buttons.*
Alternative Flows	• *Ambiguous machine name: the agent returns a disambiguation message listing up to 3 candidates; the user clarifies in the next turn.* • *Fleet-wide query: the agent triggers the composite fan-out path, querying up to 12 machines in parallel batches of 4.* • *General question: no tool calls are made; the* LLM *answers from general manufacturing knowledge.*

7.2.3 Sequence Diagrams

Figure 7.2 illustrates the two-tier intent classification flow, showing the LLM-based primary classifier, the JSON plan validation step, and the fallback path to the regex classifier. The ThinkingBlock *capturing the classification trace is also shown.*

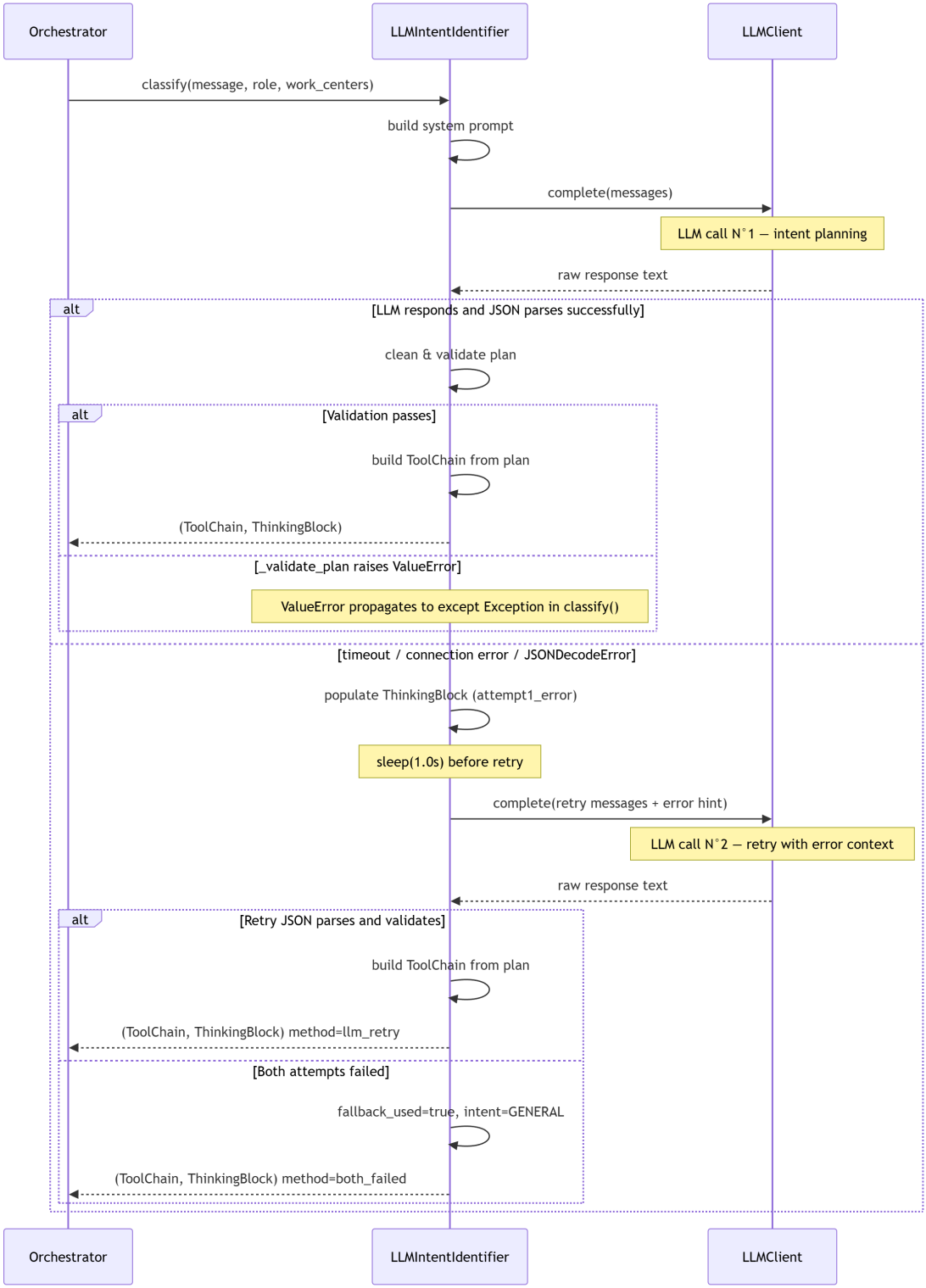

Figure 7.2. SD-31: Intent Classification, LLM with Two-Attempt Retry.

Figure 7.3 illustrates the end-to-end flow for viewing an ongoing operation, showing the Flutter client initiating the request, the Node middleware routing to the AI Module, the sequential ToolChain *execution against Business Central to resolve machines and ongoing operations, and the final LLM synthesis step that produces the* ChatResponse *consumed by the Flutter navigation*

layer.

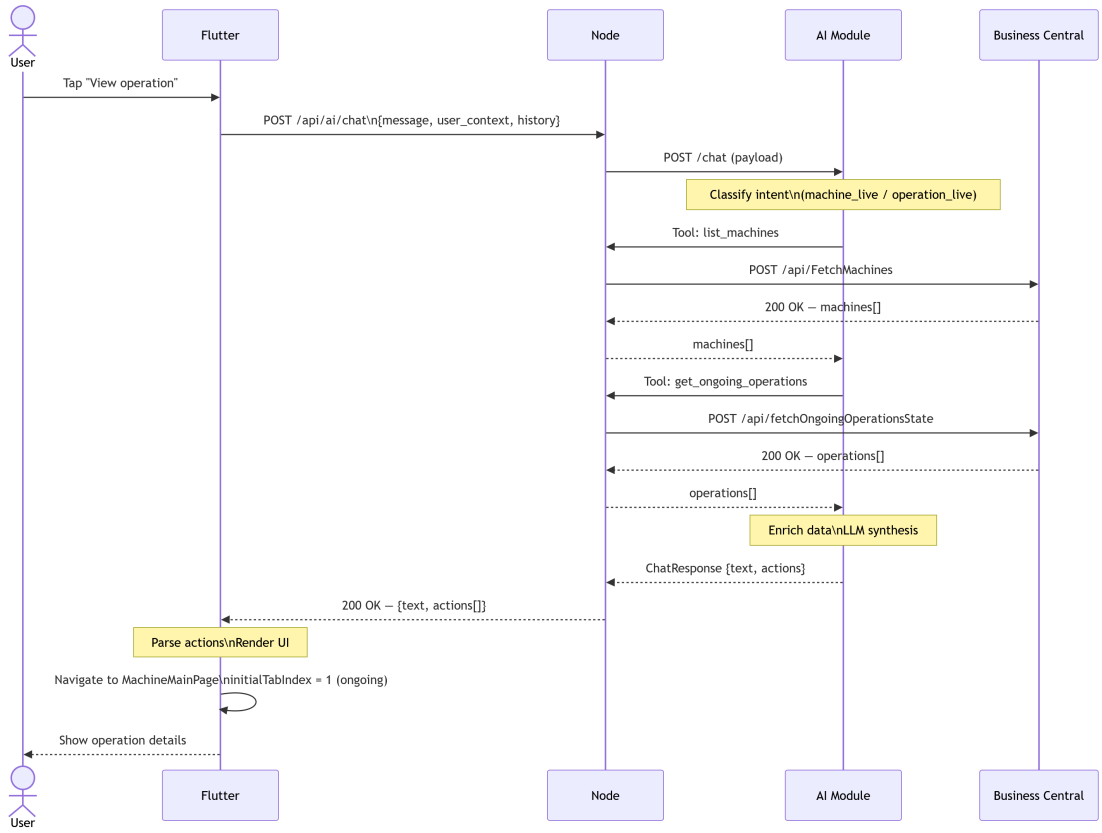

Figure 7.3. SD-31: view chat.

7.2.4 Class Diagram

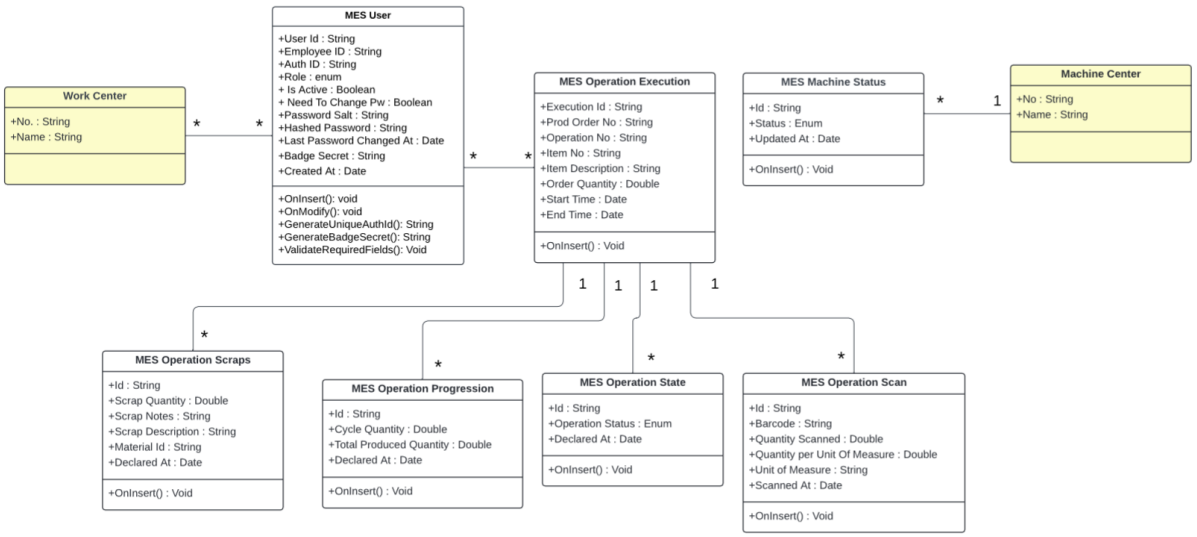

Figure 7.4. UML Class Diagram — AI Assistant & Analytical Intelligence.

7.3 Implementation

7.3.1 Node.js Middleware

Overview

The `Node.js` middleware (`node-middleware/`) is an `Express 5` application that acts as the single network bridge between the `Flutter` client and `BusinessCentral`.

The middleware exposes two distinct route families: `/api/<endpoint>` (proxied to BC) and `/api/ai/*` (forwarded to the Python agent).

AI Proxy Layer

The `src/aiProxy.js` `Express` Router, mounted at `/api/ai`, forwards `POST/chat` requests to the Python agent at `AI_AGENT_URL/chat` using the native `fetch` API. It validates that both `message` and `user_context` are present before forwarding, enforces a `AI_TIMEOUT_MS` abort timeout, and passes the Python agent's response through unchanged. A `GET/api/ai/health` endpoint aggregates the Node process uptime with the Python agent's own health status for monitoring purposes.

Configuration

Table 7.3 lists the environment variables consumed by the middleware.

Table 7.3. Node.js Middleware — Environment Variables

Variable	***Default***	***Description***
`BC_HOST`	—	*Base URL of the BC server (no trailing slash)*
`BC_INSTANCE`	—	*BC server instance name*
`BC_COMPANY`	—	*Company GUID*
`PROXY_PORT`	*3000*	*Listening port*
`ALLOWED_ORIGINS`	*””*	*Comma-separated CORS origins*
`BC_TIMEOUT_MS`	*30 000*	*BC request timeout in milliseconds*
`AI_AGENT_URL`	*port 3000*	*Python agent base URL*
`AI_TIMEOUT_MS`	*60 000*	*AI request abort timeout in milliseconds*

7.3.2 Python AI Agent

Figure 7.5 shows the file architecture of the AI assistant layer. The agent makes up to three LLM *calls per user message; one or two calls for intent classification (a second call is triggered automatically if the first response is malformed), and one call for synthesis.*

Call 1 — Intent classification. *The* `LLMIntentIdentifier` *following a ReAct-style reasoning pattern, sends the user message, role, and work centres to the LLM with a structured system prompt listing all 17 available tools and their required arguments. The LLM responds with a JSON plan containing the intent label, a list of* `ToolStep` *objects (each with a tool name, arguments which may be slot references to previous steps and a result key), and a reasoning string. The plan is validated against the tool catalogue before being converted to a* `ToolChain` *dataclass. The entire classification reasoning is stored in a* `ThinkingBlock` *and returned in* `ChatResponse.thinking` *for debugging; it is never shown to the user.*

Call 2 — Synthesis. *After all tool steps have been executed and their results enriched, the* `Orchestrator._synthesise()` *method builds a context block containing the enriched data in markdown format (truncated at* `LLM_MAX_CONTEXT_CHARS` *characters) and calls the LLM with the* `SYNTHESIS_SYSTEM` *and* `SYNTHESIS_USER_TEMPLATE` *prompts. The system prompt instructs the model to answer only from the pre-fetched data, to highlight anomalies prominently, to respond in the same language as the user, and to optionally append a* `actions` *JSON block containing up to 4* `RedirectAction` *objects.*

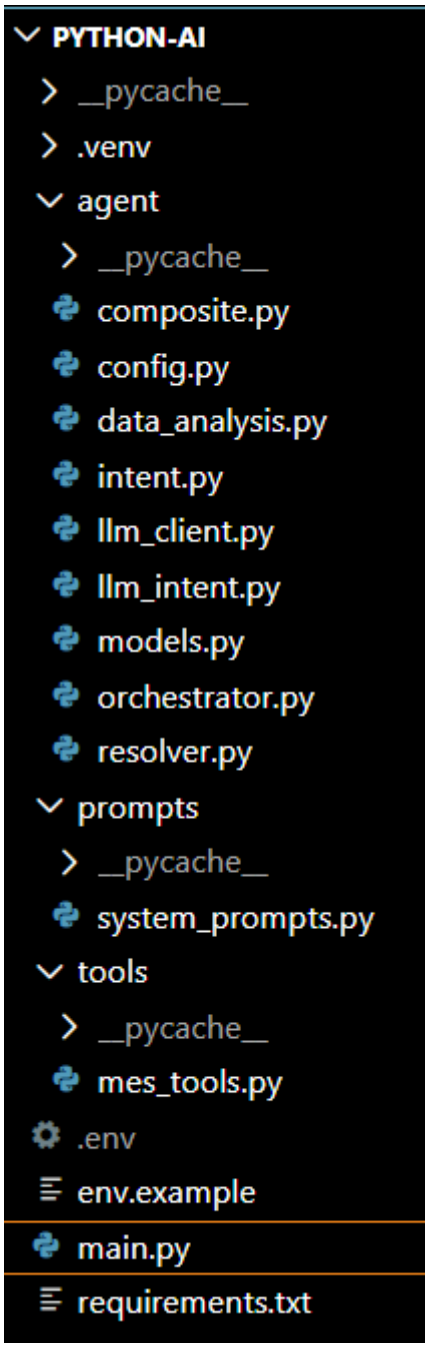

Figure 7.5. AI Assistant Layer

Intent Classification

LLM classifier. *The* `LLMIntentIdentifier` *constructs a system prompt containing the full* `TOOL_CATALOGUE` *and the list of valid intent labels.*

The user message is injected via `_IDENTIFIER_USER_TEMPLATE` *along with the user's role and work centres. The raw LLM response is stripped of JSON fences, parsed, validated against* `_VALID_TOOLS` *and* `_INTENT_MAP`*, and converted to a* `ToolChain` *via* `_plan_to_tool_chain()`*. If the first attempt fails (timeout, malformed JSON, or invalid tool names), the identifier automatically retries with a second* LLM *call that injects the original error reason into the conversation so the model can self-correct. Only after both attempts fail does the identifier return a* `general` *intent with an empty tool chain.*

Machine Name Resolution

When a user references a machine by a non-canonical name or a fuzzy identifier (e.g. "Fraiseuse 3", "milling machine", "mc 1"), the orchestrator invokes the `agent/resolver.py` *module before executing the tool chain. The resolver applies a five-tier cascade against the full machine catalogue fetched from* `BusinessCentral`*:*

1. ***Exact canonical match*** *— the query matches a* `machineNo` *character-for-character (confidence 1.0).*
2. ***Normalised exact match*** *— accents stripped, case-folded, and punctuation normalised before comparison (confidence 0.97).*
3. ***Digit-suffix match*** *— the numeric suffix of the query matches the machine's numeric suffix after stripping leading zeros (confidence 0.88–0.90).*
4. ***Token-overlap score*** *— query tokens are matched against the machine name using a containment-biased formula; short queries that are fully contained in a longer name (e.g. Presse" ⊂ Presse hydraulique") score ≥ 0.80,*
5. ***Editdistance (Levenshtein)*** *— normalised similarity against the machine ID, used as a typo-recovery fallback.*

A match at or above `HIGH_CONFIDENCE` *(0.85) is accepted silently. A match between* `MIN_CONFIDENCE` *(0.45) and* `HIGH_CONFIDENCE` *is accepted but the assumption is surfaced in the* LLM *context block. A match below* `MIN_CONFIDENCE` *is treated as ambiguous: the agent returns a clarification message listing up to three candidates with their work-centre and status, without making any tool calls.*

Table 7.4. Intent Values and Associated Tool Chains

Intent	***Primary Tool(s)***	***Trigger Keywords***
`machine_status`	*get_ongoing_operations*	*machine, mc-N, status*
`machine_orders`	*get_machine_orders*	*order, queue, po-N*
`machine_live`	*get_ongoing_operations + get_operation_live_data*	*live, real-time, progress*
`machine_history`	*get_operations_history*	*history, finished, past*
`machine_dashboard`	*get_ongoing_operations + get_machine_dashboard*	*dashboard, kpi, uptime*
`department_overview`	*list_machines (fan-out)*	*department, all machines*
`activity_log`	*get_activity_log*	*activity, log, happened*

Continued on next page

Intent	*Primary Tool(s)*	*Trigger Keywords*
`scrap_analysis`	*get_scrap_summary*	*scrap, reject, defect*
`operation_bom`	*get_bom*	*bom, component, material*
`production_cycles`	*get_production_cycles*	*cycle, declaration, shift*
`operation_live`	*get_operation_live_data*	*live + order number*
`work_center_summary`	*get_work_center_summary*	*work center, wc summary*
`operator_summary`	*get_operator_summary*	*operator, staff, who is working*
`supervisor_overview`	*get_supervisor_overview*	*overview, shift briefing, floor*
`my_data`	*get_my_data*	*my production, what did I do*
`production_orders`	*get_production_orders*	*list orders, released orders*
`delay_report`	*get_delay_report*	*overdue, delayed, blocked, late*
`consumption_summary`	*get_consumption_summary*	*material usage, BOM variance*
`general`	*(none)*	*No tool trigger matched*

Tool Catalogue

The 17 `BusinessCentral` web service wrappers in `tools/mes_tools.py` correspond directly to the OData functions exposed by `MESWebService` (Codeunit 50126). Table 7.5 lists all tools, their AL endpoint mappings, and access scope.

Table 7.5. AI Agent Tool Catalogue

Tool name	*AL Endpoint*	*Scope*
`list_machines`	*FetchMachines*	*Work-centre scoped*
`get_machine_orders`	*getMachineOrders*	*Machine scoped*
`get_ongoing_operations`	*fetchOngoingOperationsState*	*Machine scoped*
`get_operation_live_data`	*fetchOperationLiveData*	*Machine scoped*
`get_production_cycles`	*fetchProductionCycles*	*Machine scoped*
`get_operations_history`	*fetchOperationsHistory*	*Machine scoped*
`get_activity_log`	*fetchActivityLog*	*Work-centre scoped*
`get_machine_dashboard`	*fetchMachineDashboard*	*Work-centre scoped*

Continued on next page

Tool name	*AL Endpoint*	*Scope*
`get_production_orders`	*fetchProductionOrders*	*Machine scoped*
`get_work_center_summary`	*fetchWorkCenterSummary*	*Work-centre scoped*
`get_operator_summary`	*fetchOperatorSummary*	*Work-centre scoped*
`get_my_data`	*fetchMyData*	*User scoped*
`get_scrap_summary`	*fetchScrapSummary*	*Machine scoped*
`get_bom`	*fetchBom*	*Operation scoped*
`get_delay_report`	*fetchDelayReport*	*Work-centre scoped*
`get_consumption_summary`	*fetchConsumptionSummary*	*Machine scoped*
`get_supervisor_overview`	*fetchSupervisorOverview*	*Work-centre scoped*

Machine-scoped tools are subject to access control: the orchestrator verifies that the resolved `machineNo` *is present in the* `allowed_machines` *map (populated by* `list_machines` *calls for the authenticated user's work centres) before executing the step.*

Data Enrichment

The `Orchestrator._enrich()` *method runs pure-Python analytics over the raw tool results before they are passed to the synthesis LLM. Table 7.6 summarises the enrichment applied to each result key.*

Table 7.6. Data Enrichment Applied per Result Key

Result key	*Enrichment*
`machines`	`summarise_machines()` *; utilisation percentage, working/idle counts, interpretation string*
`orders`	`enrich_deadlines()` *; hours until deadline, overdue flag, risk level (at_risk threshold:* $< 4\,h$*)*
`live_data`	`analyse_scrap()` + `analyse_velocity()` *; scrap severity, spike detection, units per hour, projected end*
`cycles`	`analyse_scrap()` *; scrap trend from cycle history*
`activity`	`analyse_scrap(activity=data)` *; recent scrap event count*
`dashboard`	`summarise_dashboard()` *; fleet totals, overall scrap severity*

Continued on next page

Result key	***Enrichment***
`bom`	*Missing components list (items with zero scanned quantity)*
`ongoing`	*Operation list with count; raw* `operations` *array from* `get_ongoing_operations`
`production_orders`	*Status counts per status value, running operation count, plain-English summary string*
`work_center_summary`	`analyse_work_center_summary()` *; fleet utilisation across work centres, working/idle machine counts*
`operator_summary`	`analyse_operator_summary()` *; active vs logged-in-idle counts, top-3 producers by quantity*
`my_data`	*Plain-English interpretation of personal completed ops, produced qty, scrap qty, running ops count*
`scrap_summary`	`analyse_scrap_summary()` *; top machines, top orders, top scrap reason codes by quantity*
`delay_report`	`analyse_delay_report()` *; overdue count, abnormal-pause count, worst 3 offenders by delay minutes*
`consumption_summary`	*Over-consumed and missing-consumption execution counts*
`supervisor_overview`	*Plain-English summary: stopped machines, idle operators, abnormal pauses, high-scrap ops, delayed ops*

Scrap spike detection. *The* `analyse_scrap()` *function computes the scrap rate for the most recent quarter of declared cycles and compares it to the rate for the earlier three-quarters. If the recent rate exceeds the baseline by a factor of 2.5 or is above the critical threshold of 15%, a spike is flagged and explicitly mentioned in the synthesis prompt.*

Velocity projection. *The* `analyse_velocity()` *function derives the actual production rate (units per hour) from the last 6 declared cycles and compares it to the required rate (remaining quantity divided by time until the planned end). The result is one of ahead, on_pace, or behind, along with a projected completion timestamp.*

Context Serialisation and Budget Management

Before the synthesis LLM *call,* `Orchestrator._build_context()` *converts the enriched result dictionary into a single markdown string that fits within the* `LLM_MAX_CONTEXT_CHARS` *character budget (configurable via environment variable, default 20,000 characters). Each result key is rendered by* `_compress_section()`*, which dispatches to one of 18 type-specific*

format helpers. Instead of passing raw JSON to the LLM*, every helper produces the most token-efficient representation for its data type:*

- ***Tabular data*** *(machines, orders, history, cycles, BOM, dashboard, operators, work centres, delays, production orders, activity log, consumption) → compact markdown tables via* `_md_table()`*, with cell content capped at 38 characters and row counts capped per key (e.g. 20 rows for orders, 30 for cycles).*
- ***Scalar / KPI data*** *(live operation data, scrap summary, supervisor overview, personal data)→ key-value bullet lists with pre-computed analysis interpretations inline.*
- ***Pre-computed interpretation strings*** *(produced by the enrichment functions in* `data_analysis.py`*) are always rendered first in each section in bold, so the* LLM *receives the conclusion before the supporting data.*

Sections are added in order until the character budget is reached. If a section would overflow the budget, `_emergency_compress()` *is called first: it emits only the pre-computed interpretation string for that key, appending (detail omitted — budget). If even the interpretation string does not fit, the section is silently skipped. This guarantees the* LLM *always receives a coherent, complete context rather than a truncated JSON fragment. Table 7.7 lists the 18 format helpers and the markdown form they produce.*

Table 7.7. Context Serialisation — Format Helpers

Helper	***Result key***	***Output form***
`_fmt_machine_table`	`machines`	*Table: MachineNo, Name, Status, WC, CurrentOrder*
`_fmt_orders_table`	`orders`	*Table: OrderNo, Risk, HoursLeft, Item, Qty, Status*
`_fmt_dashboard_table`	`dashboard`	*Table: Machine, Uptime%, Produced, Scrap, OpsFinished*
`_fmt_ongoing_table`	`ongoing`	*Table: OrderNo, OpNo, Status, Progress, Produced, OrderQty*
`_fmt_history_table`	`history`	*Table: OrderNo, OpNo, Status, Start, End (capped 20 rows)*
`_fmt_cycles_table`	`cycles`	*Table: Time, Operator, CycleQty, TotalProduced (capped 30)*
`_fmt_bom_table`	`bom`	*Table: Item, Description, Required, Scanned, Status*

Continued on next page

Helper	*Result key*	*Output form*
_fmt_live_kv	*live_data*	*Key-value list: status, progress, scrap, velocity*
_fmt_wc_table	*work_center_summary*	*Table: WC, Name, Working/Total, RunningOps, Produced, Scrap*
_fmt_operator_table	*operator_summary*	*Table: UserId, Name, LoggedIn, Machine, Produced, Scrap*
_fmt_scrap_summary_kv	*scrap_summary*	*Key-value list: total, top machines, top reason codes*
_fmt_delay_table	*delay_report*	*Table: OrderNo, OpNo, Machine, Overdue, LongPause, Delay*
_fmt_consumption_table	*consumption_summary*	*Table: OrderNo, Item, Planned, Consumed, Status (capped 25)*
_fmt_supervisor_kv	*supervisor_overview*	*Key-value list: counts + stopped machines + abnormal pauses*
_fmt_production_orders_table	*production_orders*	*Table: OrderNo, Status, Item, Qty, Progress, DueDate, Running*
_fmt_activity_table	*activity*	*Table: Time, Type, Machine, Operator, Action (capped 30)*
_fmt_my_data_kv	*my_data*	*Key-value list: user stats + last 5 operations*
(fallback)	*unknown keys*	*Raw JSON capped at 600 characters*

7.3.3 Flutter Frontend

Chat Page and Provider

The `AiChatPage` widget adapts its rendering to the available screen width: on screens wider than 600 dp it is displayed as a `Positioned` overlay panel (400 dp wide, full height) anchored to the right side of the `MachineListPage`; on mobile it opens as a `Dialog`. The chat input field and conversation history list are shared between both modes.

The `AiChatProvider` maintains the in-session conversation history as a `List<ConversationTurn>`. On each `sendMessage()` call it appends the user turn, calls `AiChatService.sendMessage()` with the history (excluding the most recently appended turn, which the server receives as the primary message), appends the assistant reply, and stores the last `AiChat-`

`Response` for action button rendering.

Redirect Actions

After each assistant reply, the `AiChatPage` inspects `lastResponse.actions`. If non-empty and the message is the last in the conversation, up to 4 `ElevatedButton` widgets are rendered below the reply bubble. The `_handleAction()` method routes each action type:

- *`redirect_machine` → `Navigator.push` to `MachineMainPage` with the payload `machineNo`.*
- *`redirect_operation` → `Navigator.push` to `OperationDetailPage` with `machineNo`, `prodOrderNo`, and `operationNo`.*
- *`redirect_machine_list` → `Navigator.popUntil` to the root route (machine list).*
- *`redirect_machine_dashboard` → `Navigator.push` to the admin machine dashboard.*
- *`redirect_history` → `Navigator.push` to `MachineHistoryPage` with `machineNo`.*

User Interface

The AI Chat Panel, shown in Figure 7.6, is accessible from the Machine List Page via a floating action button. On wide screens it opens as a non-modal side panel; on mobile it opens as a full-screen dialog. The panel displays the conversation history, a text input field at the bottom, and optional action buttons below the last assistant message. A Clear button resets the conversation history.

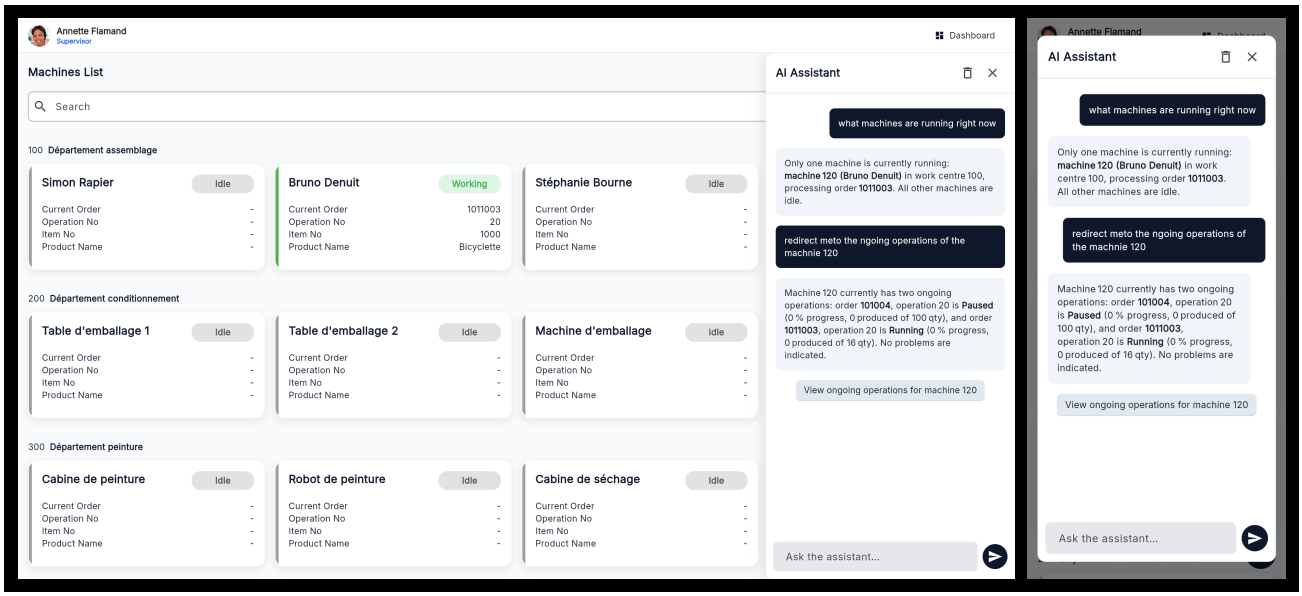

Figure 7.6. *AI Chat Panel* — desktop side panel and mobile dialog views.

7.3.4 Integration Tests

Integration test

For integration testing we used `Postman` for the Node.js middleware endpoints and the `FastAPI` built-in `/docs` interface for the Python agent.

Figure 7.7 shows a test of the `GET /api/ai/health` endpoint confirming that both the Node.js proxy and the Python agent are reachable.

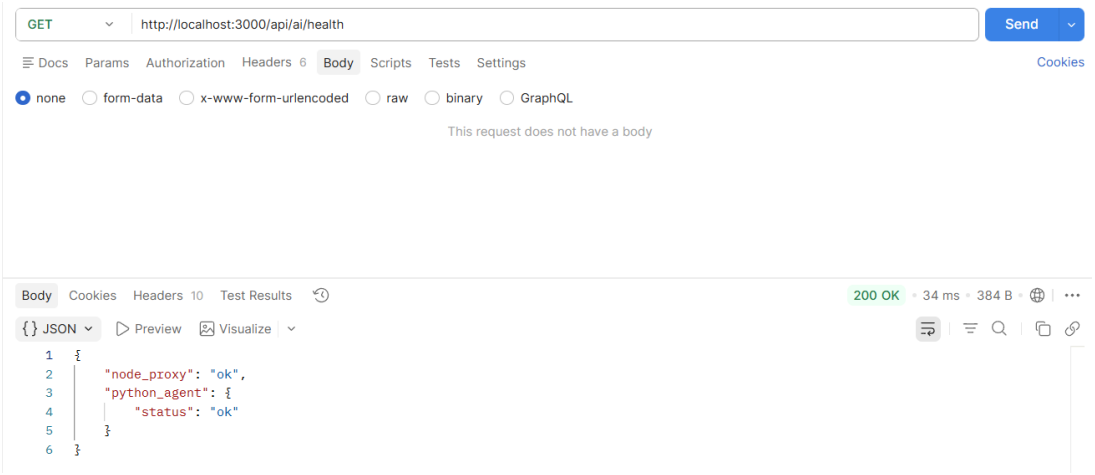

Figure 7.7. Postman endpoint test — `GET/api/ai/health`.

Figure 7.8 shows a test of the `POST /api/ai/chat` endpoint with a single-machine live data query. The response contains the synthesised text, the `thinking` block with the classified intent and tool steps, and one `redirect_machine` action.

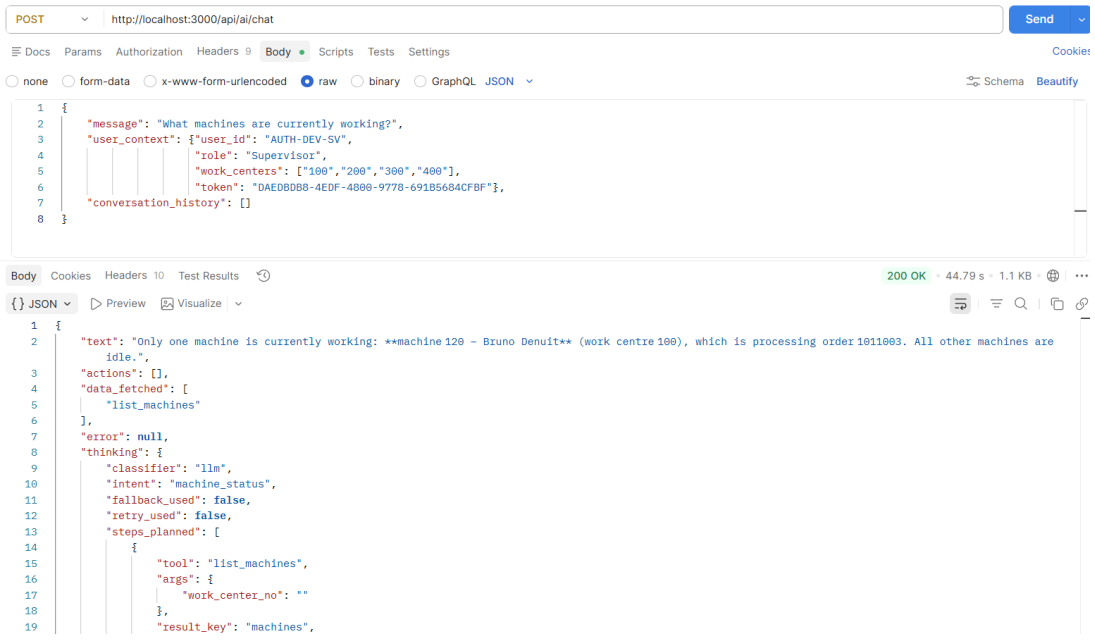

Figure 7.8. Postman endpoint test — `POST/api/ai/chat`.

Conclusion

This sprint delivered the conversational intelligence layer of the MES *solution.*

*The AI agent implements a two-*LLM*-call pipeline: a first call classifies user intent and produces an executable* `ToolChain` *plan , and a second call synthesises the final natural-language reply from the enriched tool results. Pure-Python analytics (deadline enrichment, scrap spike detection, velocity projection) pre-compute key facts so the synthesis* LLM *receives answers rather than raw numbers.*

Operators and supervisors can now ask questions in natural language, receive machine and production data scoped strictly to their work centres, navigate directly to relevant screens via action buttons embedded in the reply, and obtain proactive alerts for overdue orders, scrap anomalies, and paused operations all without leaving the chat interface.

General Conclusion

*This report has presented the full lifecycle of the MES (Manufacturing Execution System) project developed at **MAVISION** over the course of five sprints. Starting from a blank slate, the team designed and implemented a complete shop-floor execution system natively integrated with Microsoft Dynamics 365 Business Central. Sprint 1 established the security and identity foundation: a token-based authentication system, role-based access control, and a full administrator interface for user and account management. Sprint 2 delivered real-time machine monitoring, production order management, and a complete operation history trail. Sprint 3 completed the production execution loop with declarations, pause, resume, finish, and close flows, Bill of Materials visibility, component scanning, scrap tracking, and label printing. Sprint 4 added a machine performance dashboard, a paginated activity log for the Administrator, multi-language support covering English, French, and Arabic, and in-app onboarding tutorials. Sprint 5 delivered the AI intelligence layer: a natural-language chat assistant embedded in the Flutter frontend, a Node.js reverse-proxy middleware bridging the Flutter application to both Business Central and a Python AI agent, and the agent itself, providing intent classification, multi-provider LLM synthesis, fuzzy machine resolution, composite fan-out queries, and contextual redirect actions. Across all five sprints, the append-only data model ensures a tamper-evident audit trail from every machine state change through to every production cycle declaration. The reactive stream architecture in the Flutter frontend provides operators and supervisors with a consistent, real-time view of the production floor without manual refreshes. The project demonstrates that a tightly integrated MES solution can be built on top of Microsoft Dynamics 365 Business Central [10] using OData V4 unbound actions, with a Node.js middleware layer handling Windows authentication and AI routing. The extensible layered architecture positions the system for future enhancements such as statistical reporting, alert management, and additional language support.*

Webography

[1] MAVISION, Official Website, https://www.mavision.tn, visited April 2025.

[2] Dassault Systèmes,DELMIA Apriso — Manufacturing Operations Management, https://www.3ds.com/products/delmia/apriso, visited April 2025.

[3] K. Beck et al., Manifesto for Agile Software Development, 2001. https://agilemanifesto.org, visited April 2025.

[4] K. Schwaber and J. Sutherland, The Scrum Guide, Scrum.org, November 2020. https://scrumguides.org/scrum-guide.html, visited April 2025.

[5] OpenJS Foundation, Node.js — JavaScript Runtime Built on Chrome's V8 Engine, https://nodejs.org, visited April 2025.

[6] OpenJS Foundation, Express 5.x — Fast, Unopinionated, Minimalist Web Framework for Node.js, https://expressjs.com, visited April 2025.

[7] Google LLC, Flutter — Build Apps for Any Screen, https://flutter.dev, visited April 2025.

[8] S. Ramírez, FastAPI — Modern, Fast Web Framework for Building APIs with Python, https://fastapi.tiangolo.com, visited April 2025.

[9] Python Software Foundation, Python Programming Language, https://www.python.org, visited April 2025.

[10] Microsoft Corporation, Microsoft Dynamics 365 Business Central Documentation, https://learn.microsoft.com/en-us/dynamics365/business-central/, visited April 2025.

[11] Google LLC, Dart Programming Language, https://dart.dev, visited April 2025.

[12] Postman Inc., Postman — API Platform for Building and Using APIs, https://www.postman.com, visited April 2025.

[13] Ollama, Ollama — Get Up and Running with Large Language Models Locally, https://ollama.ai, visited April 2025.

[14] OpenAI, OpenAI API Reference, https://platform.openai.com/docs/api-reference, visited April 2025.

[15] Anthropic, Anthropic — AI Safety Company, https://www.anthropic.com, visited April 2025.